

Biologia & JSPlotly

Índice

1. Introdução.....	2
2. Ecologia: populações, interações e fluxo de energia.....	2
Agite antes de usar.....	7
Significado dos parâmetros dos cenários ecológicos.....	9
3. Genética Mendeliana e Molecular: dos padrões de herança à manipulação do DNA.....	12
3.1 Genética mendeliana: probabilidade ao invés de determinismo.....	17
3.2 Genética molecular: o que são genes, afinal?.....	17
3.3 Mutações e variabilidade.....	20
3.4 Biotecnologia: compreender para intervir.....	20
3.5 Pseudocódigo e outro JSPlotly.....	26
4. Origem, Organização, e Evolução da Vida.....	30
Jogo da Vida de Conway.....	45
As regras do sistema.....	46
Como a animação é construída.....	47
5. Vida em rede: Microbiologia, Botânica e Zoologia.....	48
6. Metabolismo e Fisiologia: Energia, Equilíbrio e Vida.....	61
6.1 Estrutura de JS.....	78
7. Reprogramando a Vida: DNA, Genes e Biotecnologia Moderna.....	80
CRISPR.....	90
Aritmética.....	99
Arrays.....	100
Uma palavrinha sobre métodos.....	101
JavaScript, uma linguagem orientada a objetos.....	102
8. Corpo Humano: Integração, Controle e Vida em Funcionamento.....	105
8.0.1 Sistema nervoso e sistema endócrino.....	105
8.0.2 Sistema circulatório.....	108
Separação de comandos, inserção de comentários, e boas práticas.....	125
Declaração de variáveis.....	125
“Botando a mão na massa”.....	125

Tipos de dados.....	127
Constantes.....	127
Funções.....	128

1. Introdução

Introdução

2. Ecologia: populações, interações e fluxo de energia

O MUNDO TE CHAMA



Populações de organismos não crescem de forma contínua em muitos dos *ecossistemas* naturais. Em vez disso, apresentam oscilações ao longo do tempo. Por exemplo, populações de presas, como ratos, podem crescer rapidamente quando há abundância de recursos. No entanto, esse crescimento pode favorecer o aumento de predadores, como serpentes, que passam a se alimentar dessas presas. Como consequência, a população de presas diminui, o que, posteriormente, leva também à redução dos predadores. Além disso, a quantidade de energia disponível em um ecossistema limita o tamanho das populações, sendo transferida de forma decrescente entre os *níveis tróficos*. Esses fenômenos acabam por interligar a dinâmica populacional com relações ecológicas, interações presa-predador e o fluxo de energia e biomassa.

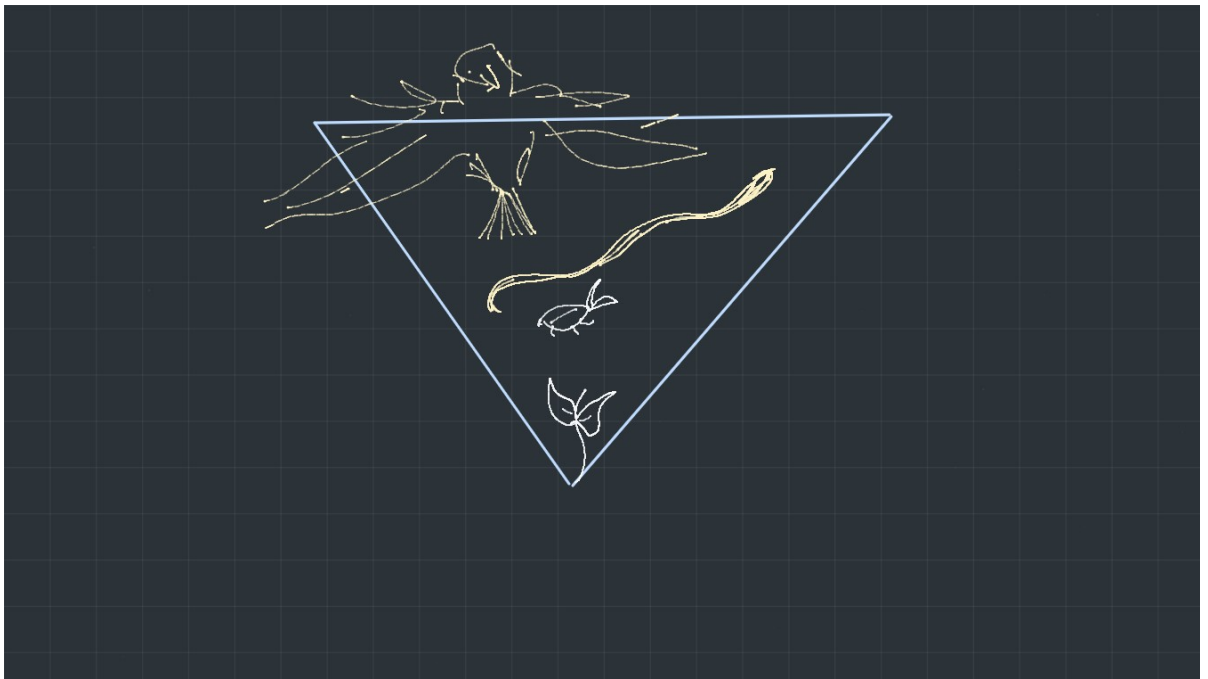


Figura 2.1: Mais presas podem sustentar mais predadores, mas muitos predadores podem reduzir as presas, criando oscilações no sistema ecológico.

Ao final, você será capaz de:

- explicar por que populações naturais oscilam;
- diferenciar tipos de relações ecológicas pelo comportamento dinâmico;
- relacionar fluxo de energia com tamanho populacional;
- interpretar modelos computacionais de sistemas biológicos.

MEXA ANTES DE ENTENDER



Sem preocupar-se com a teoria e os conceitos que envolvem este capítulo, “divirta-se” um pouco usando o aplicativo JSPlotly personalizado para a relação presa-predador a seguir. Sem pressa...curtindo o momento !

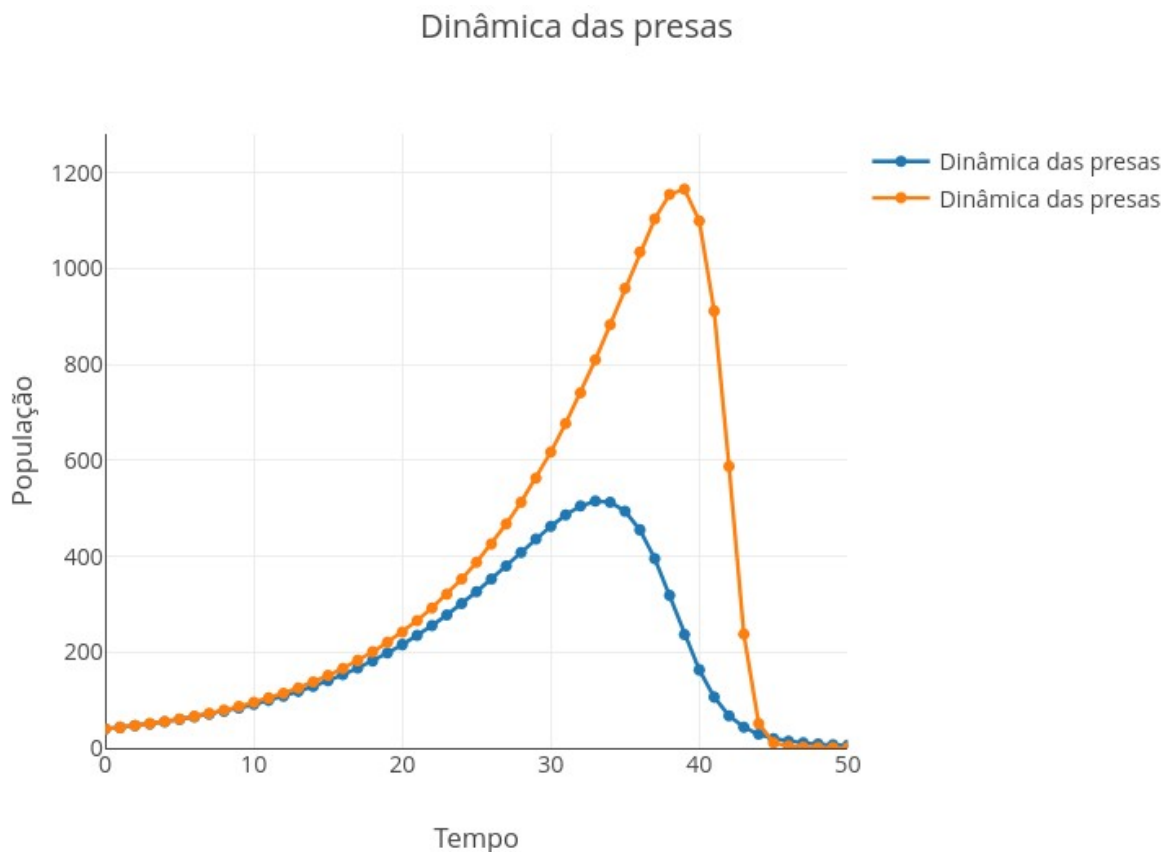


Figura 2.2: Aplicativo de JSPlotly para explorar a dinâmica de relação entre presa e predador. Clique com o botão direito do mouse neste [LINK](#) e abra-o numa nova aba. Ou use a tela capacitiva de seu dispositivo móvel (celular, tablet).

Agite antes de usar

O *app* da Figura 2.2 simula uma relação ecológica entre presas e predadores ao longo do tempo. Antes de estudar o modelo em detalhes, altere pequenos valores no código e observe como as populações respondem. Após clicar no botão *add*, observe que editor de códigos traz alguns valores em seu início:

```
let presa = 40;
let predador = 9;

const taxaCrescimento = 0.1;
const taxaPredacao = 0.002;
const taxaMorte = 0.2;
const eficiencia = 0.0008;
```

Altere os parâmetros, observe as respostas do *app*, e construa hipóteses.

1. Observe as populações iniciais. O sistema começa com:

```
let presa = 40;
let predador = 9;
```

A primeira representa a população de presas. A segunda representa a população de predadores.

2. Escolha qual população visualizar. Altere esta linha:

```
const mostrar = "presas";
```

Para observar os predadores, troque por:

```
const mostrar = "predadores";
```

3. Explore o crescimento das presas. Modifique:

```
const taxaCrescimento = 0.1;
```

Valores maiores fazem as presas crescerem mais rapidamente quando há alimento e espaço disponíveis.

4. Explore a predação. Altere:

```
const taxaPredacao = 0.002;
```

Essa taxa controla o quanto os predadores reduzem a população de presas.

5. Explore a morte dos predadores. Modifique:

```
const taxaMorte = 0.1;
```

Valores maiores fazem a população de predadores diminuir mais rapidamente quando há poucas presas.

6. Explore a eficiência ecológica. Altere:

```
const eficiencia = 0.0008;
```

Esse parâmetro indica o quanto a presença de presas contribui para o aumento dos predadores.

Agora você já está preparado para comparar cenários da relação entre presas e predadores. Assim, experimente, por exemplo:

- aumentar a quantidade inicial de presas;
- reduzir a quantidade inicial de predadores;
- aumentar a taxa de predação;
- aumentar a taxa de morte dos predadores;
- trocar entre "presas" e "predadores" para comparar as curvas.

Com um pouquinho de tempo no aplicativo, você já conseguirá responder diversas perguntas, como por exemplo, se há um crescimento contínuo, se existe algum padrão de repetição, e mesmo quem “controla” quem nessa relação !



No modelo proposto da [Figura 2.2](#), duas populações interagem ao longo do tempo. Dependendo do tipo de relação ecológica, o comportamento do sistema muda significativamente. Em *predação*, por exemplo, há oscilações típicas entre presa e predador. Em *competição*, ambas as populações disputam recursos. Em *mutualismo*, as duas populações se beneficiam. Em *parasitismo*, uma se beneficia enquanto a outra é prejudicada. Use os botões do *app* para explorar cada cenário, e tente responder as perguntas abaixo.

- O que acontece com as presas quando há muitos predadores ?
- O que acontece com os predadores quando há poucas presas ?
- Uma população pode crescer indefinidamente ?
- O equilíbrio ecológico significa ausência de mudança ?
- Há ciclos durante a relação ?
- Qual população controla a outra ?

Ou seja, em resumo...

Como as interações ecológicas regulam a dinâmica das populações ao longo do tempo ?

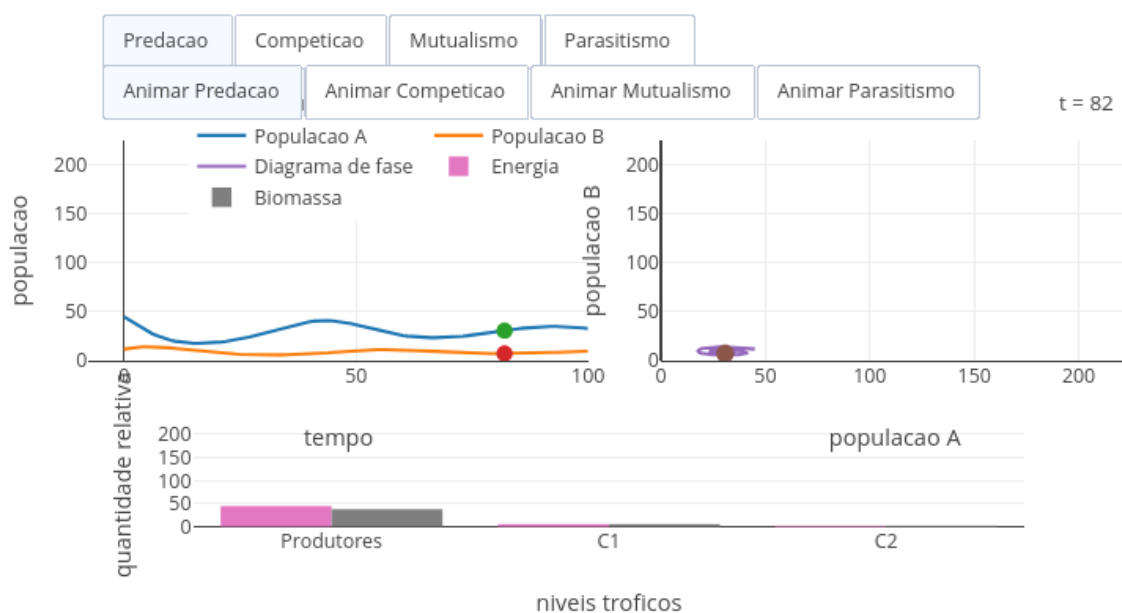
VOLTA PRO MUNDO



Esses modelos de interação presa-predador estendem-se para além do rato e serpente ilustrado ao início do capítulo. Na verdade, com a modelagem adequada, podem ser aplicados para ajudar a entender como ocorrem surtos populacionais e colapsos ecológicos, e também como se dá o controle biológico e o impacto humano em ecossistemas. Isso porque ecossistemas reais não são estáticos. A interação entre organismos e o fluxo de energia determina o equilíbrio ecológico, bem como colapsos populacionais e explosões demográficas. Um exemplo simples que você pode comprovar com o *app* da [Figura 2.2](#): a retirada de predadores pode causar superpopulação de presas, levando à escassez de recursos e colapso do sistema.

Agora experimente o JSPlotly mais completo que segue. Ele simula um controle mais abrangente para o estudo das relações ecológicas, fazendo com que a “brincadeira” vá para outro nível !

Ecologia: dinamica populacional e fluxo trofico



*Figura 2.3: As populações evoluem influenciando-se continuamente e criando padrões dinâmicos no ecossistema. JSPlotly para estudo de relações ecológicas, pirâmide de energia e biomassa. Clique com o botão direito do mouse neste [LINK](#) ou use a tela capacitiva de seu dispositivo móvel para acessar *app*.*

Agite antes de usar

O *app* simula diferentes *relações ecológicas*, apresentando a evolução das populações no tempo, um diagrama de fase (relação entre espécies), e pirâmides de energia e biomassa. Ele possui 3 painéis independentes com botões para *predação*, *competição*, *mutualismo*, e *parasitismo*. Para utilizá-lo, basta clicar em uma relação ecológica, observar os três painéis, e clicar no botão de animação correspondente à relação. O *app* possui um

menu de relações ecológicas, e apresenta gráficos de dinâmica populacional e de pirâmide de energia. O ajuste de parâmetros é feito diretamente no código, e permite:

1. controle de presas iniciais;
2. controle de predadores iniciais;
3. controle de crescimento da presa;
4. controle de taxa de predação;
5. controle de morte do predador;
6. controle de eficiência do predador.

Assim, é possível utilizar o aplicativo para explorar diversos cenários de relações ecológicas. Para um aproveitamento mais completo, siga as instruções que seguem.

1. Escolha o tipo de interação ecológica. Cada um representa uma forma diferente de interação entre duas populações. Use os botões:
 - Predação
 - Competição
 - Mutualismo
 - Parasitismo
2. Observe o gráfico principal (tempo $\times$ população) e compare o crescimento das populações, quedas abruptas, estabilização ou oscilações.
3. Use o botão Animar e interprete o movimento do ponto ao longo do tempo. Clique em:
 - Animar Predação
 - Animar Competição
 - Animar Mutualismo
 - Animar Parasitismo
5. Interprete o diagrama de fase. No gráfico da direita (população A $\times$ população B). Esse gráfico mostra como uma população depende da outra. Quando uma cresce, a outra também cresce ou diminui ?
6. Analise as pirâmides ecológicas. Na parte inferior você observa energia e biomassa. Observe produtores (maior valor) e consumidores (valores menores). Por que há menos energia nos níveis superiores ?
7. Compare os cenários:
 - Predação $\rightarrow$ oscilações cíclicas
 - Competição $\rightarrow$ limitação mútua
 - Mutualismo $\rightarrow$ crescimento conjunto

- Parasitismo → benefício de um e prejuízo do outro

8. Explore padrões e observe:

- Há equilíbrio ?
- Há ciclos ?
- Uma população domina a outra ?
- Ambas sobrevivem ?
- Qual sistema gera maior biomassa ?
- Qual mantém populações mais equilibradas ?
- Qual colapsa mais rápido ?

Agora que você explorou o *app*, reflita sobre o que observou frente às questões que seguem:

- Predação - algumas curvas oscilam entre presas e predadores. O que acontece com predadores quando há poucas presas ?
- Competição - outras curvas oscilam ou estabilizam. A competição sempre reduz as duas populações ?
- Mutualismo - outras podem crescer junto ou colapsar. No mutualismo as espécies podem crescer indefinidamente ?
- Parasitismo - alguém vence em longo prazo. O parasitismo leva à extinção de uma das espécies ?

Se você chegou até aqui testando as possibilidades acima, deverá verificar que nem toda interação gera um ciclo, que nem todo sistema é estável e que relações ecológicas têm “*assinaturas matemáticas*” que levam à oscilação, estabilidade, ou instabilidade.

Significado dos parâmetros dos cenários ecológicos

O script da [Figura 2.3](#) também possui uma série de parâmetros que definem os cenários ecológicos. Caso queira experimentar variações nesses parâmetros, ou mesmo a criação de outro cenário, como o *comensalismo*, basta modificá-los.

nome Indica o tipo de interação ecológica simulada: predação, competição, mutualismo ou parasitismo.

presa0 População inicial da primeira espécie. No cenário de predação, representa a população inicial de presas.

pred0 População inicial da segunda espécie. No cenário de predação, representa a população inicial de predadores.

rPresa Taxa de crescimento natural da população de presas, na ausência de predadores ou outras limitações fortes.

Kpresa Capacidade de suporte do ambiente para a população de presas.

Representa o tamanho máximo aproximado que essa população pode atingir no ambiente.

ataque Intensidade com que predadores encontram, capturam ou consomem presas.

eficiencia Eficiência com que o consumo de presas contribui para o crescimento da população de predadores.

mortePred Taxa de mortalidade natural dos predadores quando não há alimento suficiente.

rPred Taxa de crescimento natural da segunda espécie, usada nos cenários em que ela não depende diretamente de consumir a primeira.

Kpred Capacidade de suporte do ambiente para a segunda espécie.

compAB Efeito competitivo da espécie B sobre a espécie A. Quanto maior esse valor, maior a redução do crescimento de A causada por B.

compBA Efeito competitivo da espécie A sobre a espécie B. Quanto maior esse valor, maior a redução do crescimento de B causada por A.

beneficioAB Benefício que a espécie B exerce sobre a espécie A no mutualismo.

beneficioBA Benefício que a espécie A exerce sobre a espécie B no mutualismo.

rHosp Taxa de crescimento natural do hospedeiro no cenário de parasitismo.

Khosp Capacidade de suporte do ambiente para a população de hospedeiros.

dano Intensidade do prejuízo causado pelo parasita ao hospedeiro.

ganhoPar Ganho populacional do parasita associado à exploração do hospedeiro.

mortalidadePar Taxa de mortalidade natural do parasita quando sua manutenção não é compensada pelo hospedeiro.

E SE...



Agora pare um pouquinho para refletir sobre situações experimentadas da vida planetária ou criadas em sua mente, e nas quais as relações ecológicas são golpeadas por situações críticas. Algumas são passíveis de teste pelos aplicativos deste capítulo, outras não. Vão aí alguma ideias...

- E se não houvesse limite de recursos ?
- E se uma das espécies desaparecesse ?
- E se não houvesse perda de energia entre níveis tróficos ?
- E se existissem apenas produtores ?
- E se dois predadores competissem pela mesma presa ?
- E se a taxa de crescimento fosse muito alta ?
- E se a taxa de crescimento

Dessa forma, pode-se resumir o tema de relações ecológicas abordado aqui em função do perfil esperado pela dinâmicas das curvas encontradas nessas interações. A presa cresce proporcionalmente à sua população, enquanto o predador depende da presa para crescer. Nesse caso, a interação reduz a presa e favorece o predador, e tudo limitado pela energia disponível nessa interação. Ou seja, se as curvas oscilam, tem-se um sistema presa-predador clássico. Se as curvas estabilizam, uma provável competição com equilíbrio. E se as curvas crescem juntas, mutualismo reinando !

MESMO PADRÃO, OUTROS MUNDOS



Os conceitos deste capítulo também são aplicados em situações distintas em outros temas da Biologia. Modelos semelhantes são encontrados no crescimento bacteriano em cultura, na dinâmica de insetos em plantações, no controle biológico de pragas agrícolas (competição / parasitismo), no equilíbrio de cadeias alimentares (predação), e nas *interações simbióticas* entre alguns organismos (mutualismo), tais como as observadas em líquens (algas e musgos) e no cultivo de *quefir* e *kombucha*, dois probióticos da medicina popular. Além disso, os modelos matemáticos por detrás dessas relações são comumente aplicados em áreas bem diversas, como em alguns sistemas dinâmicos da Física e no comportamento de soluções da Química.

POR DENTRO DA FERRAMENTA



Esse espaço é dedicado para inserir o **pensamento computacional** plugado ao longo dos capítulos, e de propiciar algumas explicações que estão por detrás das **competências digitais** que você alcançou *mesmo sem perceber* ao longo dos temas. Para isso, é fornecido abaixo uma **lógica de código** explicitada na forma de um **pseudocódigo**, e que traduz o que um *script* de JSPlotly realiza para produzir um objeto interativo. Para essa seção, ilustra-se o objeto da [Figura 2.2](#), por ser mais simples de compreender.

Esse JSPlotly personalizado trabalha em cima do crescimento logístico (limite ambiental), das interações entre espécies, e da transferência de energia entre níveis tróficos. Para isso, o modelo utilizado baseia-se em interações dinâmicas entre populações. Em termos simplificados, a ideia é:

```
Para cada instante de tempo:  
  calcular crescimento da presa  
  calcular efeito da predação  
  calcular crescimento do predador  
  atualizar populações  
  calcular energia disponível
```

3. Genética Mendeliana e Molecular: dos padrões de herança à manipulação do DNA



O MUNDO TE CHAMA

A genética está presente em situações muito concretas da vida cotidiana. Ela explica por que certas características aparecem em algumas pessoas, e não em outras. Também explica como doenças hereditárias podem atravessar gerações, ou o que permite identificar paternidade, rastrear ancestralidade ou prever riscos biológicos. O avanço das últimas décadas no uso de ferramentas para investigar o DNA (ácido desoxirribonucleico) tem permitido ir além da simples observação de padrões de herança, possibilitando intervenções e modificações diretamente no material genético.

Durante muito tempo, a herança biológica foi observada apenas pelos seus efeitos visíveis. Plantas, animais e seres humanos exibiam traços que pareciam reaparecer segundo certas regularidades. A partir dessas observações, surgiram os primeiros modelos explicativos, especialmente os de Gregor *Mendel*, um frade agostiniano e biólogo, que mostrou que a transmissão de características não era aleatória. Mais tarde, a genética ganhou uma base molecular. Os genes passaram a ser entendidos como trechos de *DNA* (ácido desoxirribonucleico), capazes de armazenar informação, sofrer alterações e orientar a produção de proteínas.

Hoje, esse campo se ampliou ainda mais. A genética já não trata apenas da transmissão de características, mas também da estrutura do DNA, da expressão gênica, das mutações e das biotecnologias que permitem copiar, detectar e editar genes. Assim, o estudo da genética começou com ervilhas e proporções simples, passou pela lógica molecular da vida e alcançou técnicas como *PCR*, clonagem e edição gênica (*CRISPR*).

Dessa forma, a genética pode ser entendida em três planos que se complementam. O primeiro é o *plano mendeliano*, em que observamos padrões de herança a partir de características. O segundo é o *plano molecular*, em que investigamos a base física da informação genética. E o terceiro é o *plano biotecnológico*, em que o conhecimento acumulado passou a permitir intervenção prática sobre o material genético.

A proposta desse capítulo é percorrer esses planos de forma integrada. Em vez de estudar separadamente herança, DNA e biotecnologia, você está convidado a enxergar esses temas como partes de uma mesma história científica: a passagem do fenótipo observável ao código molecular, e deste à intervenção tecnológica.

Depois de explorar, você consegue:

- compreender como características biológicas podem ser transmitidas entre gerações, relacionando genótipo, fenótipo e probabilidade de ocorrência;

3. Genética Mendeliana e Molecular: dos padrões de herança à manipulação do DNA

- interpretar cruzamentos genéticos simples e duplos, reconhecendo padrões mendelianos e suas variações em diferentes contextos;
- explicar, em nível conceitual, como a informação genética está organizada no DNA e como ela é expressa na forma de proteínas;
- relacionar alterações na sequência de DNA (mutações) com possíveis efeitos sobre estruturas e funções biológicas;
- reconhecer que a herança genética é um processo probabilístico, sujeito a variações amostrais e influências múltiplas;
- analisar, de forma crítica, como técnicas de biotecnologia permitem identificar, modificar ou interromper sequências gênicas;
- conectar os níveis de organização da genética – do fenótipo observável ao DNA – compreendendo-os como partes de um mesmo sistema integrado;
- explorar situações-problema reais, como doenças genéticas, testes de paternidade e edição gênica, articulando conceitos científicos e implicações sociais;
- utilizar e interpretar simulações interativas voltadas à genética e biologia molecular.

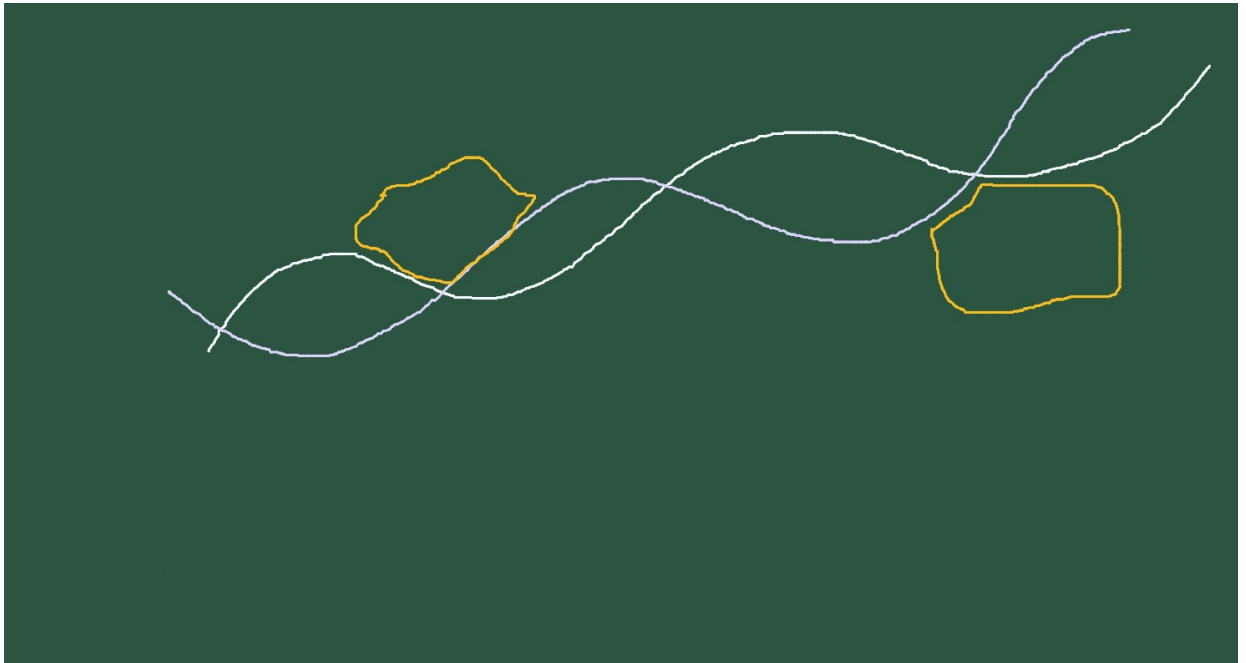


Figura 3.1: Como a informação hereditária é transmitida e expressa nos seres vivos.



Antes de qualquer definição formal para a lógica de cruzamentos genéticos e proporções emergentes (uau!), experimente o aplicativo de JSPlotly abaixo.

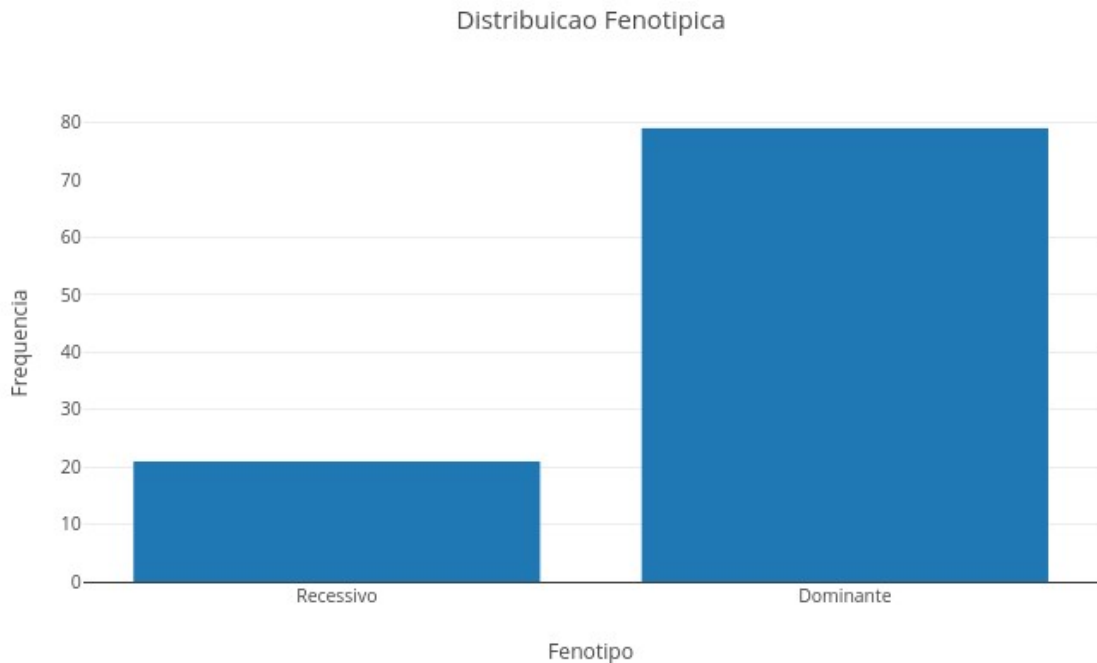


Figura 3.2: A genética mendeliana não é determinística, mas uma ponte entre biologia e probabilidade. Clique com o botão direito neste [LINK](#) e abra o JSPlotly personalizado em nova aba, seguido do botão add.

Nesse *script* de JSPlotly você pode alterar o genótipo dos pais (“AA”, “Aa”, “aa”) e o número de descendentes (n) e observar a distribuição de fenótipos emergentes, a variação estatística, bem como a diferença entre probabilidade teórica e simulação.

Agite antes de usar

Este script simula um cruzamento mendeliano simples entre dois indivíduos (*monohíbrido*).

O exemplo usado é:

$$Aa \times Aa (3.1)$$

A cada simulação, o código sorteia alelos dos pais e gera descendentes possíveis. No final, o gráfico mostra a frequência dos fenótipos (*dominante, recessivo*). Na genética mendeliana, cada indivíduo possui dois alelos para uma característica. No exemplo, Aa, o alelo A (maiúsculo) é *dominante* e o alelo a (minúsculo) é *recessivo*.

Dessa forma:

- AA → fenótipo dominante;
- Aa → fenótipo dominante;

- $aa \rightarrow$ fenótipo recessivo;

Para “se divertir” com o código, comece primeiro alterando os genótipos dos pais.

```
let pai = "Aa";  
let mae = "Aa";
```

Experimente variações, como AA ou aa para pai ou mãe, por exemplo. E modifique também o número de descendentes simulados no trecho abaixo (ex: 20 ou 1000):

```
let n = 100;
```

O cruzamento no código é dado pela probabilidade dos genótipos gerados. Assim, para $Aa \times Aa$, por exemplo, a proporção de genótipos é dada no código por:

$$Aa \times Aa \quad (3.2)$$

Dessa forma, a proporção esperada de genótipos é:

$$1AA : 2Aa : 1aa \quad (3.3)$$

Já a proporção fenotípica esperada é:

$$3 \text{ dominantes} : 1 \text{ recessivo} \quad (3.4)$$

Mas como o script usa sorteios aleatórios, os resultados podem variar. Quanto maior o valor de n , mais o resultado tende a se aproximar da proporção esperada.

Alguns cenários mostram-se interessantes para exploração do *script* da [Figura 3.2](#), tais como:

1. *Dois heterozigotos*. Experimente $\text{pai} = "Aa"; \text{mae} = "Aa"$. E verifique se o gráfico aproxima-se da proporção 3:1 (dominante:recessivo).
2. *Dominante puro com recessivo puro*. Teste $\text{pai} = "AA"; \text{mae} = "aa"$. E veja que todos os descendentes apresentam fenótipo dominante.
3. *Heterozigoto com recessivo*. Faça $\text{pai} = "Aa"; \text{mae} = "aa"$. Observe que a proporção esperada é 1:1.
4. *Efeito do tamanho da amostra*. Compare n para 20 e para 1000 descendentes. E veja que, com poucos descendentes, o acaso pesa mais. Já com muitos descendentes, a proporção mendeliana aparece com mais clareza.

Em termos técnicos, o *script* permite comparar cruzamentos entre homozigotos e heterozigotos, observar diferenças entre genótipo e fenótipo, relacionar probabilidades esperadas e resultados observados, e mesmo expandir esses conceitos de monoibridismo para diibridismo.

Além disso, também mostra que a genética mendeliana combina regras de herança com probabilidade, aleatoriedade, e padrões estatísticos. Assim, o cruzamento genético pode ser visto como uma simulação probabilística, onde cada descendente nasce de um sorteio de alelos. Em outras palavras, Mendel observava proporções, mais do que indivíduos

3. Genética Mendeliana e Molecular: dos padrões de herança à manipulação do DNA

FAZENDO APARECER



3.1 Genética mendeliana: probabilidade ao invés de determinismo

Na lógica dos padrões de herança da genética clássica, a ideia é que determinadas características são influenciadas por fatores herdáveis, hoje chamados genes. Cada gene pode apresentar diferentes versões, os alelos. Em organismos diploides, cada indivíduo recebe um alelo de cada progenitor, formando assim seu genótipo. O fenótipo corresponde à manifestação observável dessa combinação, em interação com o ambiente.

A Primeira Lei de Mendel mostra que os alelos de um mesmo gene se separam durante a formação dos gametas. Isso explica por que cruzamentos simples entre indivíduos heterozigotos produzem proporções genotípicas e fenotípicas características. Já a Segunda Lei de Mendel amplia esse raciocínio ao considerar a segregação independente de pares de genes diferentes, permitindo compreender cruzamentos diíbridos e combinações mais complexas.

Embora estejam nos primórdios históricos da experimentação genética e com relevância significativa nos dias atuais, essas leis não esgotam a herança biológica. Há casos de dominância incompleta, codominância, alelos múltiplos e herança ligada ao sexo, entre outros. Ainda assim, o modelo mendeliano permanece como porta de entrada conceitual, pois ensina a pensar a herança como combinação de unidades discretas.

3.2 Genética molecular: o que são genes, afinal?

A genética molecular aprofunda essa discussão ao identificar a base material da herança: o DNA, molécula que armazena informação em sua sequência de nucleotídeos. *Genes são regiões do DNA capazes de participar da síntese de produtos funcionais*, geralmente proteínas ou moléculas de RNA (ácido ribonucleico).

A replicação permite que o DNA seja copiado antes da divisão celular. A transcrição converte trechos de DNA em RNA, enquanto a tradução utiliza essa informação para organizar aminoácidos em proteínas. Esse é o chamado Dogma Central da Biologia Molecular. Desse modo, a herança deixa de ser apenas um padrão estatístico observado em descendentes e passa a ser compreendida como fluxo de informação molecular.

Essa passagem da genética mendeliana para a molecular é decisiva. Aquilo que Mendel inferiu indiretamente por meio de fenótipos foi, mais tarde, conectado a genes, cromossomos, DNA e mecanismos celulares. A herança biológica ganhou então uma base explicativa mais profunda.

Para ilustrar essa passagem, utilize os próximos *app* de JSPlotly que seguem. O da [Figura 3.3](#), por exemplo, busca abordar a genética mendeliana um pouco mais avançada, na passagem do monoibridismo (uma característica herdada) para o diíbridismo (duas características).

3. Genética Mendeliana e Molecular: dos padrões de herança à manipulação do DNA

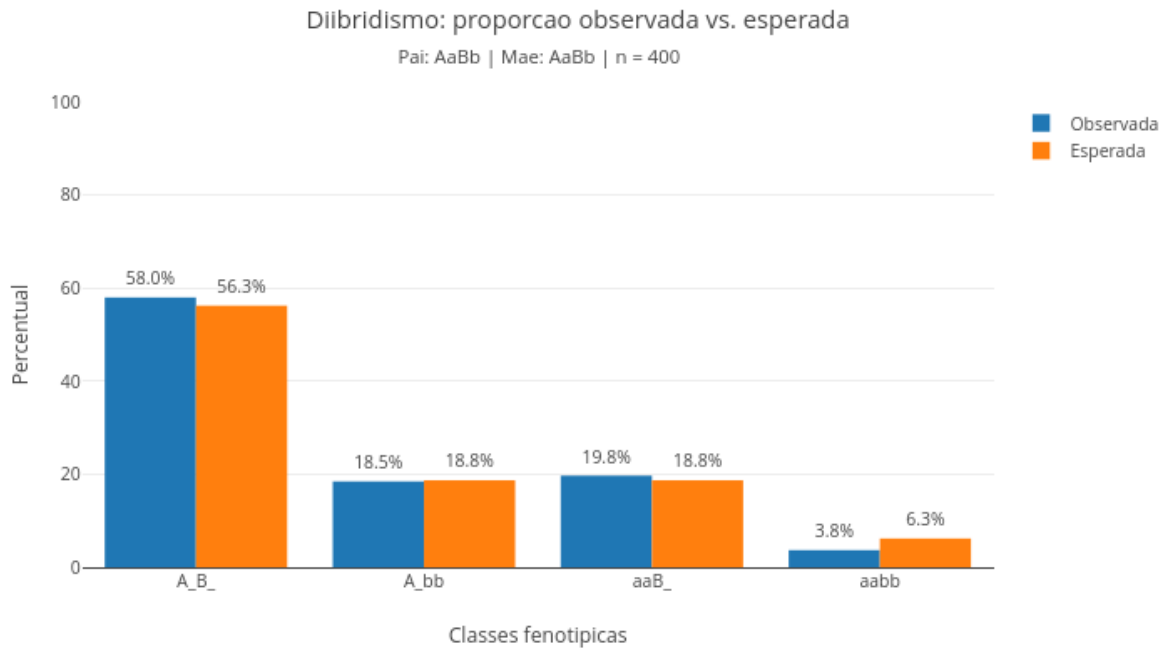


Figura 3.3: A genética mendeliana como ferramenta de probabilidade. Embora haja incerteza sobre o resultado de um indivíduo, o padrão de muitos descendentes pode ser previsto. Clique neste [LINK](#) para carregar o app.

Agite antes de usar

O script simula um cruzamento envolvendo dois genes ao mesmo tempo. Você pode alterar os genótipos para *pai*, *mãe*, e *n*. Exemplos para os dois primeiros são “AaBb”, “AABB”, “Aabb”, “aaBb”, e “aabb”. O gráfico resultante compara a proporção *observada* na simulação com a proporção *esperada* pelas regras mendelianas. Isso permite mostrar que a herança segue uma lógica probabilística, embora a amostra real possa oscilar, especialmente quando o valor de *n* é pequeno.

No diíbrido, duas características são analisadas simultaneamente. Cada indivíduo possui dois pares de alelos, AaBb. Isso representa:

- gene A: A ou a;
- gene B: B ou b.

Um indivíduo AaBb pode formar gametas como AB, Ab, aB, ou ab. Assim, no início do código, altere:

```
let pai = "AaBb";  
let mae = "AaBb";  
let n = 400;
```

Experimente “AABB”, “aabb”, “AaBb” ou “aabb”, bem como o tamanho *n* da amostra (50, 1000), e observe o resultado para um cruzamento clássico. Exemplificando, se o cruzamento for:

$$AaBb \times AaBb (3.5)$$

Então a proporção fenotípica esperada é 9:3:3:1. Ou seja:

9A_B_ : 3A_bb : 3aaB_ : 1aabb

O script calcula os dois conjuntos de valores, *observado* e *calculado*, como segue. O resultado dos sorteios simulados é dado por:

```
const filhos = cruzarDiibrido(pai, mae, n);
```

Já o resultado teórico obtido a partir de todas as combinações possíveis de gametas, é dado por:

```
const espPct = proporcaoEsperada(pai, mae);
```

Diversos são os cenários possíveis de visualização com auxílio do JSPlotly da [Figura 3.3](#) para ajudar-lhe a compreender melhor essas regras de cruzamento diíbrido. Ilustrando:

1. *Cruzamento clássico*. Use *pai* = "AaBb" e *mae* = "AaBb". Observe se a proporção observada se aproxima de 9:3:3:1.
2. *Duplo homozigoto dominante com duplo recessivo*. Faça *pai* = "AABB" e *mae* = "aabb". Dessa forma, espera-se que todos os descendentes serão AaBb.
3. *Cruzamento-teste (Testcross)*. Use *pai* = "AaBb" e *mae* = "aabb". Observe a distribuição entre as quatro classes fenotípicas. Esse teste cruzado (*testcross*) foi introduzido por Mendel para determinar o genótipo de um indivíduo com fenótipo dominante (desconhecido), cruzando-o com um homozigoto recessivo.
4. *Tamanho da amostra*. Compara $n = 40$ com $n = 4000$. Quanto maior o número de descendentes simulados, mais estável fica a proporção observada.

Em síntese, acompanhamos uma característica apenas no monoibridismo, enquanto que no diibridismo, acompanhamos duas. O cruzamento diíbrido mostra como a herança mendeliana pode gerar padrões complexos também a partir de regras simples.



A genética integrada ajuda a interpretar uma enorme variedade de situações reais. Questões familiares sobre herança de traços, testes de paternidade, doenças hereditárias e compatibilidade sanguínea aderem-se com praticidade à genética mendeliana. Já diagnósticos moleculares, identificação de mutações e compreensão de mecanismos celulares, por sua vez, dependem da genética molecular. E para essa, vacinas de RNA, testes baseados em DNA, transgênicos e terapias gênicas mostram o alcance biotecnológico desse campo.

Perceba que a genética organiza modos distintos de se pensar o corpo, a herança, a diversidade biológica e o próprio futuro das intervenções biomédicas, um exemplo fértil no qual a ciência traduz observações em modelos, modelos em mecanismos e mecanismos em tecnologias.

3.3 Mutações e variabilidade

Se os genes são sequências de DNA, alterações nessas sequências podem modificar a informação biológica. Essas alterações são chamadas mutações. Algumas não produzem efeitos perceptíveis, outras modificam proteínas, alteram funções celulares ou contribuem para doenças genéticas. Em certos contextos, também, podem gerar variabilidade útil à evolução.

As mutações ajudam a conectar o nível molecular ao nível fenotípico. Uma troca de uma base nitrogenada, por exemplo, pode alterar um códon (trinca de bases), modificar um aminoácido e influenciar a estrutura final de uma proteína. Assim, uma pequena alteração microscópica pode produzir grandes efeitos biológicos.

3.4 Biotecnologia: compreender para intervir

Quando a genética passou a ser entendida em nível molecular, surgiram também ferramentas para manipular o DNA. A biotecnologia genética passou então a incluir estratégias como PCR, DNA recombinante, produção de organismos transgênicos, clonagem e edição gênica.

Explicando, PCR (*Reação em Cadeia da Polimerase*) é uma técnica de biologia molecular que amplifica exponencialmente sequências específicas de DNA, facilitando análises e diagnósticos. A tecnologia do DNA recombinante tornou possível inserir genes em outros organismos. A clonagem mostrou que materiais genéticos podem ser copiados em contextos específicos. Mais recentemente, sistemas de edição como CRISPR (*Clustered Regularly Interspaced Short Palindromic Repeats*), ampliaram enormemente a capacidade de modificar sequências gênicas de forma direcionada, utilizando uma “tesoura molecular” (sistema Cas9) para cortar, inserir ou corrigir sequências específicas de DNA de forma rápida e precisa.

Essas técnicas trazem aplicações promissoras em saúde, agricultura, pesquisa e indústria, mas também levantam questões éticas importantes, já que intervir no DNA deixou de ser uma “possibilidade” técnica para ser uma decisão social.



E SE...

Dos *apps* apresentados acima, é possível abstrair um sem-número de reflexões sobre a genética mendeliana. Considerando o JSPlotly da [Figura 3.2](#), por exemplo:

- E se os dois pais forem homozigotos dominantes ?
- E se um dos pais for recessivo puro ?
- E se aumentarmos muito o número de descendentes simulados ?
- E se o resultado observado for diferente da proporção esperada ?
- E se quisermos contar também os genótipos, e não apenas os fenótipos ?
- E se uma característica não seguir exatamente o padrão dominante-recessivo ?

Para o script da [Figura 3.3](#), por sua vez, também é possível perguntar:

- E se os dois pais forem duplo heterozigotos ?
- E se um dos pais for duplo recessivo ?
- E se aumentarmos muito o número de descendentes simulados ?
- E se a proporção observada não for exatamente igual à esperada ?
- E se os genes não fossem independentes ?
- E se quisermos comparar genótipo e fenótipo no mesmo gráfico ?

Assim, perceba que nem toda herança segue o modelo mais simples apresentado por Mendel. Em alguns casos, os heterozigotos apresentam fenótipos intermediários, enquanto em outros, dois alelos se manifestam simultaneamente. Essas situações mostram que os modelos básicos são ponto de partida, mas não o ponto final.

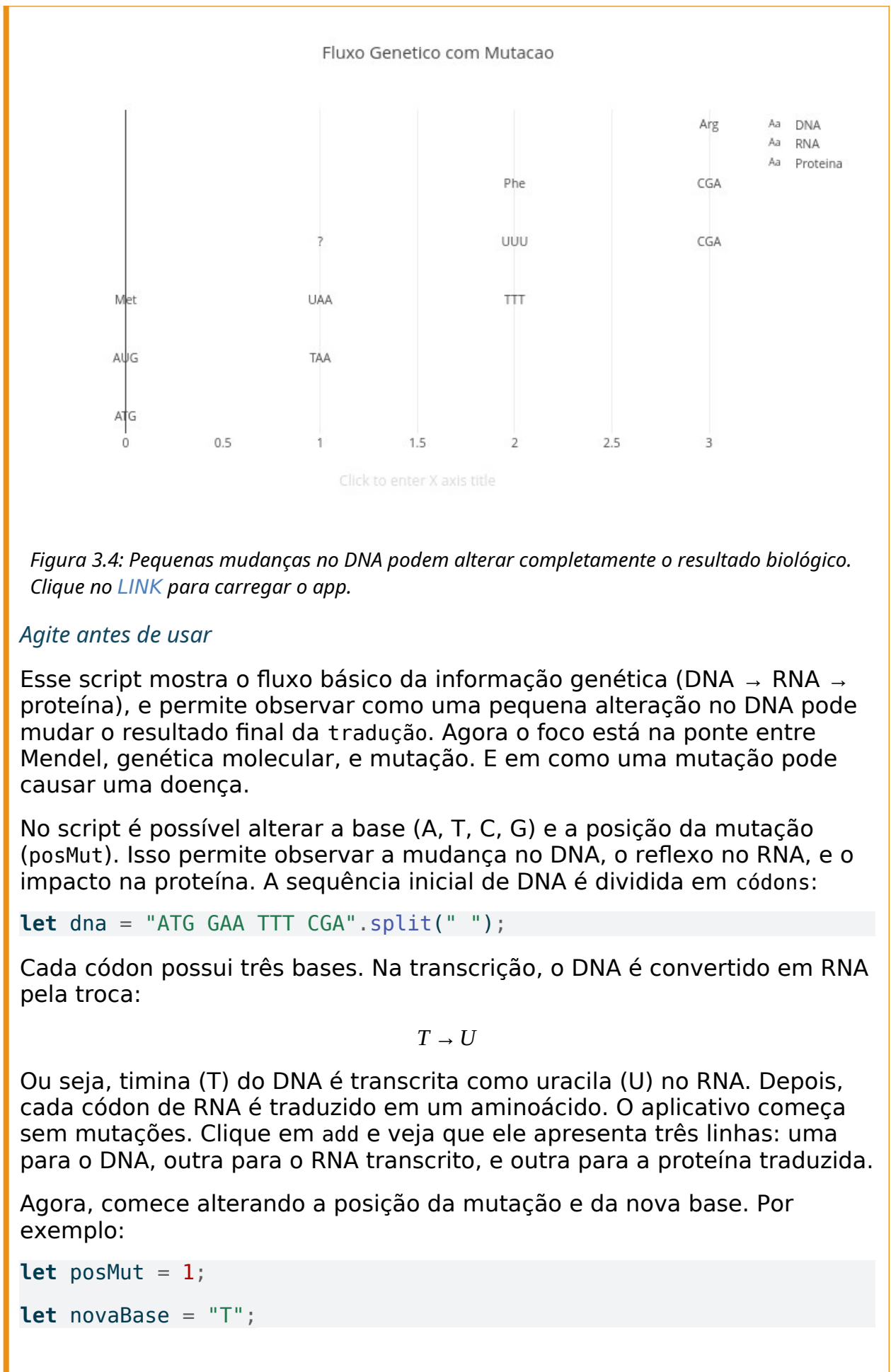
E se a ciência puder corrigir genes defeituosos ?. A edição gênica abre possibilidades de amplo espectro, como corrigir mutações associadas a doenças. Ao mesmo tempo, essa capacidade exige um debate ético: quais usos são justificáveis ? Quais limites devem existir ? Como a Inteligência Artificial entra nesse jogo ? Quem decide ?

E se a herança genética não determinar tudo ?. Mesmo quando a herança genética influencia fortemente um fenótipo, o ambiente continua importante. A genética não deve ser vista como destino absoluto, mas como parte de uma rede de interações biológicas e ambientais.

E se uma única mutação alterar uma proteína essencial ?. Uma pequena alteração em uma sequência de DNA pode não produzir efeito aparente, mas também pode comprometer a função de uma proteína importante. Esse tipo de raciocínio ajuda a conectar biologia molecular, fisiologia e saúde.

Para essa última indagação, que tal explorar mais um JSPlotly ?!

3. Genética Mendeliana e Molecular: dos padrões de herança à manipulação do DNA



Se a mutação gerar um códon conhecido na tabela, aparecerá um aminoácido. Se o códon não estiver na tabela simplificada, aparecerá uma interrogação " ? " na linha do aminoácido correspondente.

A transcrição acontece aqui:

```
codon.replace(/T/g, "U")
```

Já a tradução usa uma tabela simplificada:

```
const tabela = {  
  "AUG": "Met",  
  "GAA": "Glu",  
  "UUU": "Phe",  
  "CGA": "Arg"  
};
```

E a mutação altera a primeira base de um códon escolhido:

```
codon[0] = base;
```

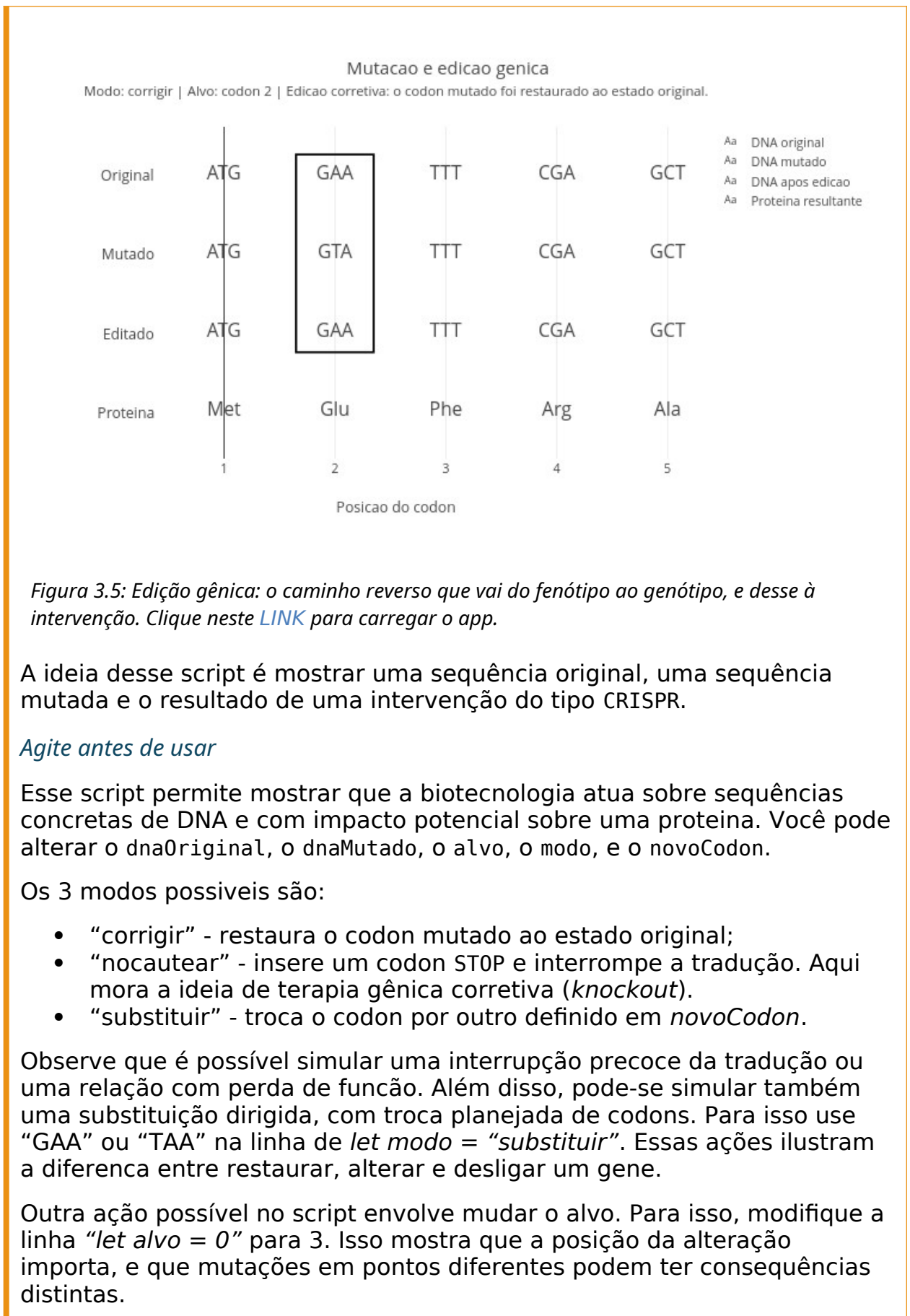
Perceba então que uma mutação pequena no DNA pode ou não alterar a proteína.

Como para os demais *apps* deste livro, procure montar cenários para se apropriar dos conceitos envolvidos. Ilustrando:

- E se a mutação não alterar o aminoácido ?
- E se a mutação gerar um códon desconhecido pela tabela ?
- E se a mutação ocorrer no primeiro códon ?
- E se ampliarmos a tabela genética ?

E para te auxiliar a responder essa última questão “E se...”, bem como ilustrar uma aplicação biotecnológica, que tal um último JSPlotly ?!

3. Genética Mendeliana e Molecular: dos padrões de herança à manipulação do DNA



MESMO PADRÃO, OUTROS MUNDOS



Os JSPlotly trazidos nesse capítulo pretendem ilustrar a genética em múltiplas faces. Enquanto o JSPlotly da [Figura 3.2](#) busca a percepção de regularidades simples, o da [Figura 3.3](#) direciona-se para perguntar onde está essa informação, ao passo que o da [Figura 3.4](#) amplia a discussão para dois genes e probabilidade, e o da [Figura 3.5](#) mostra que hoje a genética pode ser editada.

Assim, percebe-se que a genética pode ser estudada com um racional explicativo. Primeiro, reconhecem-se padrões de herança em fenótipos observáveis. Depois, identifica-se que esses padrões dependem de genes localizados no DNA. Em seguida, entende-se que a informação genética pode ser expressa, alterada e até manipulada tecnologicamente. As técnicas de DNA recombinante levam hoje a novas fronteiras de aplicação, como *CRISPR* em embriões humanos, células artificiais e biologia sintética, design de proteínas por inteligência artificial, edição genética *in vivo* e edição epigenética (ambiente e estilo de vida afetando genes sem alterar o DNA), entre muitas outras.

Ao integrar genética mendeliana, molecular e biotecnologia, esse capítulo mostra que os padrões observados nos organismos se conectam a mecanismos moleculares que, por sua vez, contribuem para tecnologias capazes de transformar pesquisa, medicina e sociedade.

POR DENTRO DA FERRAMENTA



O capítulo utiliza duas propostas para o estudo interativo de genética, a *mendeliana*, que permite observar cruzamentos, proporções e padrões de herança de modo visual e manipulável, e a *molecular*, mostrando como alterações no DNA podem repercutir em produtos celulares, e como certas técnicas exploram essa lógica em aplicações reais.

Para ilustrar um pouco mais do conteúdo da “caixa-preta” do script de JSPlotly da [Figura 3.2](#), segue uma explicação rápida para algumas funções.

A função abaixo transforma o genótipo em gametas possíveis:

```
function gametas(genotipo) {  
  return genotipo.split("");  
}
```

Assim:

```
"Aa"
```

vira:

```
["A", "a"]
```

Depois, o script sorteia um alelo do pai e um alelo da mãe:

```
const a1 = gp[Math.floor(Math.random() * gp.length)];  
const a2 = gm[Math.floor(Math.random() * gm.length)];
```

O genótipo do filho é formado por:

```
const filho = [a1, a2].sort().join("");
```

A função abaixo define o fenótipo:

```
function fenotipo(gen) {  
  return gen.includes("A") ? "Dominante" : "Recessivo";  
}
```

Ou seja, se existir pelo menos um alelo dominante A o fenótipo será dominante. Assim, pode-se “transcrever” uma *lógica de código* para o script da [Figura 3.2](#) como:

```
genótipos dos pais  
  ↓  
gametas possíveis  
  ↓  
sorteio de alelos  
  ↓  
genótipo dos filhos  
  ↓  
fenótipo  
  ↓  
gráfico de frequências
```

3.5 Pseudocódigo e outro JSPlotly

Aqui vamos quebrar um pouco a estrutura da seção, e apresentar um trecho e um *pseudocódigo* para o 2o. script de JSPlotly ([Figura 3.3](#)), para

3. Genética Mendeliana e Molecular: dos padrões de herança à manipulação do DNA

ilustrar como algoritmos um pouco mais elaborados podem ser traduzidos em “*linguagem humana*”. No código, a função abaixo separa o genótipo em dois pares de genes:

```
function separarGenotipo(g) {  
  return [  
    g.slice(0, 2),  
    g.slice(2, 4)  
  ];  
}
```

Assim:

```
"AaBb"
```

vira:

```
["Aa", "Bb"]
```

Depois, o script gera todas as combinações possíveis de gametas:

```
function gametasDiibrido(g) {  
  ...  
}
```

Para:

```
AaBb
```

os gametas possíveis são:

```
AB, Ab, aB, ab
```

A cada descendente, o script sorteia um gameta de cada progenitor:

```
const a = gp1[Math.floor(Math.random() * gp1.length)];  
const b = gp2[Math.floor(Math.random() * gp2.length)];
```

Depois, combina os gametas para formar o genótipo do filho. O gráfico compara essa previsão com os resultados sorteados pela simulação. Com poucos descendentes, a proporção observada pode variar bastante. Com muitos descendentes, ela tende a se aproximar da proporção esperada.

Uma proposta de *pseudocódigo* para esse script é:

```
INÍCIO  
  
// 1. Definir genótipos dos pais  
definir pai  
definir mae  
definir n (número de descendentes)  
  
// 2. Separar genes em pares (A e B)  
função separarGenotipo(genotipo):  
  retornar [geneA, geneB]  
  
// 3. Gerar gametas possíveis
```

3. Genética Mendeliana e Molecular: dos padrões de herança à manipulação do DNA

```
função gerarGametas(genotipo):
    separar em dois pares (geneA, geneB)
    para cada alelo de geneA:
        para cada alelo de geneB:
            combinar alelos → formar gameta
    retornar lista de gametas

// 4. Combinar gametas para formar descendente
função combinarGametas(gameta_pai, gameta_mae):
    formar par para gene A
    formar par para gene B
    organizar (dominante antes de recessivo)
    retornar genótipo completo

// 5. Determinar fenótipo
função fenotipo(genotipo):
    verificar presença de alelo dominante em A
    verificar presença de alelo dominante em B

    se A dominante e B dominante → "A_B_"
    se A dominante e B recessivo → "A_bb"
    se A recessivo e B dominante → "aaB_"
    se ambos recessivos → "aabb"

    retornar classe fenotípica

// 6. Simular descendentes (parte probabilística)
gerar lista de gametas do pai
gerar lista de gametas da mãe

criar contador de fenótipos

PARA i de 1 até n:

    sortear gameta do pai
    sortear gameta da mãe

    combinar → formar descendente
    identificar fenótipo
    incrementar contador correspondente

FIM PARA

// 7. Calcular proporção observada
converter contagem em porcentagem

// 8. Calcular proporção esperada (todas as combinações)
PARA cada gameta do pai:
    PARA cada gameta da mãe:
        combinar gametas
        determinar fenótipo
        contar ocorrência
```

3. Genética Mendeliana e Molecular: dos padrões de herança à manipulação do DNA

```
FIM
FIM

converter contagem em porcentagem

// 9. Construir gráfico
plotar:
    barras → proporção observada
    barras → proporção esperada

FIM
```

4. Origem, Organização, e Evolução da Vida

O MUNDO TE CHAMA 🌱

Tudo o que existe vivo, de uma bactéria a uma floresta inteira, parece seguir algum tipo de organização. De fato, a vida se organiza em células, tecidos, órgãos e sistemas. No ser humano, por exemplo, os sistemas, tais como o circulatório e o respiratório, são formados por órgãos com funções específicas. Essas funções são desempenhadas por tecidos peculiares. Os tecidos possuem células vivas, e essas, organelas que desempenham papéis bastante específicos. As organelas, por sua vez, são formadas de moléculas, tais como proteínas, DNA, lipídios e carboidratos.

Assim, pode-se dizer a vida é um problema de organização da matéria. Pode ter surgido a partir da matéria não viva, e passou a se organizar em compartimentos, células, tecidos e órgãos. Neste capítulo você terá uma panorâmica de como a vida surgiu e por que ela se organiza em níveis cada vez mais complexos. E apenas pra colocar-lhe uma *“pulga atrás da orelha”*, reflita se ecossistemas também não formariam um nível de organização acima do indivíduo !

Depois de explorar, você consegue:

- compreender hipóteses gerais sobre a origem da vida;
- reconhecer a importância da compartimentalização nos sistemas vivos;
- identificar níveis de organização biológica, das moléculas aos sistemas;
- relacionar estrutura, função e cooperação entre unidades vivas;
- interpretar propriedades emergentes na passagem entre níveis biológicos;
- explorar modelos computacionais simples para representar organização da vida.

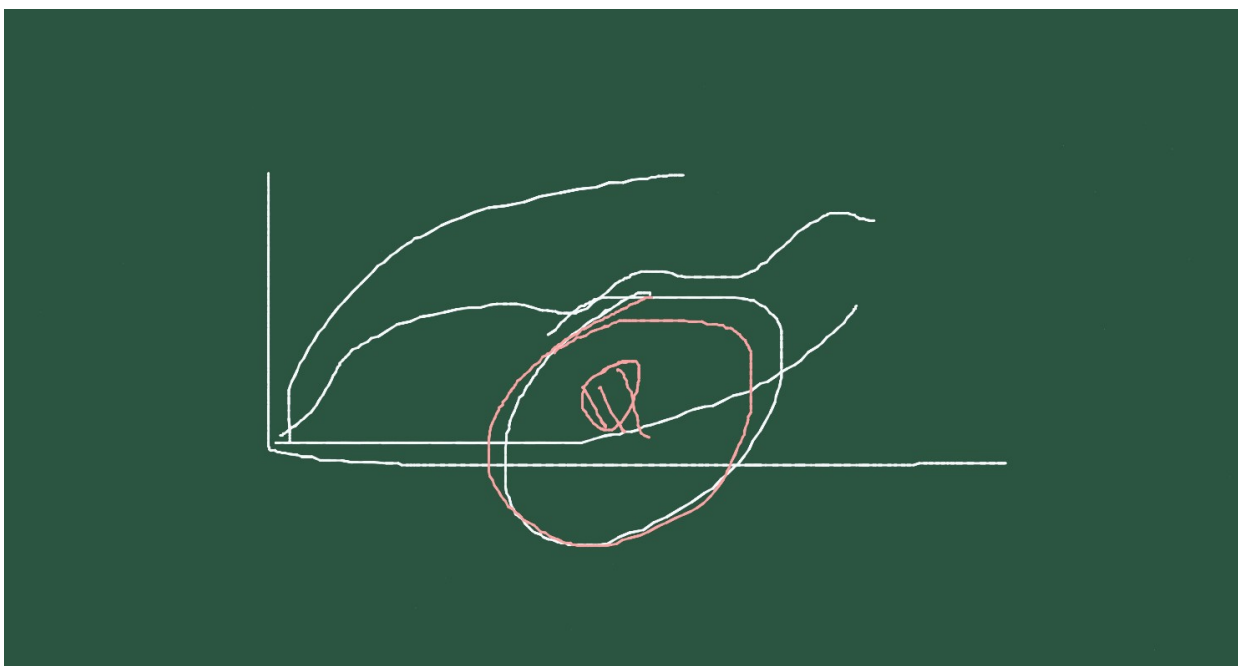


Figura 4.1: A vida é um problema físico-químico que integra informação e estrutura.

MEXA ANTES DE ENTENDER



Antes de qualquer definição formal, experimente o JSPlotly abaixo. Interaja com o modelo e observe como estruturas simples podem dar origem a níveis mais complexos de organização.

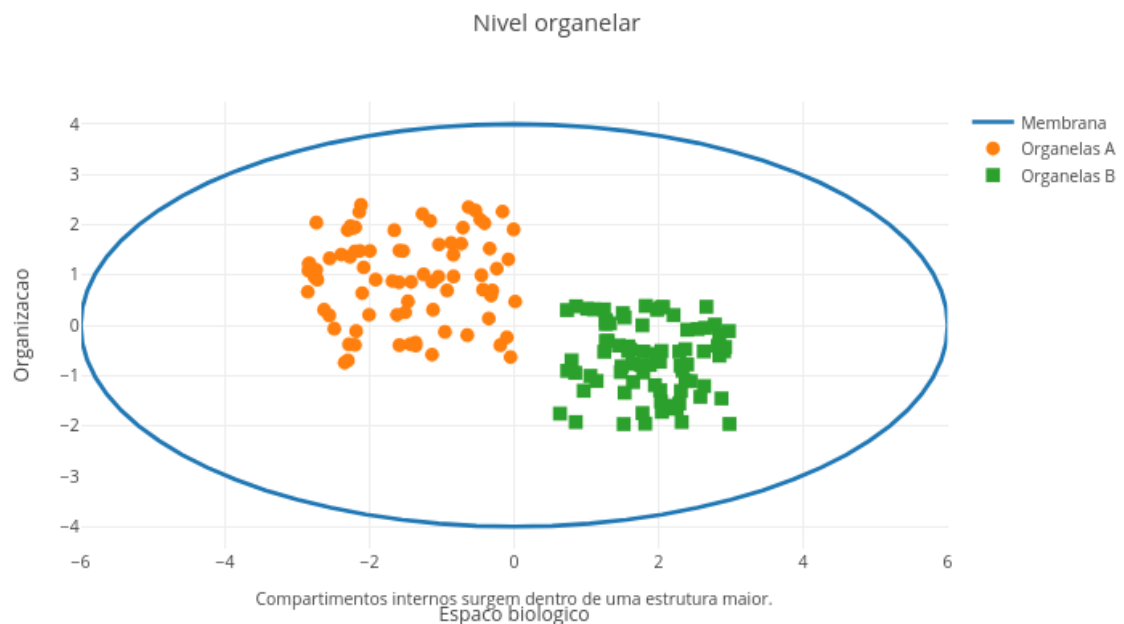


Figura 4.2: A vida depende de organização funcional mais do que quantidade ou de forma. Clique com o botão direito neste [LINK](#) e abra o app em nova aba.

Nesse primeiro JSPlotly, a proposta não é “provar” como a vida surgiu, mas perceber que a organização biológica pode ser pensada em níveis. Ao variar o nível observado, a quantidade de unidades e sua dispersão, você compara desde arranjos mais simples até estruturas mais integradas, discutindo quando a simples presença de matéria passa a sugerir organização.

Agite antes de usar

No script você altera o o nível de organização observado, a quantidade de unidades e a dispersao. A ideia é que a vida não é só matéria, mas matéria organizada em níveis.

1. *Nível de organização.* Você pode escolher o nível molecular, organelar, celular, tecidual, órgão, ou sistema. Numericamente, ok ? Ou seja: 0, 1, 2, 3, 4, ou 5, respectivamente. Clique no botão add e veja a diferença nos níveis de organização biológica.
2. *Quantidade de unidades.* Varie o quanto quiser aqui.
3. *Dispersão.* Esse campo do código mostra quão espalhadas ou concentradas aparecem as estruturas.

Então, você pode experimentar:

```
const nivel = 0;
```

```
const quantidade = 20;
const dispersao = 1.2;
```

Rode e observe. Depois mude para:

```
const nivel = 3;
const quantidade = 40;
const dispersao = 0.4;
```

E compare com o anterior.

Outro contexto interessante é comparar o grau de *dispersão* dos elementos. Para isso, faça:

```
const nivel = 1;
const quantidade = 400;
const dispersao = 0.4;
```

Depois mude a dispersão para 0.9.

Aqui vale uma consideração importante, sobre como interpretar a dispersão. A dispersão não significa diretamente “vida” ou “não vida”. Ela serve apenas como um recurso visual para o conceito de organização. Assim, uma dispersão alta indica elementos mais espalhados e menos concentrados, enquanto que uma dispersão baixa indica elementos mais agrupados, sugerindo maior proximidade e integração. Traduzindo para o conceito de vida, é como se em algum ponto intermediário houvesse um conjunto de partes que começa a se parecer com uma organização, e não apenas um amontoamento de elementos.

Seguem alguns cenários exploráveis para esse script da [Figura 4.2](#).

1. Comece no nível molecular e suba até sistema, discutindo o que aparece de novo em cada etapa.
2. Aumente a quantidade de elementos. Será que quantidade sozinha gera vida?
3. Reduza a dispersão e discuta sobre organização *versus* amontoamento.

“Brincando” com o *app*, você será capaz de discutir grandes questões da humanidade sobre o assunto, como o que diferencia matéria viva de não viva, como moléculas podem formar estruturas organizadas, e por que a célula é considerada a unidade básica da vida.



FAZENDO APARECER

A vida pode ser compreendida como uma escala de organização. Moléculas podem compor compartimentos e esses compartimentos podem integrar as células. Células cooperam em tecidos e esses em órgãos, os quais se articulam em sistemas. Em cada passagem, surgem propriedades novas, que não aparecem do mesmo modo no nível anterior. Essas

propriedades emergentes para novas funções e padrões estão na base do conceito de vida, tendo como marco a célula biológica com suas múltiplas compartimentalizações.

Para adentrar um pouco mais nessa ideia de função compartimentalizada, teste o *app* de JSPlotly que segue. Ele pretende simular, ainda que (pra lá de) grosseiramente, um modelo celular com organelas ativáveis, para se observar compartimentalização e função.

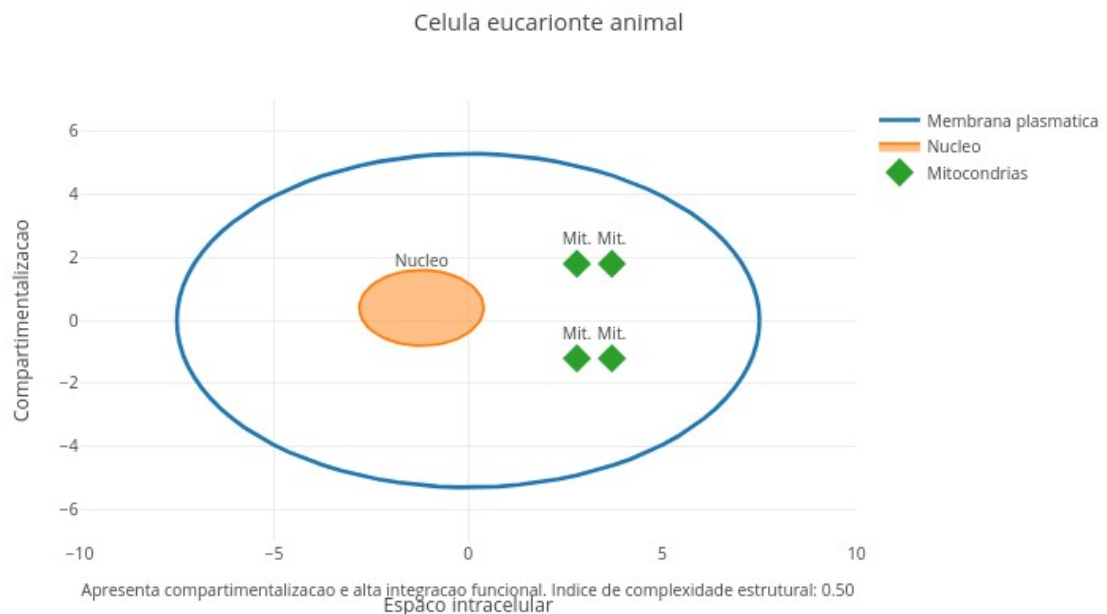


Figura 4.3: Propriedades emergem quando certas estruturas aparecem nos seres vivos. Clique neste [LINK](#) para abrir o aplicativo em nova aba.

Esse script explora uma “célula por dentro”, sua compartimentalização, organelas, e a relação entre estrutura e função advinda dessa organização.

Agite antes de usar

Agora as variáveis são do tipo liga/desliga e tipo de célula (*tipoCelula*). Quanto à primeira, você pode ativar (1) ou desativar (0) algumas estruturas, como membrana, parede celular, núcleo, mitocôndria, cloroplasto, e vacúolo. Em relação aos tipos celulares, você pode escolher entre "procarionte", "eucarionte animal", ou "eucarionte vegetal".

Assim, você pode editar uma célula eucarionte no código, por exemplo:

```
const tipoCelula = "eucarionte vegetal";
const temNucleo = 1;
const temMitocondria = 1;
const temCloroplasto = 1;
const temVacuolo = 1;
const temMembrana = 1;
const temParede = 1;
```

E depois, comparar com uma célula procarionte:

```
const tipoCelula = "procarionte";
const temNucleo = 0;
const temMitochondria = 0;
const temCloroplasto = 0;
const temVacuolo = 0;
const temMembrana = 1;
const temParede = 1;
```

Veja que as condições variam pela escolha do tipo celular e pela ativação ou desativação de estruturas. Em vez de alterar quantidades contínuas, como no script da [Figura 4.2](#), agora você compara configurações biológicas distintas. Por exemplo, uma célula procarionte sem núcleo, uma célula animal com mitocôndrias e membrana, ou uma célula vegetal com parede celular, cloroplastos e vacúolo. Dessa forma, você pode verificar que as partes da célula por si só não acrescentam muita coisa. O “chique” é comprovar o que muda na função quando certas estruturas aparecem ou desaparecem.

E o que muda na função depende também das condições que tornam essa organização possível, como o meio (água), a energia disponível, e a cooperação entre estruturas. Essas condições são simuladas no *app* de JSPlotly da [Figura 4.4](#) abaixo.

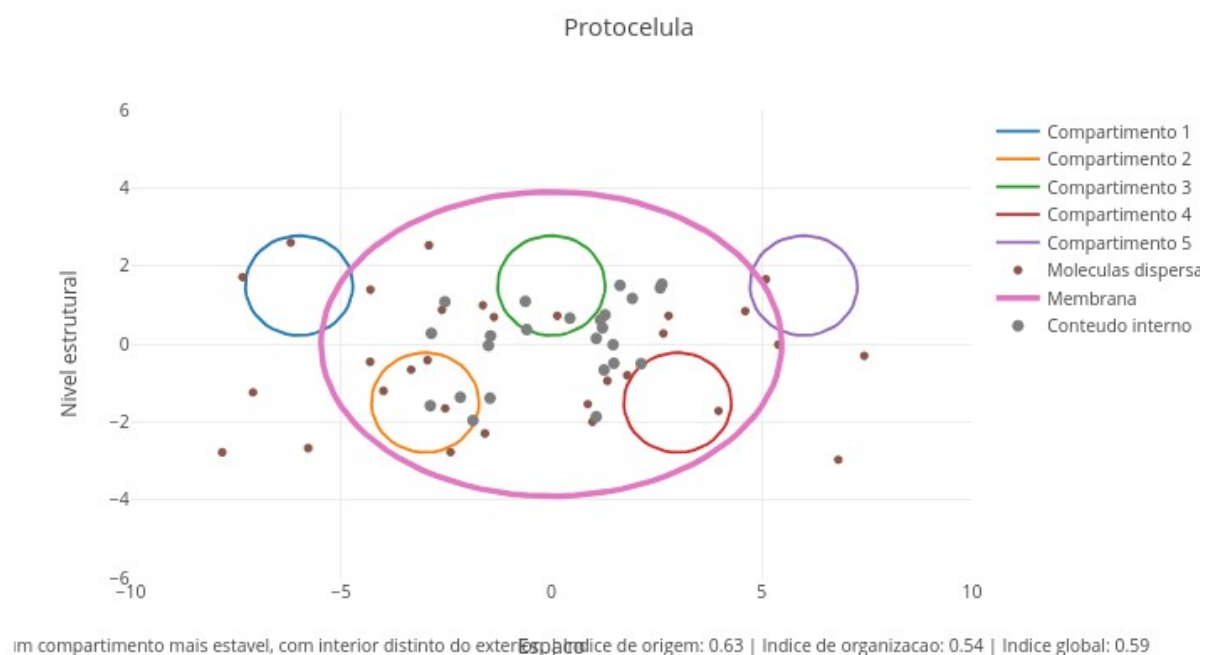


Figura 4.4: A vida não depende somente da matéria e energia, mas das condições que permitem uma organização estável e integrada. Clique neste [LINK](#) para abrir em nova aba.

Agite antes de usar

Esse script conecta o ambiente primitivo, a formação de compartimentos, e o surgimento de protocélulas, células, órgãos e tecidos. Tudo controlado

por parâmetros que representam condições do sistema. Para utilizá-lo, você controla dois aspectos principais:

- *Etapa da organização.* Varia numericamente entre Terra primitiva (0), compartimentos iniciais (1), protocélulas (2), células (3), tecido (4), e órgão (5).

```
const etapa = 0; // 0 a 5
```

- *Condições do sistema.* Aqui você seleciona valores numéricos para as seguintes situações:
- energia: intensidade de transformações;
- agua: meio para interação e estabilidade;
- membrana: formação de compartimentos;
- cooperacao: integração entre unidades;

Exemplificando:

```
const energia = 8;
const agua = 7;
const membrana = 3;
const cooperacao = 1;
```

Observe que, para algumas condições, nada se organiza, enquanto que para outras surgem compartimentos. Já para níveis mais altos de organização, torna-se essencial a *cooperação*. Assim, esse modelo mostra que a organização não depende só de componentes, mas das condições que permitem sua integração.

Que tal explorar alguns cenários ?

1. *Terra primitiva energética, mas desorganizada.*

```
const etapa = 0;
const energia = 9;
const agua = 6;
const membrana = 1;
const cooperacao = 0;
```

Perceba que nessa situação há muita atividade, mas pouca organização. Ou seja, energia sozinha não gera vida.

2. *Compartimentalização emergente.* Modifique o código como segue, e veja que surgem compartimentos estáveis.

```
const etapa = 1;
const energia = 7;
const agua = 8;
const membrana = 7;
const cooperacao = 1;
```

Podemos então considerar as membranas como um verdadeiro divisor de águas na evolução da vida.

3. *Protocélulas viáveis.* Edite o código como segue, e veja que

unidades mais estáveis aparecem, sugerindo que uma organização começa a persistir.

```
const etapa = 2;  
const energia = 7;  
const agua = 8;  
const membrana = 8;  
const cooperacao = 3;
```

4. *Células funcionais*. Pelos valores que seguem, pode-se observar que funções começam a se integrar.

```
const etapa = 3;  
const energia = 7;  
const agua = 8;  
const membrana = 8;  
const cooperacao = 6;
```

Essas funções são consequência natural da compartimentalização e da cooperação.

5. *Tecidos e órgãos*. Agora você tem uma integração entre unidades, revelando que a complexidade depende de cooperação.

```
const etapa = 4;  
const energia = 7;  
const agua = 8;  
const membrana = 8;  
const cooperacao = 9;
```

6. *Ambiente limitante*. Quando as condições ambientais limitam a organização, há chances de surgirem estruturas frágeis.

```
const etapa = 2;  
const energia = 2;  
const agua = 3;  
const membrana = 6;  
const cooperacao = 2;
```

Experimente também as situações-limite abaixo:

7. *Energia alta, sem compartimento*. Observe que agora há muita reação, mas nenhuma estabilidade, inibindo o surgimento da vida.

```
const energia = 9;  
const membrana = 0;
```

8. *Compartimento sem função*. Agora a estrutura não apresenta uma dinâmica clara. Então não dá pra saber se estamos diante de uma célula ou de uma “bolha” !

```
const membrana = 9;  
const energia = 2;
```

9. *Cooperação zero*. Nessa situação as unidades não se integram. Existe organismo sem cooperação ?

```
const cooperacao = 0;  
const etapa = 4;
```

10. *Pouca água*. Cenário com baixa interação, em função do papel central da água na formação da vida.

`const agua = 1;`

Em síntese, perceba que, ao variar energia, água, membrana e cooperação, nem toda combinação gera organização, nem toda organização é estável, e nem toda estabilidade gera complexidade. Por isso, a vida pode ser entendida como um processo que depende de um equilíbrio entre condições físicas, organização estrutural e integração funcional.

Volta pro mundo



Quando você observa um músculo contraindo, um neurônio transmitindo sinais, ou uma planta crescendo, você está vendo o resultado de múltiplos níveis de organização funcionando juntos. Podemos imaginar que essa organização é o que leva a algumas *propriedades emergentes* que definem o surgimento e a evolução da vida, mais do que somente uma organização em níveis e compartimentalização funcional. Nesse sentido, a vida começa a aparecer quando certas propriedades emergem da organização de estruturas.

Para explorar essa ideia, um JSPlotly literalmente “*mais animado*” !

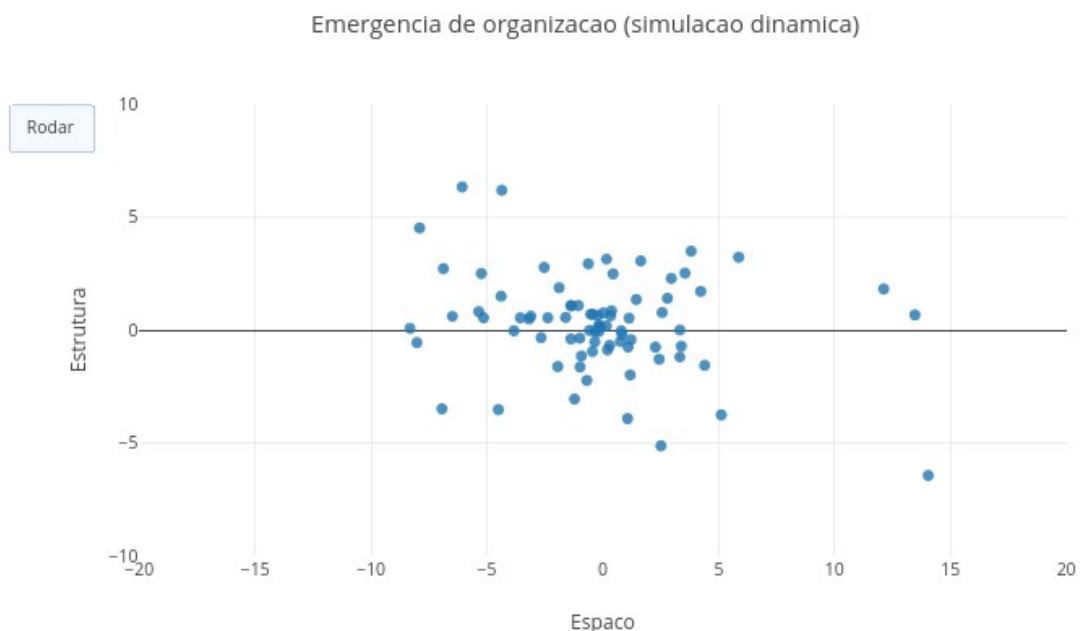


Figura 4.5: A vida pode surgir do equilíbrio entre ordem e desordem. Clique neste [LINK](#) para abrir o app em nova aba.

Agite antes de usar

Nesse novo modelo, o sistema evolui ao longo do tempo a partir de algumas regras definidas por você. A grande diferença em relação aos anteriores é que aqui você não define a forma final. Seguem os parâmetros iniciais do código:

- N: número de partículas;
- atracao: tendência de agregação;
- repulsao: evita colapso total;
- ruído: nível de agitação (pense numa analogia com temperatura);
- tempoTotal: duração da simulação.

Após ajustar algum parâmetro, clique em add e observe a dinâmica. O sistema pode apresentar comportamentos muito distintos, tais como dispersão completa, colapso em um único ponto, ou formação de agrupamentos estáveis. O mais interessante ocorre quando surgem *estruturas intermediárias*, nem totalmente caóticas nem totalmente rígidas. Para verificar isso, explore alguns cenários.

1. *Caos total*. As partículas não se organizam.

```
const atracao = 0.01;  
const repulsao = 0.01;  
const ruído = 1.0;
```

Então, o que garante a organização não é somente a matéria.

2. *Colapso total*. Tudo converge para um único ponto.

```
const atracao = 0.2;  
const repulsao = 0;  
const ruído = 0.2;
```

Então, a organização não é simplesmente “grudar o que aparece”.

3. *Zona de emergência*. Aqui surgem agrupamentos estáveis.

```
const atracao = 0.05;  
const repulsao = 0.02;  
const ruído = 0.3;
```

Então, a organização aparece em uma faixa intermediária de condições.

Agora, o que pode acontecer quando agrupamentos viram unidades autossuficientes, como *protocélulas* ?

Adivinha !!! Surge mais um JSPlotly !

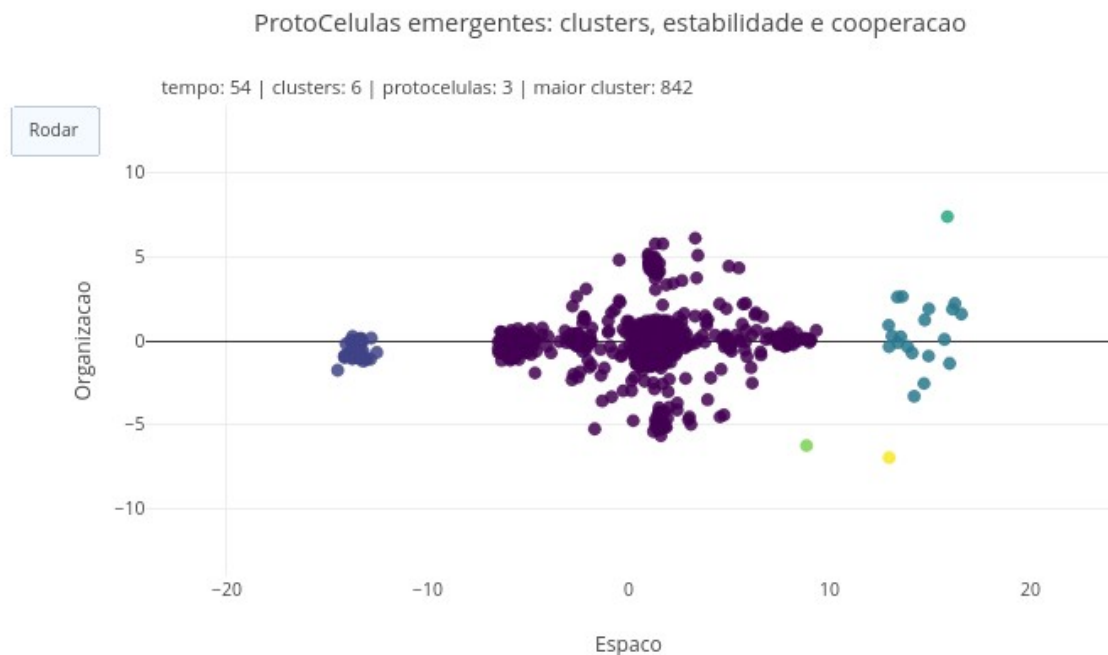


Figura 4.6: A vida pode começar quando uma organização passa a ter identidade e persistência. Clique neste [LINK](#) para um novo JSPlotly.

Agite antes de usar

Agora, o sistema identifica agrupamentos (*clusters*) e os destaca visualmente. Os parâmetros importantes dessa nova simulação são:

- `raioLigacao`: define proximidade entre partículas;
- `tamanhoProto`: tamanho mínimo para considerar um agrupamento;
- `atracao`, `repulsao`, `ruido`: mesmo significado do script da [Figura 4.5](#);

Nesse novo script, você poderá observar como pequenos encontros geram unidades estáveis. Também conseguirá ver que agrupamentos maiores tendem a persistir, até que alguns passem a se comportar como unidades. Cenários ? É pra já !!

1. *Zona fértil*. Aparecem múltiplas protocélulas.

```
const atracao = 0.045;
const repulsao = 0.018;
const ruido = 0.35;
const tamanhoProto = 6;
```

2. *Sistema instável*. Agrupamentos não se mantêm.

```
const ruido = 1.2;
```

E o que ocorre quando proto-organismos evoluem para *crescimento* e *seleção natural* ? Bom, acho que você já adivinhou !

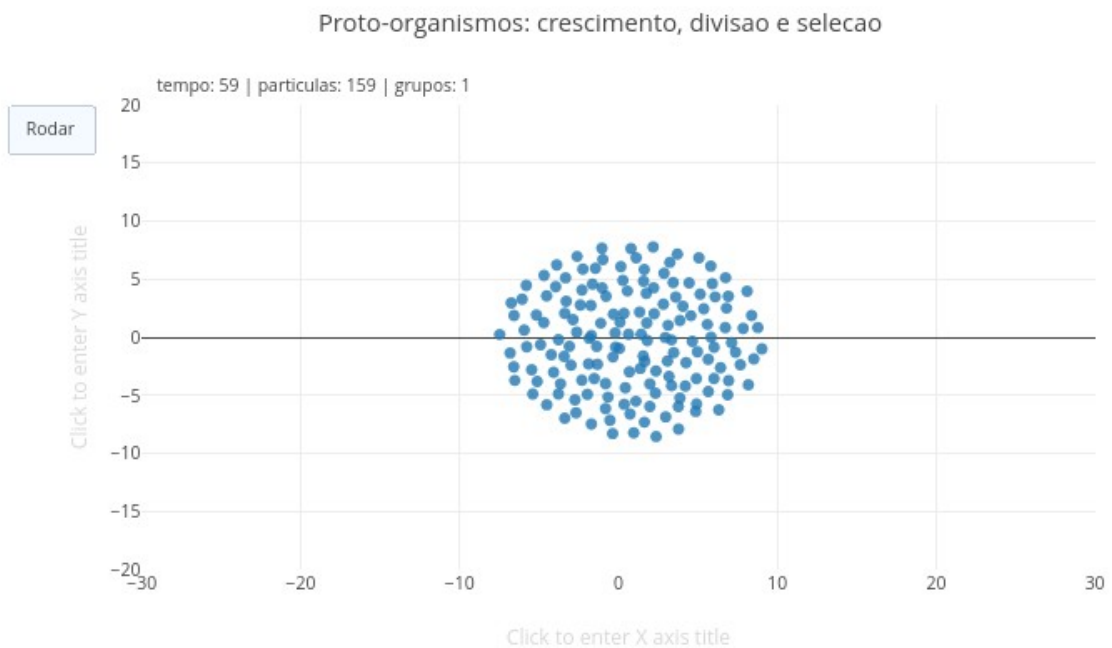


Figura 4.7: A evolução começa quando estruturas conseguem persistir, variar e se multiplicar. Clique neste [LINK](#) para um novo JSPlotly.

Agite antes de usar

Nesse modelo, os agrupamentos crescem, se dividem, e variam. Os parâmetros agora são mutacao, ruído, atracao, e repulsao.

Observe que alguns grupos crescem mais enquanto outros desaparecem. Observe também que surgem padrões de persistência nos agrupamentos. Cenários ?

1. Zona fértil. Crescimento e divisão.

```
const atracao = 0.05;  
const repulsao = 0.02;  
const ruído = 0.3;  
const mutacao = 0.4;
```

2. Alta mutação. Sistema instável e diverso.

```
const mutacao = 0.8;
```

E o que pode ocorrer quando agrupamentos persistentes passam a interagir num novo nível de organização ? Aí podem emergir verdadeiros ecossistemas. Bom...

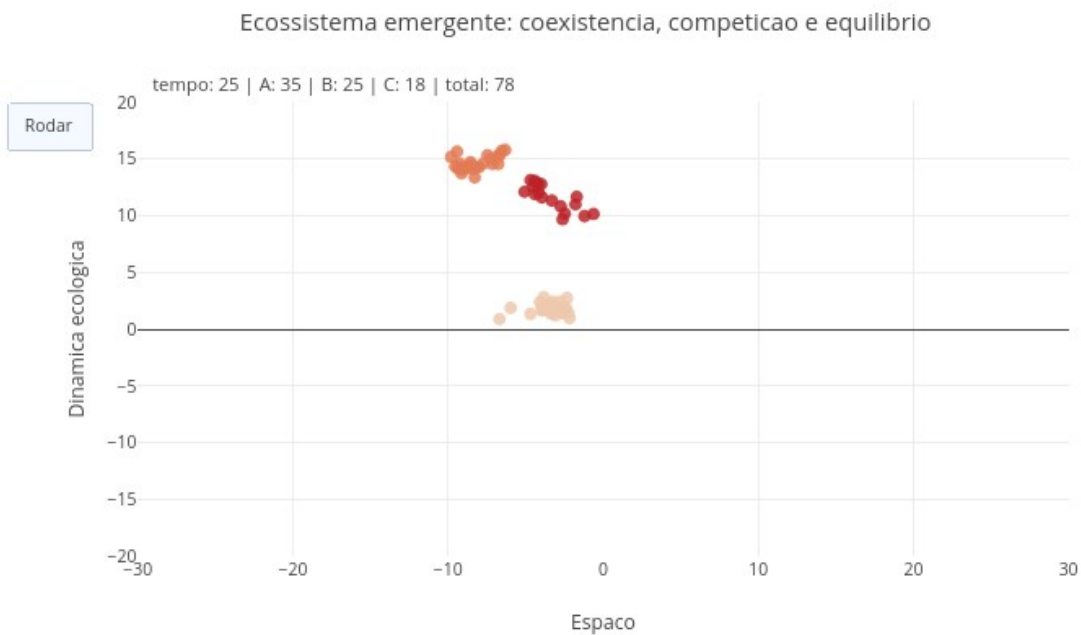


Figura 4.8: A vida não evolui isoladamente, mas depende de interações. Clique neste [LINK](#) para acessar o app.

Agite antes de usar

Nesse novo app há três tipos de entidades que interagem: A (mais estáveis), B (intermediárias) e C (competitivas). Os parâmetros para você editar no código são:

- nA, nB, nC;
- predacao;
- cooperacaoA;
- ruido;

Agora você poderá observar alguns fenômenos entre grupos, como coexistência ou colapso, dominância de grupos, ou equilíbrio dinâmico. Seguem cenários.

1. Equilíbrio.

```
const nA = 35;  
const nB = 25;  
const nC = 18;  
const predacao = 0.05;  
const cooperacaoA = 0.04;
```

2. Colapso competitivo. Aqui a diversidade reduz.

```
const predacao = 0.12;
```

“Se você chegou até aqui é porque....”. Bom, essa frase é clássica nos dias de hoje. Mas tenha certeza que terá valido a pena sua resiliência incontestemente em meio a tantos scripts ! Além de todo o conhecimento sobre o surgimento e evolução da vida auferido durante toda essa sua saga

peçoal, o final do arco-íris te aguarda com um pote de ouro !! Bem, ou quase isso !

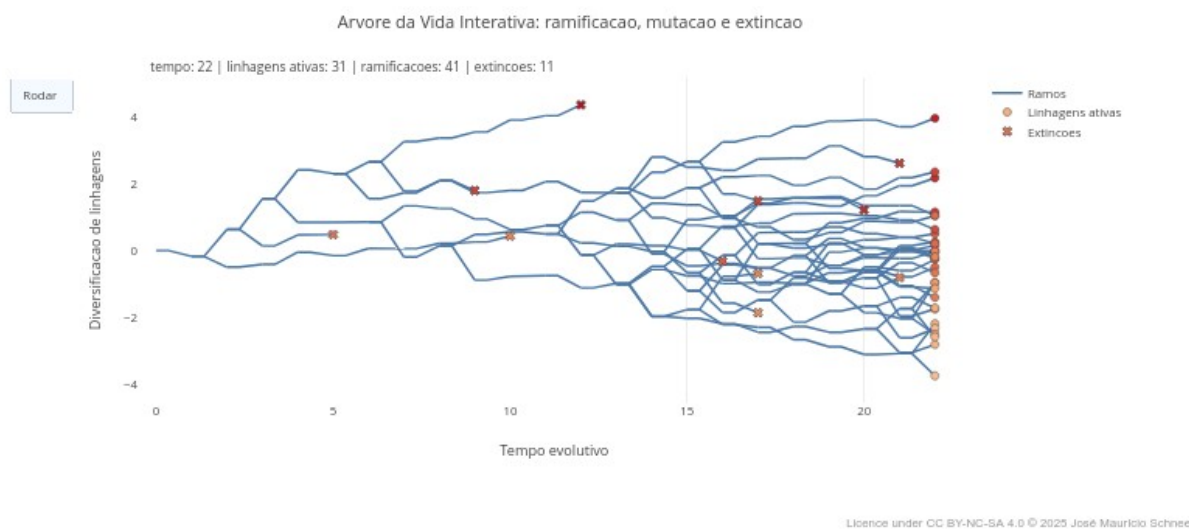


Figura 4.9: A evolução é ramificada, contingente e dinâmica. Clique neste [LINK](#) para acessar esse último (ufa!) aplicativo do capítulo.

Agite antes de usar

Nesse outro modelo, a história da vida é representada como uma *árvore dinâmica de linhagens*. Cada ramo pode persistir, se dividir em novos ramos, mudar ligeiramente de característica ou desaparecer. Dessa forma, a árvore aparece como resultado de processos simultâneos de continuidade, inovação e perda. O *script* favorece a compreensão de que a evolução não é uma escada linear, mas uma história ramificada, contingente e marcada tanto por diversificação quanto por extinção.

Você construir uma verdadeira “*árvore da vida*” com o *script*, e decidir como determinados organismos evoluem para ramificar em novos grupos (*chanceRamificacao*), e derivados ou não de episódios de mutação em seu DNA (*intensidadeMutacao*). Ou mesmo como regridem à completa extinção (*chanceExtincao*). Tudo na mais santa paz, é claro !

Seguem dois cenários para você para explorar o aplicativo.

1. Árvore da vida exuberante.

```
const chanceRamificacao = 0.22;
const chanceExtincao = 0.08;
```

2. Mundo hostil.

```
const chanceExtincao = 0.18;
```

Observe que linhagens podem surgir ou desaparecer. Veja também que a diversidade varia ao longo do tempo, por vezes seguindo algum padrão. E que padrões não são lineares na evolução ! Eles têm uma história. Podem surgir a partir de sistemas físico-químicos, passando por organização em níveis estruturais, compartimentalização e funcionalidade, interação entre unidades e transformação ao longo do tempo. Enfim, a vida está muito



E SE...

Que tal agora integrar os *apps* todos nesta seção de sublime abstração mental e perfusão oxigenada ?

- E se não houvesse membranas capazes de separar interior e exterior ?
- E se existissem moléculas complexas, mas sem compartimentalização ?
- E se as primeiras células não cooperassem entre si ?
- E se a vida tivesse surgido em ambientes muito diferentes dos oceanos primitivos ?
- E se os organismos permanecessem sempre unicelulares ?
- E se tecidos não gerassem propriedades novas em relação às células isoladas ?
- E se houver energia, mas nenhuma compartimentalização ?
- E se houver organização, mas nenhuma cooperação ?
- E se houver crescimento, mas nenhuma variação ?
- E se houver competição extrema ?
- E se não houver extinção ?



MESMO PADRÃO, OUTROS MUNDOS

A vida pode ser compreendida não apenas como matéria, mas como matéria organizada, delimitada, funcional e capaz de cooperar em múltiplos níveis. Das hipóteses sobre a Terra primitiva até a formação de tecidos e órgãos, o que se observa é uma história de compartimentalização, integração e emergência. Compreender a organização dos seres vivos é também compreender como novas propriedades surgem quando unidades menores passam a atuar em conjunto.

E essas mesmas regras são vistas também na biologia de sistemas, na teoria da complexidade, na modelagem computacional, em redes neurais, na computação quântica, e na comunicação, entre vários outros. Em relação a esse “outro mundo”, também os *Grandes Modelos de Linguagem* dos assistentes de Inteligência Artificial - LLMs ("Large Language Models"). Essas estruturas algorítmicas são treinadas em volumes massivos de dados para compilação estatística, compreensão e geração de texto humanizado, podendo traduzir idiomas, criar objetos multimidiáticos, e realizar tarefas de processamento de linguagem natural.



POR DENTRO DA FERRAMENTA

No capítulo foram abordados diversos temas e *apps* de JSPlotly correlatos para uma “*mão na massa*” sobre os conceitos que definem a vida e sua evolução no planeta. Fugindo da *praxis* dessa seção, a de apresentar a lógica de programação de um determinado script, preferiu-se resumir “*evolutivamente*” todos os códigos apresentados.

Script 1: níveis de organização da matéria viva; Script 2: célula como sistema compartimentalizado;

Script 3: condições que tornam a vida possível; Script 4: organização emergente a partir de interações; Script 5: surgimento de unidades (protocélulas);

Script 6: persistência, variação e seleção (pré-evolução);

Script 7: interações entre formas de vida (ecossistema);

Script 8: diversificação e história das linhagens (árvore da vida).

Perceba que essa sequência também apresenta, ainda que timidamente, um fluxo evolutivo para a vida. Mas, pra não ficarmos apenas no blá-blá-blá, um último script de JSPlotly, esse uma releitura de um aplicativo muito famoso nos anos 1970 !

Jogo da Vida de Conway

O *Jogo da Vida* é um autômato celular criado pelo matemático britânico John Horton Conway em 1970. Baseia-se na construção de máquinas autorreprodutivas, tendo sido popularizado pela revista *Scientific American*. Trata-se de um “jogo sem jogadores” que ocorre em um tabuleiro quadriculado infinito, e onde a evolução de cada célula (viva ou morta) é determinada inteiramente por seu estado inicial e por quatro regras simples de vizinhança. Essas regras simulam processos de *nascimento*, *sobrevivência* e *morte* por isolamento ou superpopulação. Apesar da simplicidade das regras, o sistema é capaz de gerar padrões complexos e imprevisíveis, funcionando como um modelo matemático para estudar sistemas dinâmicos e emergência na computação.

E...é claro...também passível de ser apresentado em códigos pelo JSPlotly !

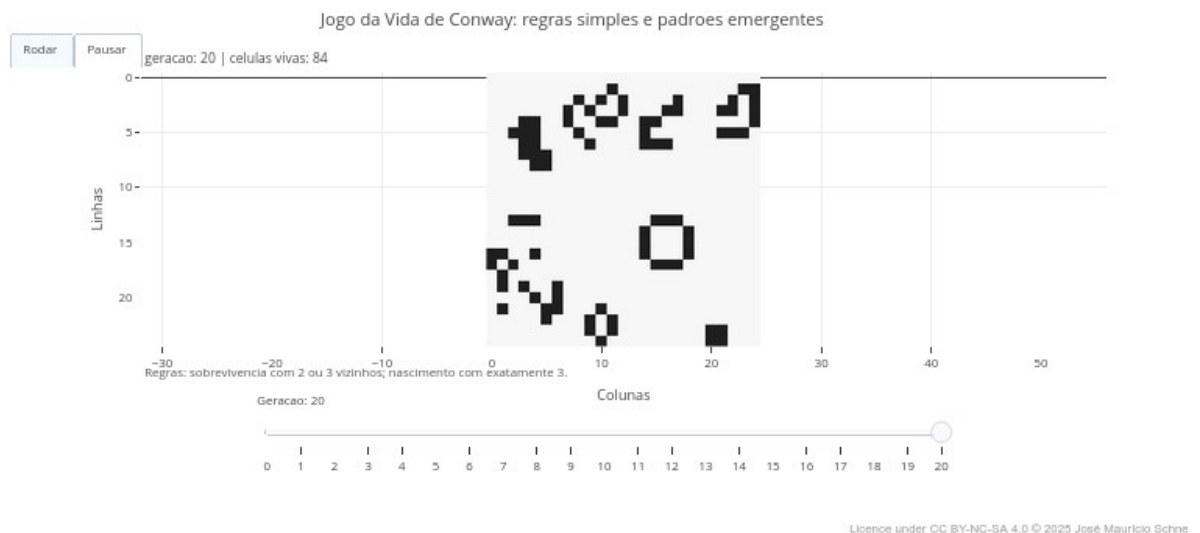


Figura 4.10: Jogo da Vida de Conway - padrões surgidos de regras simples e sem definição prévia. Clique com o botão direito neste [LINK](#) e abra o jogo em nova aba.

O Jogo da Vida de Conway não é um modelo histórico da origem da vida, mas uma metáfora computacional extremamente útil. A partir de regras locais muito simples, surgem padrões estáveis, oscilatórios ou móveis. Isso ajuda a introduzir a ideia de emergência: formas complexas podem emergir de regras muito simples, e sem que tenham sido desenhadas diretamente desde o início.

No Jogo da Vida da [Figura 4.10](#) você altera apenas um bloco:

```
const linhas = 25;
const colunas = 25;
const geracoes = 20;
const densidadeInicial = 0.28;
const padrao = "aleatorio";
Padrões sugeridos
const padrao = "aleatorio";
```

Cada posição da grade pode assumir dois estados: 0 (célula morta), ou 1 (célula viva). A dinâmica do sistema não depende de um “controle central”, mas apenas de regras locais aplicadas a cada célula.

As regras do sistema

A cada geração, o estado de cada célula depende apenas do número de vizinhos vivos ao seu redor. Uma célula viva sobrevive se tiver *2 ou 3 vizinhos vivos*, e uma célula morta passa a viver se tiver *exatamente 3 vizinhos vivos*. Essas regras estão implementadas na função:

```
function proximaGeracao(g) {
  const nova = criarGradeVazia(g.length, g[0].length);

  for (let i = 0; i < g.length; i++) {
    for (let j = 0; j < g[0].length; j++) {
```

```

    const vivos = contarVizinhos(g, i, j);

    if (g[i][j] === 1) {
      nova[i][j] = (vivos === 2 || vivos === 3) ? 1 : 0;
    } else {
      nova[i][j] = (vivos === 3) ? 1 : 0;
    }
  }
}

return nova;
}

```

Observe que, diferentemente dos scripts anteriores, não há nenhuma instrução dizendo “forme estruturas” ou “crie padrões”. Os padrões surgem naturalmente. O sistema começa com uma grade inicial, e que pode ser aleatório ou um padrão específico (glider-movível, blinker-oscilatório, block-estável).

```
const padrao = "aleatorio";
```

A cada passo conta-se o número de vizinhos vivos, aplica-se a regra local, e gera-se uma nova grade. Esse processo é repetido ao longo do tempo, formando uma sequência de gerações.

Como a animação é construída

Cada estado da grade é armazenado como um *frame*:

```

frames.push({
  name: "g" + t,
  data: [{ z: gradeParaHeatmap(grade), type: "heatmap" }]
});

```

Esses frames são exibidos com um botão *Rodar/Pausar* e um *slider temporal*, possibilitando uma exploração do modelo.

Mesmo sem representar moléculas, células ou organismos reais, o modelo é capaz de mostrar que estruturas podem surgir sem serem programadas diretamente. Além disso, também permite observar que padrões podem persistir, oscilar ou se deslocar, e que *pequenas regras locais geram comportamentos globais complexos*.

Em suma, o modelo do Jogo da Vida de Conway é um exemplo clássico de como regras extremamente simples podem gerar comportamentos complexos.

5. Vida em rede: Microbiologia, Botânica e Zoologia



O MUNDO TE CHAMA

É realmente bem fácil perceber animais e plantas ao seu redor. Eles ocupam o quintal, a praça, a horta, a mata, o aquário, a mesa de casa e até o próprio corpo humano, quando pensamos em alimentação, saúde e ambiente. Parte desses organismos são invisíveis por seu tamanho, os microrganismos, e também habitam solos, água, ar, alimentos e nosso corpo.

Essa quantidade de seres vivos, entre micróbios, plantas e animais, definem os campos de estudo da Microbiologia, Botânica e Zoologia, respectivamente. Microrganismos participam de processos que sustentam ou transformam os ecossistemas, decompondo a matéria e reciclando nutrientes. Plantas capturam energia luminosa e produzem biomassa. E animais interagem com plantas, com micróbios e com outros animais em teias complexas de alimentação, defesa, simbiose (interações) e equilíbrio ecológico, tal como já visto nesse livro.

Mas esses grupos não existem de forma independente. O solo fértil depende da ação microbiana. A planta depende de água, minerais, luz e, muitas vezes, de interações com fungos e bactérias. O animal depende direta ou indiretamente das plantas, e também convive com comunidades microbianas internas e externas.

Embora sejam áreas distintas e que costumam aparecer separadas, o mundo não faz distinção entre elas. Estudar esses três ramos em conjunto permite perceber padrões que atravessam toda a vida, como organização celular, nutrição, reprodução, adaptação, trocas com o ambiente, relações ecológicas e diversidade de formas. A proposta deste capítulo é integrá-los para se enxergar conexões e compreender a vida como uma rede.



Figura 5.1: Vida não é um ser vivo isolado. Cada célula participa de um organismo que participa de uma população que participa de ecossistemas, os quais compõem a biosfera.

Depois de explorar, você consegue:

- reconhecer diferenças e aproximações entre microrganismos, plantas e animais;
- comparar formas de organização corporal e modos de vida;
- relacionar nutrição, respiração, reprodução e adaptação em diferentes grupos;
- interpretar o papel ecológico de bactérias, fungos, vegetais e animais;
- explorar modelos computacionais interativos que conectam os três ramos da Biologia



Como explicado acima, a ideia aqui não é classificar todos os organismos com precisão taxonômica, mas criar um mapa para perceber como microrganismos, plantas e animais podem ocupar posições diferentes numa rede viva. Para iniciar o capítulo nessa direção, um JSPlotly criado como um “classificador vivo” simples.

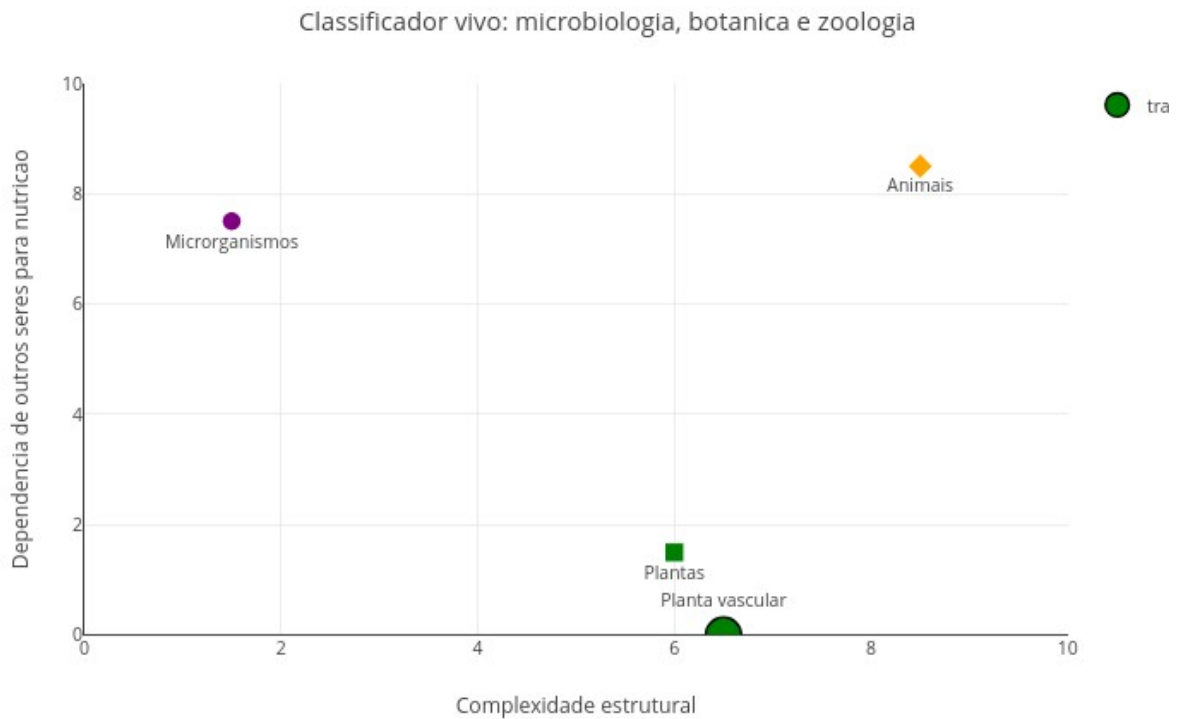


Figura 5.2: A vida em rede começa quando percebemos que produtores, consumidores e decompositores formam um sistema integrado. Clique com o botão direito do mouse neste [LINK](#) e abra-o numa nova aba.

Agite antes de usar

O script compara diferentes grupos de seres vivos por complexidade estrutural, autotrofia, mobilidade e papel ecológico.

Altere os valores iniciais do código:

```
const grupo = "Planta vascular";
const complexidade = 6.5;
const autotrofia = 1.0;
const mobilidade = 0.0;
const papel_ecologico = "produtor";
```

A variável complexidade indica o grau de organização estrutural. A variável autotrofia indica o quanto o organismo produz seu próprio alimento. A variável mobilidade altera o tamanho do marcador no gráfico. E o papel_ecologico, que estabelece a posição que um organismo ocupa em uma cadeia alimentar, é definido pela cor do ponto:

```
if (papel === "produtor") return "green";
```

```
if (papel === "consumidor") return "orange";
if (papel === "decompositor") return "brown";
```

O eixo horizontal representa a complexidade estrutural, enquanto que o vertical a dependência de outros seres para nutrição.

Essa dependência é calculada assim:

```
const dependencia = 10 - autotrofia * 10;
```

Desse modo, quanto maior a autotrofia, menor a dependência trófica. Por isso, plantas tendem a aparecer mais abaixo no gráfico, enquanto animais, que dependem de outros seres para obter matéria orgânica, aparecem mais acima. Agora você pode explorar alguns cenários.

1. *Bactéria decompositora*. Decompositores podem ter baixa complexidade estrutural, embora exerçam papel ecológico gigantesco, reciclando a matéria.

```
const grupo = "Bacteria decompositora";
const complexidade = 1.2;
const autotrofia = 0.1;
const mobilidade = 0.3;
const papel_ecologico = "decompositor";
```

2. *Planta vascular*. Plantas podem ter alta complexidade e baixa mobilidade, mas sustentam redes alimentares pela produção de matéria orgânica.

```
const grupo = "Planta vascular";
const complexidade = 6.5;
const autotrofia = 1.0;
const mobilidade = 0.0;
const papel_ecologico = "produtor";
```

3. *Animal vertebrado*. Animais tendem a apresentar alta mobilidade e alta dependência trófica, pois precisam obter alimento de outros organismos.

```
const grupo = "Animal vertebrado";
const complexidade = 8.5;
const autotrofia = 0.0;
const mobilidade = 1.0;
const papel_ecologico = "consumidor";
```

4. *Fungo decompositor*. Fungos mostram que baixa mobilidade não significa pouca importância ecológica.

```
const grupo = "Fungo decompositor";
const complexidade = 4.5;
const autotrofia = 0.0;
const mobilidade = 0.1;
const papel_ecologico = "decompositor";
```

Usando o *app*, você consegue perceber que cada grupo, microorganismo, planta ou animal, pode ser compreendido por sua organização, forma de nutrição, mobilidade e função ecológica. Plantas produzem matéria orgânica, enquanto animais consomem e se deslocam, e microrganismos e fungos reciclam matéria. E todos participam de uma rede de relações.



FAZENDO APARECER

Ao comparar formas de vida, você pode listar nomes ou grupos taxonômicos. Mas talvez seja mais produtivo observar dimensões comparáveis entre eles, como a organização celular, presença de tecidos, mobilidade, nutrição, reprodução, interação com o ambiente e função ecológica.

Microrganismos mostram que a vida pode ser extremamente eficiente, mesmo sem grande complexidade corporal, enquanto as plantas mostram como a fixação ao substrato ocorre com uma boa dose de sofisticação fisiológica. Já os animais, esses mostram como mobilidade, sensibilidade e integração ao ambiente abrem portas para outras possibilidades adaptativas.

Para ilustrar essas relações, segue mais um *app*. No primeiro script de JSPlotly comparamos seres vivos por sua complexidade, mobilidade, modo de nutrição e papel ecológico. Agora vamos utilizar um *radar comparativo* entre grupos de seres vivos.

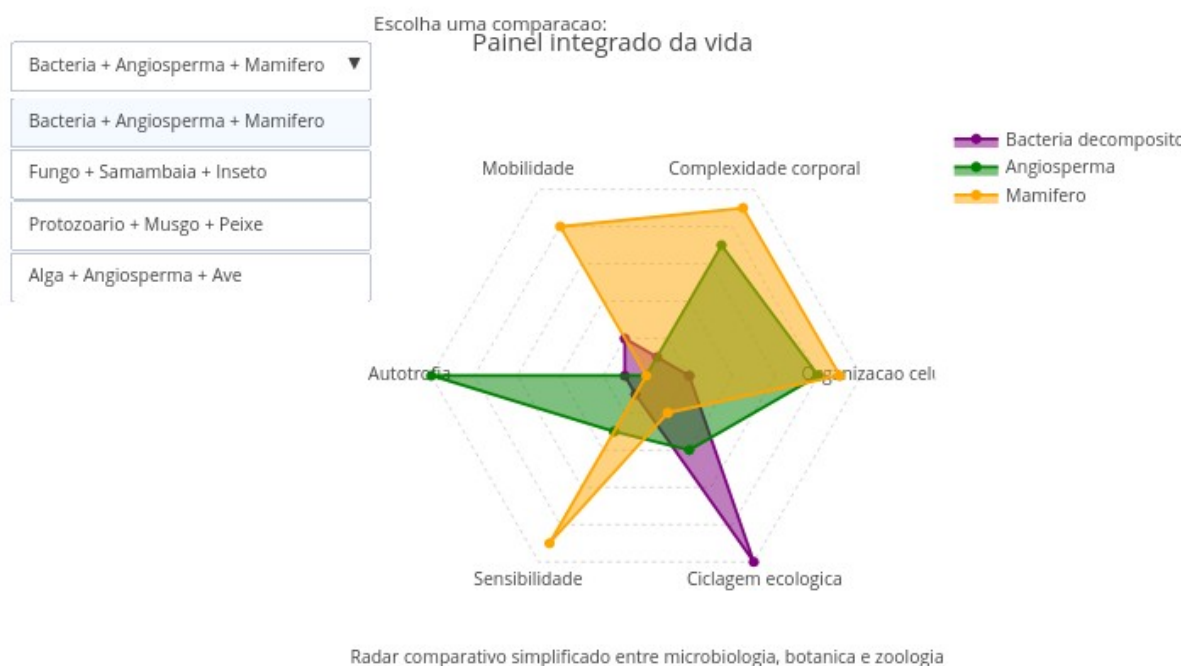


Figura 5.3: Microrganismos, plantas e animais são formas diferentes de resolver problemas biológicos. Clique neste [LINK](#) para abrir o app.

O *app* compara diversos grupos como bactéria, protozoário, fungo, alga, musgo, samambaia, angiosperma, inseto, peixe, ave e mamífero. Para

isso, ele utiliza seis dimensões: organização celular, complexidade corporal, mobilidade, autotrofia, sensibilidade e ciclagem ecológica.

Agite antes de usar

Você pode comparar os diferentes grupos de seres vivos por meio do painel em forma de radar simplificado da [Figura 5.3](#). Cada eixo representa uma dimensão biológica:

- organização celular;
- complexidade corporal;
- mobilidade;
- autotrofia;
- sensibilidade;
- ciclagem ecológica.

Use o *menu suspenso* para alternar entre diferentes combinações de organismos. O objetivo não é produzir uma classificação taxonômica rigorosa, mas perceber como microbiologia, botânica e zoologia podem ser comparadas a partir de características funcionais.

Por exemplo, a combinação inicial compara:

```
const gruposIniciais = ["bacteria", "angiosperma", "mamifero"];
```

Ela coloca lado a lado uma bactéria decompositora, uma planta com flor e um mamífero. O banco de dados interno do script contém os perfis de cada grupo. Por exemplo:

```
bacteria: {
  nome: "Bacteria decompositora",
  valores: [2, 1, 2, 1, 1, 10]
}
```

Esses valores correspondem, na ordem, às dimensões do radar:

```
const dimensoes = [
  "Organizacao celular",
  "Complexidade corporal",
  "Mobilidade",
  "Autotrofia",
  "Sensibilidade",
  "Ciclagem ecologica"
];
```

Assim, a bactéria aparece com baixa complexidade corporal, mas com alto papel na ciclagem ecológica. Já uma angiosperma aparece com alta autotrofia e boa complexidade estrutural:

```
angiosperma: {
  nome: "Angiosperma",
  valores: [8, 7, 0, 10, 3, 4]
}
```

E um mamífero aparece com alta complexidade, mobilidade e sensibilidade, mas baixa autotrofia:

```
mamifero: {
  nome: "Mamifero",
  valores: [9, 9, 8, 0, 9, 2]
}
```

Alguns cenários interessantes são propostos para auxiliar em seu aprendizado.

1. *Bactéria + angiosperma + mamífero*. Compare três grandes modos de vida: decomposição, produção e consumo. A bactéria se destaca na ciclagem ecológica, a angiosperma na autotrofia, e o mamífero na mobilidade e sensibilidade.
2. *Fungo + samambaia + inseto*. Observe como um decompositor, uma planta vascular sem sementes e um animal invertebrado ocupam perfis ecológicos diferentes. O fungo contribui fortemente para reciclagem, a samambaia para produção, e o inseto para mobilidade e interação.
3. *Protozoário + musgo + peixe*. Essa comparação ajuda a perceber que organismos aquáticos, simples ou complexos, podem ter estratégias muito diferentes de nutrição, locomoção e sensibilidade.
4. *Alga + angiosperma + ave*. Aqui aparecem dois produtores fotossintetizantes, em níveis diferentes de complexidade, e comparados a um animal altamente móvel.

O radar mostra que nenhum grupo é “melhor” em tudo. Cada forma de vida ocupa um perfil próprio, com forças e limitações. Assim, o script ajuda a mostrar que obter energia, manter organização, interagir com o ambiente, perceber estímulos e participar da ciclagem da matéria, são papéis desempenhados pela “equipe” toda da microbiologia, botânica e zoologia e, mais do que nomes de grupos, são conectados por funções e relações ecológicas.

VOLTA PRO MUNDO



As ideias deste capítulo aparecem o tempo todo no mundo, mesmo quando não as percebemos. Em um jardim, em uma floresta, numa plantação, num lago ou no intestino humano, a vida é sempre coletiva. Microrganismos participam da decomposição do lixo orgânico, da produção de alimentos fermentados, da digestão e até da reciclagem de nutrientes no solo.

Plantas sustentam cadeias alimentares, regulam ciclos biogeoquímicos e transformam energia luminosa em matéria orgânica. Animais redistribuem energia, interagem em cadeias tróficas e modificam ecossistemas

continuamente. Desse modo, plantas, animais e microrganismos se cruzam em trocas permanentes. Retirar um desses componentes do quadro não apenas empobrece a paisagem, mas altera ciclos, relações tróficas, decomposição, fertilidade, produtividade e saúde.

Todos os organismos dependem de células organizadas, metabolismo, troca de matéria e interação ecológica. Quando um alimento estraga, quando uma floresta cresce, quando um lago sofre contaminação ou quando uma população desaparece, estamos observando relações biológicas em rede, e não em partes isoladas.

Dessa forma, conclui-se também que microrganismos, plantas e animais formam sistemas conectados por fluxo de energia, circulação de nutrientes, relações ecológicas, equilíbrio ambiental, e transformação contínua da matéria. Os dois primeiros *apps* do capítulo evidenciam esse fato de que organismos possuem funções ecológicas e que são as células que organizam essas funções. Mas, e o que dizer sobre o que conecta todos os grupos vivos em uma dinâmica comum nos ecossistemas ?

Esse é o assunto do próximo JSPlotly, um *app* literalmente “*mais animadinho*” !

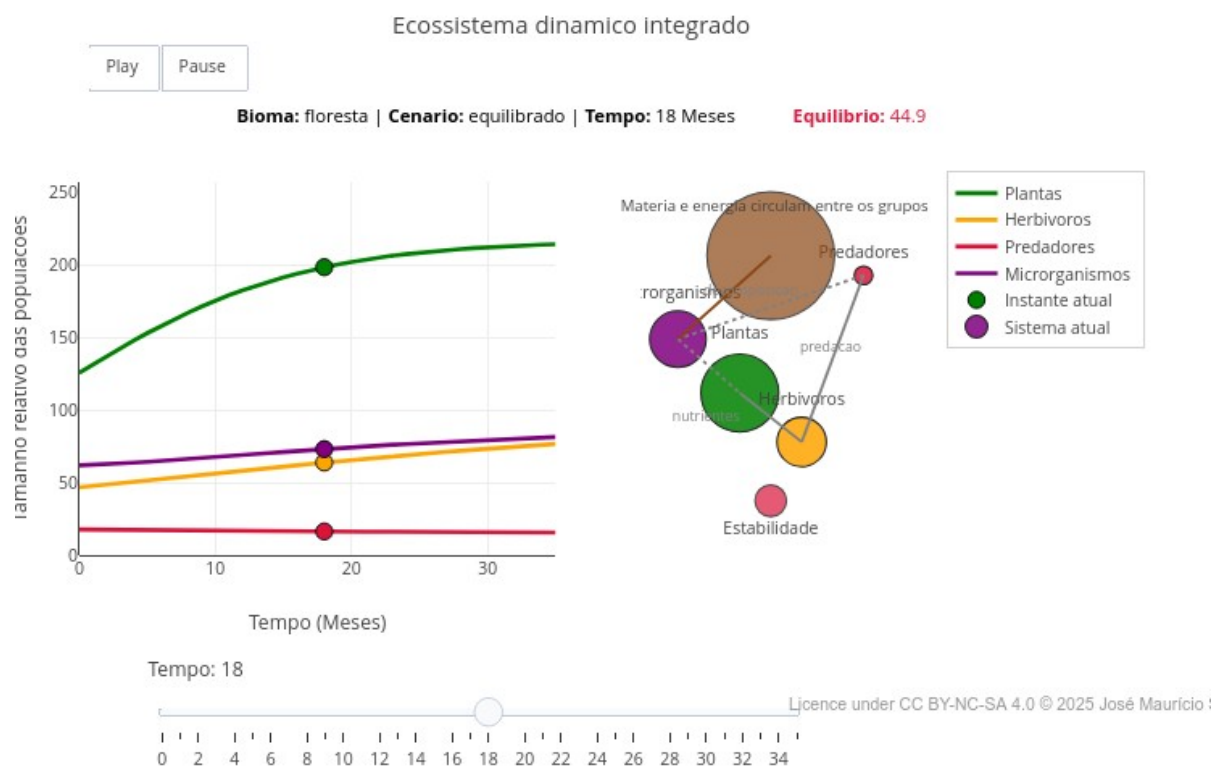


Figura 5.4: A vida em rede começa dentro da célula e expande sua ideia de organização para seres vivos, populações e ecossistemas. Clique neste [LINK](#) para abrir o *app*.

Agite antes de usar

O script integra microbiologia, botânica e zoologia em um mesmo sistema dinâmico. Plantas, herbívoros, predadores e microrganismos mudam

juntos ao longo do tempo, formando uma rede ecológica interdependente. O gráfico agora constitui uma animação no tempo, mostrando como as populações variam. Já o painel ecológico instantâneo representa visualmente as relações entre os grupos.

Altere principalmente:

```
const bioma = "floresta";
const cenario = "equilibrado";
```

Para a primeira variável, bioma, você pode experimentar “floresta”, “campo”, “lago”, e “cerrado”. Já para segunda variável, cenario, você pode tentar “seca”, “contaminacao”, e “recuperacao”.

Depois utilize os *widgets* da área gráfica, que são ajustes interativos, como a barra deslizante de tempo (*slider*), os botões Play/Pause, e as intensidades ambientais.

Observe que as populações não mudam isoladamente. Quando plantas diminuem, herbívoros podem sofrer redução. Quando decompositores aumentam, nutrientes retornam ao sistema. Quando a contaminação cresce, vários grupos podem ser afetados ao mesmo tempo. Esse modelo computacional simplificado trabalha com algumas ideias presentes em interações ecológicas, como produtores, consumidores, predadores, decompositores, detritos, e estabilidade ecológica.

No modelo, o crescimento das plantas, por exemplo, depende de clima, do suporte ambiental e da competição, e é inserido nesta parte do código:

```
const crescimentoPlantas =
  cfg.prodBase *
  clima *
  f.impactoPlantas *
  P *
  (1 - P / cfg.suportePlantas);
```

Já os decompositores ajudam a devolver nutrientes ao sistema:

```
const retornoNutrientes =
  0.032 * D * (M / (M + 25)) + 7 * rec;
```

Seguem alguns cenários para você experimentar.

1. *Floresta equilibrada*. Ecossistemas equilibrados apresentam oscilações, mas conseguem manter estabilidade relativa ao longo do tempo.

```
const bioma = "floresta";
const cenario = "equilibrado";
```

2. *Seca*. Mudanças climáticas podem reduzir produtores e afetar toda a cadeia ecológica.

```
const bioma = "cerrado";
const cenario = "seca";
```

3. **Contaminação**.

```
``js  
const bioma = "lago";  
const cenario = "contaminacao";
```

4. *Recuperação ambiental*. Ecossistemas possuem capacidade de regeneração, especialmente quando processos ecológicos são restaurados.

```
const bioma = "campo";  
const cenario = "recuperacao";
```

5. *Reforço microbiano*. Decompositores podem acelerar reciclagem de nutrientes e favorecer recuperação do sistema.

```
const reforcoMicrobiano = 20;
```

Veja que os ecossistemas funcionam como redes dinâmicas. Plantas produzem matéria orgânica (capturam energia) que é consumida por herbívoros (redistribuem energia). Predadores regulam populações, enquanto decompositores reciclam matéria e devolvem nutrientes ao ambiente, resultando em equilíbrio ecológico.

E SE...



Agora é hora de ir além, e deixar o pensamento sair do corpo !

- E se os decompositores desaparecessem de um ecossistema ?
- E se houver decompositores abundantes, mas poucos produtores ?
- E se plantas perdessem a capacidade de fotossíntese ?
- E se organismos extremamente simples dominassem um ambiente inteiro ?
- E se uma cadeia alimentar perdesse seus predadores ?
- E se aumentássemos drasticamente a contaminação ambiental ?
- E se o número de herbívoros crescesse muito rapidamente ?
- E se um ecossistema tivesse alta biodiversidade, mas baixa estabilidade ?
- E se toda a produção de matéria orgânica dependesse apenas de microrganismos ?
- E se um ambiente tivesse produtores, mas quase nenhum decompositor ?
- E se um organismo tivesse alta complexidade estrutural, mas baixa integração ecológica ?
- E se microrganismos simbióticos desaparecerem de plantas ou animais ?
- E se uma população animal crescer muito acima da capacidade do ambiente ?
- E se um pequeno desequilíbrio ecológico se propagasse por toda a rede viva ?
- E se organismos muito diferentes ocupassem papéis ecológicos semelhantes ?
- E se um ecossistema inteiro dependesse de apenas uma espécie dominante ?



Os padrões mostrados nesse capítulo aparecem em muitos contextos reais, já que o planeta Terra abriga *“uma célula em cada esquina”*. Assim é para a ação de microrganismos na produção de alimentos durante a fermentação, ou de decompositores reciclando a matéria orgânica na compostagem.

Ou a microbiota humana, nome que se dá ao conjunto de microrganismos integrado ao metabolismo do nosso corpo (antigamente chamado de microflora), e que, pasme, é quase três vezes maior em número do que nossas próprias células somáticas !! E o que dizer da intervenção humana danosa sobre diversos biomas, como o desmatamento que promove alterações drásticas nas relações ecológicas, ou as queimadas, que impactam simultaneamente a flora, a fauna, e a microbiota dos solos.

POR DENTRO DA FERRAMENTA



Os scripts desse capítulo não pretendem reproduzir a complexidade taxonômica da Biologia. Muito longe disso, seu papel é oferecer um modelo comparativo, manipulável e visual. Ao mudar poucas variáveis, você percebe que conceitos biológicos podem ser traduzidos em parâmetros, relações e padrões. Essa passagem entre a linguagem da Biologia e a linguagem computacional pode ajudar-lhe a desenvolver um raciocínio analítico temperado com pensamento computacional por modelagem e leitura crítica de simplificações.

Por falar em modelagem computacional, que tal aprender um pouco como funciona o script da [Figura 5.3](#) pelo pseudocódigo abaixo ?

INICIAR banco de organismos

armazenar diferentes grupos vivos:

bactérias
fungos
algas
plantas
animais

definir valores para:

organização celular
complexidade corporal
mobilidade
autotrofia
sensibilidade
ciclagem ecológica

CRIAR lista de dimensões biológicas

GERAR estrutura do radar

- criar ângulos igualmente distribuídos
- associar cada dimensão a um eixo

PARA cada organismo selecionado

- obter valores biológicos
- converter valores em coordenadas x e y
- fechar polígono do radar
- criar gráfico do organismo:
 - linhas
 - marcadores
 - preenchimento interno

FIM

CRIAR grade de referência

- desenhar círculos concêntricos
- indicar níveis de intensidade

CRIAR rótulos das dimensões

- posicionar textos ao redor do radar

CONFIGURAR menu interativo

- permitir troca entre conjuntos de organismos

ATUALIZAR gráfico

- mostrar organismos selecionados
- atualizar título comparativo

CONFIGURAR layout final

- esconder eixos cartesianos
- manter proporção circular
- adicionar legenda e instruções

RETORNAR

- dados do radar
- layout do gráfico

6. Metabolismo e Fisiologia: Energia, Equilíbrio e Vida



O MUNDO TE CHAMA

Você já percebeu que seu corpo nunca para ? Até quando você está “parado” ele gasta energia, mesmo dormindo. E se você acha que são milhões de reações (bio)químicas acontecendo ao mesmo tempo, engana-se: são *septilhões de reações por segundo*, ou até mais ! Sabe quanto é isso ? 10^{24} reações em um único segundo ! Um valor maior até que o *número de Avogadro*, usado para representar 1 mol de qualquer substância ($6,02 \times 10^{23}$ moléculas). A esse conjunto imenso de reações dá-se o nome de metabolismo. É o metabolismo que permite que uma célula “queime” açúcar, produza proteínas e armazene gordura. Ou que fabrique açúcar, degrade proteínas, e “queime” gorduras. Tudo sendo controlado por hormônios e outras substâncias com ação sobre os tecidos. Essas transformações todas não são aleatórias, e sim controladas e integradas para que o corpo funcione em equilíbrio, algo que é bastante estudado na Fisiologia de órgãos e tecidos.

Ao final, você será capaz de:

- compreender o metabolismo como conjunto integrado de reações químicas da vida;
- diferenciar processos de obtenção, armazenamento e uso de energia no organismo;
- relacionar glicose, ATP, hormônios e atividade fisiológica com situações do cotidiano;
- interpretar a homeostase como equilíbrio dinâmico, e não como estado fixo;
- explorar, modificar e discutir modelos computacionais simples de regulação metabólica.

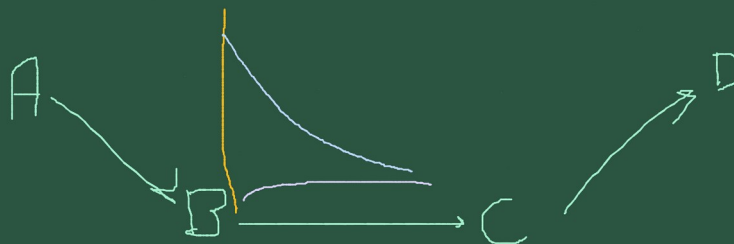


Figura 6.1: Energia produz biomoléculas cuja quebra produz energia.

MEXA ANTES DE ENTENDER



Para esse capítulo integrado de metabolismo e fisiologia, comece pelo *app* de JSPlotly que segue, sobre o que ocorre no metabolismo antes e depois das refeições ao longo do dia.

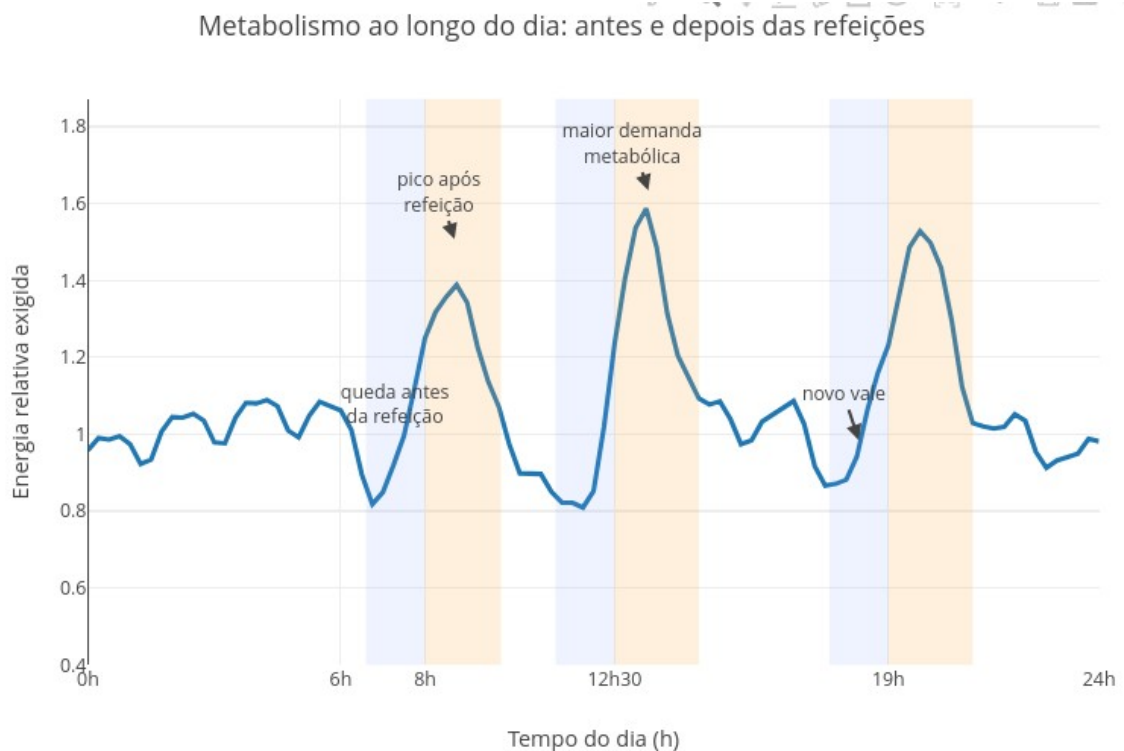


Figura 6.2: O corpo não exige sempre a mesma quantidade de energia. Clique com o botão direito neste [LINK](#) e abra o aplicativo em nova aba.

O script simula num gráfico a energia relativa exigida pelo organismo ao longo de um dia. Veja que essa curva não é constante, podendo subir, descer, e se ajustar de acordo com o horário, a atividade e as refeições.

Agite antes de usar

O gráfico destaca dois momentos importantes: uma queda antes da refeição, e um pico logo depois da refeição.

Execute o script com os valores originais.

```
const taxa_metabolica = 1.0;
const atividade = 1.0;
const variacao = 0.05;
```

Depois altere o código em:

```
const taxa_metabolica = 1.3;
```

ou:

```
const atividade = 1.8;
```

Observe como a curva sobe, oscila e muda de forma.

Aumente também:

```
const variacao = 0.12;
```

Antes de uma refeição, a energia disponível pode estar menor, e o organismo tende a funcionar usando reservas e ajustes internos. Logo depois da refeição, a demanda aumenta porque o corpo precisa digerir, absorver, transportar e armazenar nutrientes. Entre as refeições, o metabolismo retorna gradualmente a um nível mais estável. No gráfico, isso fica claro junto às etapas de queda, refeição, pico, e retorno.

Olhando internamente para o script, ele calcula a energia exigida em pequenos intervalos de tempo:

```
for (let t = 0; t <= tempo_max; t += 0.25) {
```

Depois soma vários componentes:

```
ritmoDiario
efeitoAtividade
vales
picos
ruídoSuave(t)
```

Os vales simulam quedas antes das refeições:

```
valeAntesRefeicao(t, hora, intensidade, largura)
```

Os picos simulam aumento de demanda logo após as refeições:

```
picoRefeicao(t, hora, intensidade, largura)
```

O ruídoSuave(t) adiciona pequenas variações fisiológicas, evitando uma curva artificialmente perfeita.

No fim, a curva representa um metabolismo em ajuste contínuo, bem longe de qualquer simulação real, é bem verdade, mas muito distante também das imagens estáticas presentes nos livros didáticos. Para utilizar o script rapidamente:

1. Rode o script com os valores originais;
2. Observe os vales antes das refeições;
3. Observe os picos logo após as refeições;
4. Passe o mouse sobre a curva para ver o momento indicado;
5. Aumente atividade;
6. Veja como a curva fica mais variável;
7. Aumente taxa_metabolica;
8. Observe se toda a curva sobe;
9. Aumente variacao;

10. Veja como pequenas oscilações aparecem sobre a tendência principal.

Veja que o metabolismo muda ao longo do dia, sempre respondendo à dinâmica do corpo. Quer alguns cenários para se “divertir” ?

1 *Menor demanda metabólica*. Como uma menor taxa metabólica altera a altura geral da curva ?

```
const taxa_metabolica = 0.7;
```

2. *Maior demanda metabólica*. Agora mude a taxa para 1.4. A curva muda de forma ou principalmente de intensidade ?
3. *Mais atividade*. Observe se uma maior atividade altera as oscilações ao longo do dia.

```
const atividade = 2.0;
```

4. *Pouca variação*. Um gráfico muito “liso” parece menos parecido com um organismo real ?

```
const variacao = 0.01;
```

5. *Alta variação*. Agora mude para 0.15. Quando a variação aumenta, fica mais difícil perceber os padrões principais ?

A curva mostra que o organismo alterna momentos de menor e maior exigência energética. Antes das refeições, aparecem vales que representam menor disponibilidade imediata de energia. Após as refeições, aparecem picos relacionados à digestão, absorção e processamento dos nutrientes. Depois, a curva tende a voltar para níveis intermediários. Em poucas palavras, o metabolismo não fica parado, mas acompanha as necessidades do corpo.

Essas necessidades variam ao longo do dia, fazendo com que órgãos, tecidos e células ajustem suas atividades metabólicas em função de moléculas e da energia presente. Um bom exemplo dessa flutuação é o que ocorre com os níveis de glicose do sangue na presença de ATP, adenosina trifosfato, a *moeda de troca energética* das células, e da insulina, o principal hormônio do metabolismo de glicose. O script de JSPlotly seguinte busca ilustrar esse metabolismo integrado.

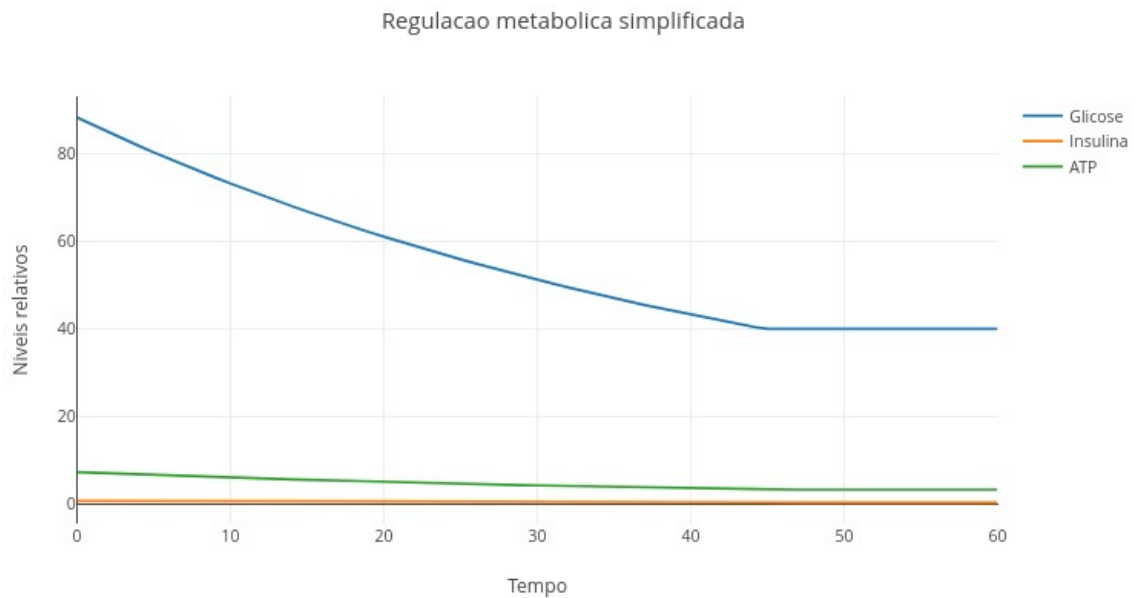


Figura 6.3: O metabolismo funciona como uma rede de ajustes sobre eventos controlados. Clique com o botão direito neste [LINK](#) e abra o aplicativo em nova aba.

O script simula, de forma simplificada, a relação entre glicose, insulina, atividade corporal e produção de ATP, mostrando que o metabolismo não depende de uma variável somente. Quando a atividade aumenta, o corpo tende a consumir mais glicose. Quando há glicose disponível, a resposta da insulina favorece sua entrada nas células. E, dentro das células, parte dessa glicose pode ser usada para produzir ATP.

Agite antes de usar

O gráfico mostra um sistema de regulação simplificado, e em que a glicose representa combustível disponível, a insulina um sinal de controle, e o ATP a energia celular produzida a partir desse combustível.

Execute o script com os valores originais:

```
let nivel_glicose = 90;
const atividade = 5.0;
const sensibilidade_insulina = 0.7;
const reposicao_basal = 0.2;
```

Depois altere a glicose inicial:

```
let nivel_glicose = 140;
```

Aumente a atividade:

```
const atividade = 9.0;
```

E reduza a sensibilidade à insulina:

```
const sensibilidade_insulina = 0.3;
```

Observe como as curvas mudam juntas, e veja que o corpo ajusta o uso de

glicose para manter energia disponível. A glicose diminui conforme é consumida pela atividade, sendo acompanhada pelo nível de insulina, uma resposta de regulação do açúcar do sangue. E a produção de ATP depende da glicose disponível. Por isso, quando uma curva muda, as outras também respondem.

Por dentro do script define-se inicialmente alguns parâmetros principais:

```
let nivel_glicose = 90;
const atividade = 5.0;
const sensibilidade_insulina = 0.7;
const reposicao_basal = 0.2;
```

Depois calcula-se a resposta da insulina:

```
const ins = sensibilidade_insulina * (nivel_glicose / 100);
```

Em seguida, estima-se o consumo de glicose:

```
const consumo = atividade * ins * 0.6;
```

E a produção relativa de ATP:

```
const producao_atp = Math.max(0, nivel_glicose * 0.08);
```

Por fim, atualiza-se o nível de glicose:

```
nivel_glicose = nivel_glicose - consumo + reposicao_basal;
```

A cada volta do laço (ou *loop*, no jargão da computação), o organismo simulado entra em um novo estado. Use o tutorial abaixo para experimentar o *app*.

1. Rode o script com os valores originais;
2. Observe a curva da glicose;
3. Compare glicose e ATP;
4. Veja como a insulina acompanha a glicose;
5. Aumente atividade;
6. Observe se a glicose cai mais rapidamente;
7. Aumente reposicao_basal;
8. Veja se a glicose se mantém por mais tempo;
9. Reduza sensibilidade_insulina;
10. Observe como o sistema muda de comportamento.

Depois de testar o script, você deve perceber que o equilíbrio energético depende da glicose disponível, bem como da atividade e da resposta hormonal. Todas “*juntas e misturadas*”, e não uma mais importante que a outra. Seguem alguns cenários para você explorar.

!. *Glicose inicial normal*. Como o sistema se comporta quando parte de um valor intermediário ?

```
let nivel_glicose = 90;
```

2. *Glicose inicial elevada*. Mude a glicose para 160 (valor comum na

hiperglicemia, ou excesso de açúcar no sangue). Maior glicose inicial sustenta maior produção de ATP por mais tempo ?

3. *Atividade intensa*. Pense num exercício físico. Por que maior atividade acelera o consumo de glicose ?

```
const atividade = 10.0;
```

4. *Baixa sensibilidade à insulina*. O que acontece quando a resposta à insulina fica menos eficiente, o que é comum no quadro de *diabetes* ?

```
const sensibilidade_insulina = 0.25;
```

5. *Maior reposição basal*. Como a reposição contínua ajuda a manter a glicose disponível ?

```
const reposicao_basal = 1.0;
```

Portanto, quando a atividade aumenta, o consumo cresce. Quando a glicose cai, a produção de ATP tende a diminuir. Em síntese, o corpo precisa equilibrar entrada, armazenamento, consumo e reposição de energia.

FAZENDO APARECER



O *app* da [Figura 6.3](#) integrou a glicose circulante, a ação da insulina e a produção de ATP, unindo a Bioquímica e a Fisiologia para um exemplo de homeostasia (equilíbrio) entre a produção e o consumo de energia corporal. Imagine agora que tudo isso está acontecendo ao mesmo tempo em tecidos corporais diferentes, e com um fluxo de informação que permite ajustes mais complexos. Essa é a proposta do próximo script ! Apresentar o metabolismo como uma rede regulatória, com sangue, fígado, músculo, hormônios e ATP se comunicando no mesmo sistema integrado.

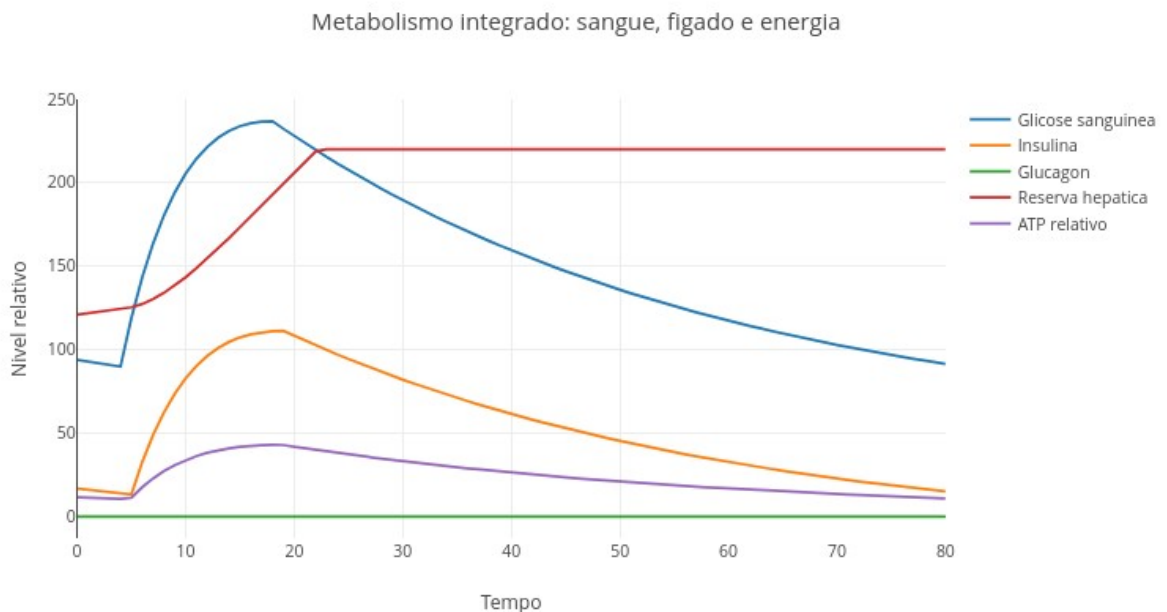


Figura 6.4: O metabolismo envolve uma regulação entre entrada, uso e reserva de energia e de moléculas. Clique com o botão direito neste [LINK](#) e abra o aplicativo em nova aba.

Agora o script roda um painel (*dashboard*) que simula um sistema metabólico mais completo, anos-luz do que ocorre nas células, mas modelado para que você entenda de um jeito mais *lúdico* como funciona o metabolismo do corpo num nível fisiológico. A glicose dessa vez não aparece sozinha, mas interage com a entrada de alimento, captação pelos músculos, reserva no fígado, ação da insulina, ação do glucagon (ou *glicagon* em português) e a produção relativa de ATP. A insulina é o “*hormônio da saciedade celular*”, ou seja, é liberada quando a célula tem nutrientes suficientes, como a glicose. Já o glucagon atua ao contrário da insulina, como “*hormônio da fome celular*”, porque ele é liberado quando a célula está carente de nutrientes, como a glicose.

Agite antes de usar

Ao rodar o *app*, veja que o gráfico apresenta um organismo que não apenas usa energia, regula, armazena, libera e redistribui essa energia conforme as condições mudam.

Execute o script com os valores originais:

```
let glicose_sangue = 95;
let reserva_figado = 120;
const atividade = 1.2;
const sensibilidade = 1.0;
const refeicao = 30;
```

Depois altere a refeição:

```
const refeicao = 70;
```

Aumente a atividade:

```
const atividade = 2.5;
```

Reduza a reserva hepática:

```
let reserva_figado = 40;
```

Observe como o corpo tenta se reorganizar, mantendo energia disponível mesmo quando a glicose sobe e desce. Logo após a entrada de alimento, a glicose sanguínea tende a aumentar. Quando a glicose está alta, a insulina aumenta e favorece captação e armazenamento. Quando a glicose cai, o glicagon aumenta e favorece a liberação de reservas do fígado. Ao mesmo tempo, os músculos consomem glicose para produzir ATP. Em síntese, o metabolismo funciona trocando informações entre órgãos e sinais químicos.

O código começa com valores iniciais para glicose no sangue e reserva do fígado:

```
let glicose_sangue = 95;
let reserva_figado = 120;
```

Depois simula uma entrada alimentar entre os tempos 5 e 18 (horas):

```
if (t >= 5 && t <= 18) {
  entrada_alimentar = refeicao * Math.exp(-(t - 5) / 7);
}
```

Essa entrada representa nutrientes chegando ao sangue de forma gradual e depois diminuindo.

A insulina aumenta quando a glicose está acima de certo nível:

```
const ins = Math.max(0, sensibilidade * (glicose_sangue - 70) / 30);
```

O glicagon aumenta quando a glicose fica mais baixa. Esses dois sinais funcionam como forças opostas de regulação.

```
const glu = Math.max(0, (90 - glicose_sangue) / 25);
```

A captação muscular representa o uso da glicose pelos tecidos:

```
const captacao_muscular = atividade * (0.4 + 0.6 * ins);
```

O fígado pode liberar glicose quando o glucagon aumenta:

```
const liberacao_figado = Math.min(reserva_figado * 0.04, glu * 2.5);
```

E também armazenar glicose quando a insulina está mais alta:

```
const armazenamento_figado = Math.max(0, ins * 1.2);
```

No final, o ATP relativo depende tanto da captação muscular quanto da glicose disponível:

```
const energia_atp = captacao_muscular * 8 + glicose_sangue * 0.03;
```

O resultado é uma simulação integrada de entrada, uso, e reserva de energia. Você pode usar o tutorial abaixo para rodar o aplicativo.

1. Rode o script com os valores originais;
2. Observe o aumento inicial da glicose após a refeição;
3. Veja como a insulina responde a esse aumento;
4. Observe a queda posterior da glicose;
5. Veja quando o glucagon começa a aparecer;
6. Acompanhe a reserva hepática;
7. Observe como o ATP relativo responde à atividade muscular;
8. Aumente refeicao;
9. Aumente atividade;
10. Reduza reserva_figado;
11. Compare como o sistema muda em cada situação.

Seguem também alguns cenários para estudo e compreensão do tema.

1. *Refeição pequena.* Uma entrada menor de alimento gera menor pico de glicose e insulina ?

```
const refeicao = 10;
```

2. *Refeição intensa.* Aumente a refeicao para 80. Como uma grande entrada alimentar altera glicose, insulina e armazenamento no fígado ?

```
const refeicao = 80;
```

3. *Atividade muscular elevada.* Por que maior atividade aumenta a captação de glicose e a produção relativa de ATP ?

```
const atividade = 3.0;
```

4. *Baixa reserva no fígado.* O que acontece quando o corpo tem pouca reserva para liberar glicose ?

```
let reserva_figado = 30;
```

5. *Menor sensibilidade à insulina.* Como o sistema muda quando a resposta à insulina fica menos eficiente ?

```
const sensibilidade = 0.35;
```

6. *Alta sensibilidade à insulina.* Mude a sensibilidade para 1.8. Maior sensibilidade facilita o controle da glicose após a refeição ?

O gráfico mostra que glicose, insulina, glicagon, fígado e ATP não são variáveis independentes. Quando a glicose sobe, a insulina tende a subir. Quando a glicose cai, o glucagon tende a aparecer. Quando há atividade, os músculos aumentam a captação de glicose. Quando o corpo precisa manter energia, o fígado pode liberar parte de sua reserva.

VOLTA PRO MUNDO



Metabolismo é o conjunto de processos que mantém a vida funcionando, como produzir ATP, regular temperatura, transportar substâncias, contrair músculos, sintetizar moléculas e manter o equilíbrio interno. No mundo real, o metabolismo envolve muitos órgãos, como fígado, rins, cérebro, intestino, pâncreas, músculos, tecido adiposo, e sangue, entre outros. E também muitas moléculas, como glicose, aminoácidos, proteínas, glicogênio, lipídios, ácidos nucleicos, hormônios, enzimas..e por aí vai. Tudo muito bem controlado.

vida depende dessa coordenação contínua entre energia, sinalização e equilíbrio interno. E esse equilíbrio é o que faz com que moléculas que estejam em excesso sejam eliminadas, e as que estejam em falta sejam produzidas. Se houver algum desvio excessivo no níveis dessas biomoléculas, bem como na integridade dos tecidos que as mantêm sob controle, aí então podem surgir enfermidades dos mais variados tipos e complexidades.

Nesse nível fisiológico, portanto, é a coordenação entre sistemas que mantêm a vida em funcionamento. Quando nos alimentamos, o sistema digestório disponibiliza nutrientes, e o sistema endócrino participa do controle hormonal. Enquanto isso, o sangue transporta moléculas, as células ajustam suas taxas de consumo, e o sistema nervoso age como uma central de controle para responder de forma rápida e diversificada a fim de manter tudo organizado. Em outras palavras, metabolismo e fisiologia se encontram a todo momento. A produção de energia depende de substratos (reagentes das enzimas) e de vias metabólicas, mas também de sinais, controles e prioridades do organismo.

Para entender mais ainda esses controles diversificados, segue um último JSPlotly para o capítulo, mas desta vez bem mais “*animadinho*” !

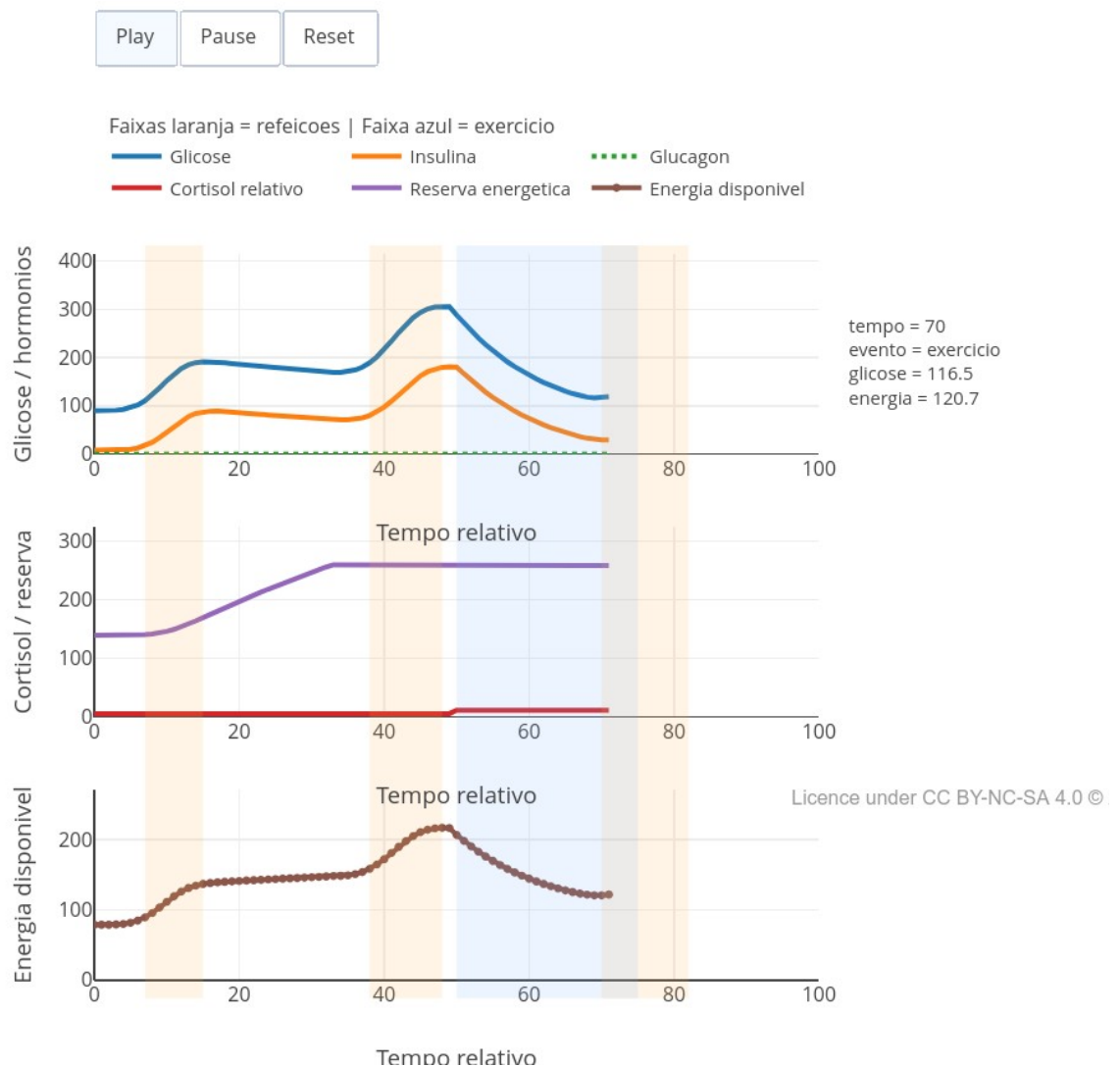


Figura 6.5: O corpo permanece equilibrado porque responde continuamente às mudanças. Clique com o botão direito neste [LINK](#) e abra o aplicativo em nova aba.

Agora o *app* aborda uma “homeostase sistêmica” !!! Nome “chique” para quando o corpo inteiro entra em cena !

O código apresenta uma animação para a resposta integrada do organismo a refeições, exercício, hormônios e reservas energéticas. Ao clicar em Play, você acompanha a evolução de glicose, insulina, glicagon, cortisol relativo (um hormônio ativo do exercício e esforço), reserva energética e energia disponível.

As faixas laranja indicam momentos de refeição. A faixa azul indica um período de exercício. O script tem a intenção de ilustrar como o corpo mantém equilíbrio mesmo quando as condições mudam, como alimentação, exercício e estresse.

Agite antes de usar

Execute o script com os valores originais, observando a sequência de eventos:

```
input_glicose = 90;
input_reserva = 140;
input_atividade = 1.5;
input_sens = 1.0;
input_estresse = 0.6;
input_mod0 = "animado";
```

Depois teste:

```
input_atividade = 3.0;
```

ou:

```
input_estresse = 1.5;
```

Observe como o sistema muda quando o corpo precisa lidar com maior demanda. Durante uma refeição, a glicose tende a aumentar, e esse aumento direciona a resposta da insulina. Durante o exercício, a demanda energética cresce, a reserva pode diminuir e o cortisol relativo aumenta. Quando a glicose cai, o glicagon ajuda a mobilizar as reservas. E como um todo, o corpo busca um equilíbrio dinâmico.

Para entender um pouco como funciona o script, ele começa com variáveis que representam o estado inicial do organismo:

```
let glicose = 90;
let reserva = 140;
```

Depois simula três refeições usando pulsos matemáticos:

```
pulso(t, 10, 4, 16)
pulso(t, 42, 5, 20)
pulso(t, 76, 5, 18)
```

Esses pulsos criam aumentos temporários na glicose. O exercício aparece em um intervalo específico:

```
if (t >= 50 && t <= 75) {
  exercicio = atividade * 2.5;
}
```

A insulina aumenta quando a glicose está mais alta:

```
const ins = Math.max(0, sens * (glicose - 80) / 25);
```

O glicagon aumenta quando a glicose está mais baixa:

```
const glu = Math.max(0, (85 - glicose) / 20);
```

O cortisol relativo representa uma resposta fisiológica ao estresse e ao exercício:

```
const cort = estresse + exercicio * 0.15;
```

A cada instante, o script atualiza glicose, reserva e energia disponível. Depois, os frames fazem a curva aparecer progressivamente, criando a animação. Não custa lembrar que essa simulação visa cumprir uma função didática, e não é baseada em relações matemáticas cientificamente

rigorosas. Contudo, as equações apresentadas acima e a lógica do algoritmo permitem explorar muitos conceitos do metabolismo fisiológico. Segue um rápido tutorial de uso.

1. Rode o script em modo "animado";
2. Clique em Play;
3. Observe primeiro os momentos de refeição;
4. Veja como a glicose responde;
5. Compare a resposta da insulina;
6. Observe o período de exercício;
7. Veja como o cortisol relativo aumenta;
8. Acompanhe a reserva energética;
9. Observe a energia disponível;
10. Use o *slider* para voltar a momentos específicos;
11. Aumente `input_atividade`;
12. Aumente `input_estresse`;
13. Reduza `input_sens`;
14. Compare como o sistema tenta manter equilíbrio.

Agora compare alguns cenários:

1. *Exercício moderado*. Como o organismo responde a uma demanda energética intermediária ?

```
input_atividade = 1.5;
```

2. *Exercício intenso*. Mude a atividade para 3.5. Por que uma maior atividade reduz reservas e altera a energia disponível ?
3. *Maior estresse fisiológico*. Como o estresse altera a disponibilidade de energia e a resposta hormonal ?

```
input_estresse = 1.8;
```

4. *Menor sensibilidade à insulina*. O que muda quando a glicose sobe, mas a resposta à insulina fica menos eficiente ?

```
input_sens = 0.4;
```

5. *Baixa reserva energética*. O que acontece quando o organismo tem pouca reserva para sustentar exercício e equilíbrio ?

```
input_reserva = 50;
```

6. *Maior glicose inicial*. Como uma glicose inicial elevada muda a resposta da insulina e da energia disponível ?

```
input_glicose = 150;
```

A animação mostra que a homeostase do corpo significa capacidade de responder, mais do que uma monotonia por falta de mudanças. Após as refeições, a glicose aumenta e a insulina participa do controle. Durante o exercício, o uso de energia aumenta, as reservas são mobilizadas e o cortisol relativo sobe. Quando a glicose cai, o glicagon ajuda a sustentar a disponibilidade de combustível. Ou seja, o organismo está sempre ajustando entrada, uso, armazenamento e liberação de energia.

No mundo real, a fisiologia sistêmica envolve muitos órgãos trabalhando juntos, como intestino, pâncreas, fígado, músculos, tecido adiposo,

E SE...



São virtualmente ilimitadas as situações do cotidiano no tema para essa seção. Algumas selecionadas estão abaixo.

- E se o metabolismo nunca variasse ao longo do dia ?
- E se refeições deixassem de produzir picos metabólicos perceptíveis ?
- E se o organismo não conseguisse reduzir o gasto energético entre refeições ?
- E se pequenas flutuações fisiológicas desaparecessem completamente ?
- E se o corpo consumisse energia mais rápido do que consegue repor ?
- E se a atividade física aumentasse sem aumento proporcional da disponibilidade de glicose ?
- E se a glicose estivesse alta, mas as células respondessem pouco à insulina ?
- E se a produção de ATP diminuísse mesmo com glicose ainda disponível ?
- E se o organismo tivesse glicose circulante, mas pouca capacidade de captá-la ?
- E se a atividade muscular aumentasse muito rapidamente ?
- E se as reservas hepáticas fossem pequenas demais para sustentar longos períodos sem alimentação ?
- E se o fígado liberasse glicose continuamente, mesmo sem necessidade ?
- E se o glucagon permanecesse elevado por muito tempo ?
- E se refeições muito intensas provocassem oscilações exageradas no sistema ?
- E se o metabolismo precisasse equilibrar simultaneamente entrada de alimento, atividade muscular e reserva energética ?
- E se o corpo precisasse continuar produzindo ATP mesmo durante queda de glicose ?
- E se o exercício aumentasse ao mesmo tempo em que o estresse fisiológico também aumentasse ?
- E se o cortisol permanecesse elevado durante longos períodos ?
- E se o organismo tivesse pouca reserva energética para sustentar exercício intenso ?
- E se a homeostase fosse uma linha fixa, em vez de um equilíbrio dinâmico ?
- E se pequenas alterações hormonais fossem capazes de reorganizar todo o metabolismo ?
- E se o equilíbrio fisiológico dependesse não da ausência de mudanças, mas da capacidade de responder continuamente a elas ?
- E se o corpo não armazenasse energia ?
- E se músculos, fígado, hormônios e sangue não funcionassem de forma integrada ?



Esse capítulo pode se conectar facilmente a inúmeras situações e conteúdos que são estudados na Bioquímica e Fisiologia, tais como a respiração celular e a fermentação, e seu envolvimento com a produção de ATP. Os scripts do capítulo também permitem associações variadas, como a digestão e absorção, ao abordar a entrada de nutrientes no sistema, assim como o sistema endócrino, ao tratar da ação hormonal na regulação.

Esses scripts também são correlacionados com o sistema muscular, ao se discutir consumo energético durante esforço, bem como saúde pública, ao relacionar metabolismo com obesidade, diabetes e hábitos de vida. Também percorrem um pouco a relação do metabolismo com esporte e desempenho, ao interpretar demanda energética em diferentes intensidades de atividade, e de ritmos biológicos, ao observar variações ao longo do dia.

Todas essas relações também surgem panoramicamente em áreas bastante diversas. Por exemplo, na Química, ao se discutir energia, reações exergônicas e endergônicas, e na Física, quando se relaciona trabalho, potência e transformação de energia.

POR DENTRO DA FERRAMENTA



Ao longo desta obra, essa seção tem sido dedicada a espelhar o pensamento computacional e a lógica de programação junto aos diversos scripts de JSPlotly, finalizando com um pseudocódigo voltado a um desses scripts em cada capítulo.

Neste capítulo os scripts de JSPlotly foram utilizados para se ter uma visão dinâmica de como funciona o metabolismo e suas relações com os diversos tecidos do corpo. O Script no. 1, por exemplo, trata de acompanhar a flutuação diária da demanda energética do corpo, enquanto que o Script no. 2 ocupa-se de ilustrar de forma simplificada a regulação da glicose por insulina. Já o Script no. 3 aborda um metabolismo mais integrado junto a tecidos. Por fim, a animação executada pelo Script no. 4 estende essa visão para uma regulação hormonal e de substratos ligada a diferentes condições de atividade metabólica.

Mas você já deve ter percebido também que, ao longos dos capítulos deste trabalho, aquelas competências digitais acima passaram a integrar paulatinamente o próprio capítulo, diminuindo a necessidade de sua exclusividade para o tópico que se apresenta. A partir de agora, então, essa seção irá dedicar-se à **verdadeira ferramenta** por detrás de todos os scripts que podem ser feitos com o JSPlotly: a linguagem **JavaScript**.

O entendimento de como opera essa ferramenta permitirá a você compreender e modificar qualquer código deste livro, ajustá-lo às suas preferências ou necessidades, bem como criar outros tantos que desejar. Além disso, poderá também entender o que há por detrás das páginas de

internet e criar as suas próprias, como blogs e sites. Isso porque o JavaScript opera silenciosamente quando você clica num botão da tela de seu celular, preenche um formulário ou acompanha uma animação gráfica na internet. Por óbvio, entretanto, esta nanométrica seção não lhe dará condições de sair por aí programando em JavaScript. Mas lhe dará munção básica para que avance ao próximo nível nessa e em qualquer linguagem de programação.

JavaScript (JS), é uma linguagem de programação moderna, utilizada por grandes empresas e *big techs*, e focada na interatividade entre a página web e o usuário. Diferente de outras linguagens, *JavaScript* não precisa ser compilada separadamente por um programa, pois é interpretada a partir de máquinas homônimas presentes em navegadores comuns (*browser*), como Firefox, Chrome, Safari, Opera, Edge, etc.

Assim, *Javascript (JS)* é uma linguagem para web. Ela completa a tríade essencial, *HTML-CSS-JS*. Enquanto *HTML* preocupa-se com o conteúdo e *CSS* com o estilo, *JavaScript* ocupa-se do comportamento, ou seja, da interatividade do usuário frente à uma página web. Vamos então chamá-la agora carinhosamente por *JS*, somente.

Como ocorre com outras linguagens de programação, a linguagem *JS* possui bibliotecas, que são módulos independentes de códigos para cumprir certas funções. Nesse sentido, *JS* cumpre metade do nome do *JSPlotly*. A outra metade vem da principal biblioteca que é utilizada por esse para a construção de gráficos, mapas, e alguns outros objetos interativos, [Plotly.js](#).

6.1 Estrutura de JS

Uma linguagem de programação normalmente possui características estruturais comuns. Isso quer dizer que há comandos para declarar uma variável e seu tipo, a entrada e saída de dados (resultados), a criação e uso de funções, e cálculos aritméticos, por exemplo. E tal como outras linguagens de programação, *JS* opera com uma lógica similar de declarações (palavras-chave, operadores, valores, expressões). Em resumo, a gente dizer as declarações de *JS* compõem:

1. Palavras-chave: sintaxe da linguagem JS;
2. Operadores: caracteres que realizam operações;
3. Valores: texto, números, verdadeiro/falso (variável "booleana"), "não definido", "nulo";
4. Expressões: trechos de código que produzem um único valor.

Do ponto de vista estrutural, por conseguinte, *JS* possui características que também são comuns a outras linguagens de programação:

1. Saída de dados;
2. Declaração de variáveis;
3. Tipos de dados;
4. Constantes;
5. Aritmética;
6. Vetores;

- 7. Funções;
- 8. Geração de dados aleatórios;
- 9. Objetos;
- 10. Operadores;
- 11. Estruturas de controle;

Por enquanto é só. Se quiser aprender um pouquinho mais de JS, basta vira a página !

7. Reprogramando a Vida: DNA, Genes e Biotecnologia Moderna



O MUNDO TE CHAMA

Pense num filme de ficção científica. E num personagem típico...um daqueles pesquisadores que os cineastas adoram apresentar como descabelados, com rostos vidrados em tubos de ensaio, e personalidade que mais parece a de um assassino serial de baratas. No filme, o cientista maluco usa uma ferramenta, modifica um DNA e o põe numa célula, pra gerar um novo organismo com novas funções, e ainda por cima resistente a uma doença mortal !

Bom, isso já deixou de ser ficção! É a Biologia Molecular à serviço da Biotecnologia moderna. Quando passamos a *manipular o DNA* já a algumas décadas, começamos a experimentar novas técnicas e aplicações da engenharia genética, do DNA recombinante, e da edição de genes, permitindo hoje que vida possa ser *programada*. Este capítulo pretende passear um pouco sobre alguns conceitos da estrutura do DNA, o código da vida, bem como aplicações biotecnológicas compreendidas nos estudos de expressão gênica (informação virando função) e na modificação do DNA (DNA recombinante).

Depois de explorar, você consegue:

- Entender o DNA como sistema de informação
- Explicar como genes são expressos
- Compreender como o DNA pode ser manipulado
- Relacionar biotecnologia com aplicações reais
- Experimentar visualizações interativas com objetos computacionais.

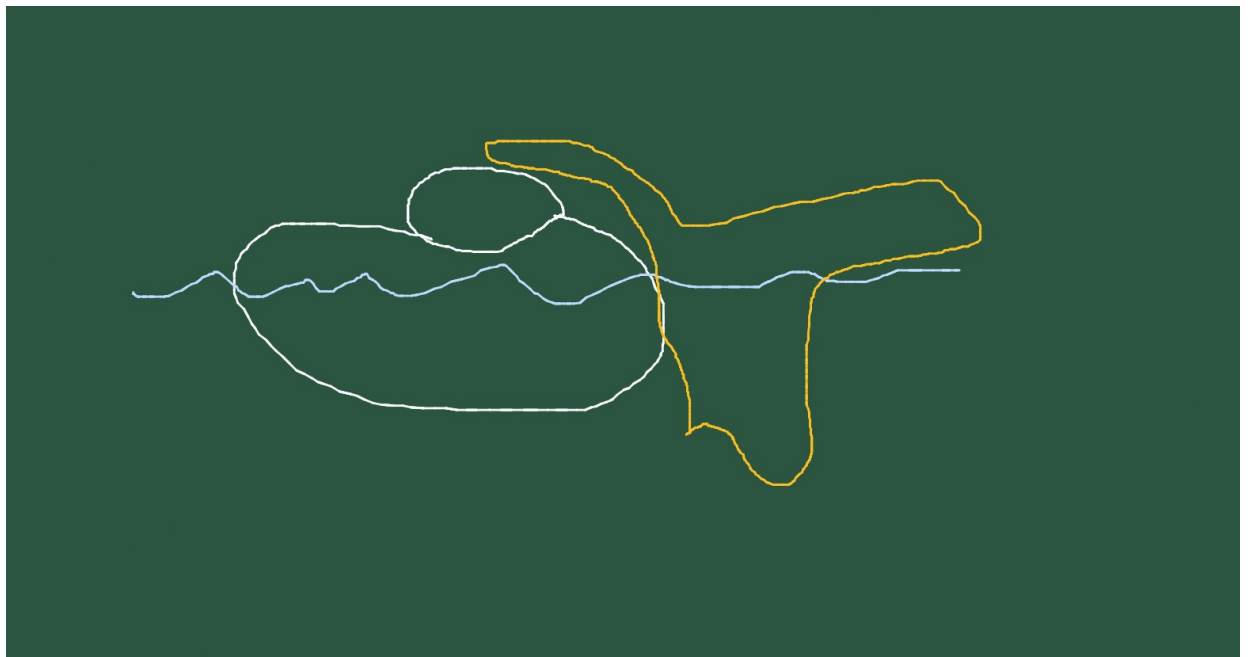


Figura 7.1: A Biologia Molecular permite editar o futuro.

MEXA ANTES DE ENTENDER



Para iniciar o capítulo, o JSPlotly abaixo busca simular, ainda que bem longe da realidade, por óbvio, a expressão gênica que envolve a síntese de uma proteína a partir do DNA.

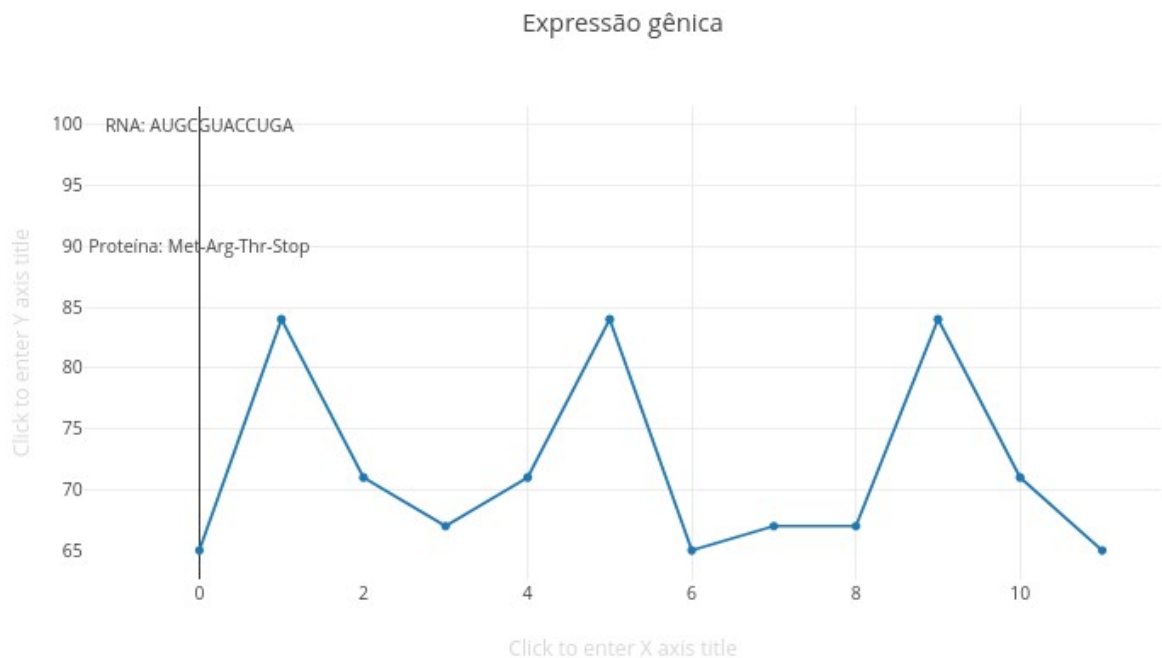
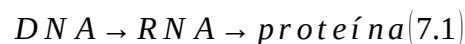


Figura 7.2: Uma sequência de letras pode gerar uma função biológica. Clique com o botão direito do mouse neste [LINK](#) e abra-o numa nova aba.

O *app* mostra, de forma simplificada, que a informação genética pode ser lida, copiada e transformada em função biológica. O processo ocorre em 2 etapas. Primeiro, o DNA é convertido a RNA pela transcrição, seguido depois pela tradução, essa convertendo o RNA em uma proteína. Necessário dizer que, ao falarmos em “converter o DNA em...”, na verdade trata-se um gene apenas, a estrutura molecular que contém o código necessário para a construção final de uma proteína ou somente de uma parte. Essa passagem de DNA para proteína é denominada por *Dogma Central da Biologia Molecular*. E, como um dogma, é inquestionável e definitiva, embora haja, e como sempre há em tudo, algumas poucas exceções na Biologia.



Agite antes de usar

O gráfico mostra o DNA como uma sequência numérica (representação simbólica). Mas o mais importante não está na forma do gráfico, mas sim nas anotações que aparecem nele. Veja que lá tem informações sobre o RNA gerado e a proteína formada.

Execute o script com a sequência original, ou seja:

```
let dna = "ATGCGTACCTGA";
```

Depois altere a sequência para outras variações, tais como:

```
let dna = "ATGACCUGA";
let dna = "ATGCGTCGTACCUGA";
```

Observe a sequência de RNA gerada, bem como a proteína formada. Experimente também mudar apenas uma letra. Em relação à sequência original, poderia ser, por exemplo:

```
let dna = "ATGCGTACCTAA";
```

Veja como pequenas mudanças podem alterar toda a proteína. Isso ocorre porque o DNA é como um código, e cujas letras representam os monômeros de suas bases nitrogenadas. Durante sua transcrição, T (timina) vira U (uracila), formando o RNA. Durante a tradução o RNA é lido em trincas (ou *tripletes*) chamadas códon, cada um representando um aminoácido. E há códigos também de início e de parada para a “leitura” do RNA, por exemplo:

- AUG → Met (início)
- UGA → Stop (fim)

Assim, uma sequência simples de bases se transforma em uma cadeia de aminoácidos ao final, algo que é realizado por um grande complexo proteico contendo o RNA, o ribossomo. Segue uma “sequência” rápida para uso do *app*:

1. Rode o script;
2. Observe a sequência de DNA;
3. Veja a sequência de RNA gerada;
4. Observe a proteína formada;
5. Identifique o códon de início (AUG);
6. Identifique o códon de parada (UGA);
7. Modifique um nucleotídeo;
8. Veja se a proteína muda;
9. Modifique vários nucleotídeos;
10. Observe como a sequência final se altera.

Note que uma pequena mudança no DNA pode alterar pouco ou muito uma proteína final. Isso ocorre em função do tipo da *base nitrogenada* que é substituída, por exemplo, numa mutação gênica. Se a base trocada for redundante, isto é, permite que o aminoácido produzido se mantenha o mesmo, a proteína não sofre alteração. Mas se a base alterar o aminoácido traduzido, aí o “*bicho pega*”, e a proteína pode até deixar de desempenhar suas funções.

Essa última observação é vista em algumas doenças, como a anemia falciforme, por exemplo. Nessa, uma *única base* nitrogenada é trocada, levando a hemoglobina a ter um aminoácido diferente justamente num local importante para mantê-la estável dentro da hemácia. Essa instabilidade leva à precipitação da hemoglobina, “entortando” a hemácia para uma forma de foice. Daí o nome da doença.

Alguns contextos para você praticar com o JSPlotly.

1. *Sequência funcional*. Essa sequência gera uma proteína completa ?

```
let dna = "ATGCGTACCTGA";
```

2. *Mutação pontual*. Uma única troca de base pode interromper a produção da proteína ?

```
let dna = "ATGCGTACCTAA";
```

3. *Sem códon de parada*. O que acontece quando não há sinal de parada ?

```
let dna = "ATGCGTACCACC";
```

4. *Sequência curta*. Uma proteína muito curta pode ser funcional ?

```
let dna = "ATGUGA";
```

5. *Sequência aleatória*. Toda sequência de DNA gera uma proteína funcional ?

```
let dna = "GTCGTCGTC";
```

A ideia é visualizar o *dogma central da Biologia Molecular*, mas aromatizado com propósito: *informação → transcrição → tradução → função.**

Por dentro do script

O script começa com a sequência de DNA:

```
let dna = "ATGCGTACCTGA";
```

Depois realiza a transcrição:

```
let rna = dna.replace(/T/g, "U");
```

Isso simula a troca de timina por uracila.

Em seguida, define uma tabela simplificada de códons:

```
let codons = {
  "AUG": "Met",
  "CGU": "Arg",
  "ACC": "Thr",
  "UGA": "Stop"
};
```

Depois o RNA é lido de três em três:

```
for (let i = 0; i < rna.length; i += 3)
```

Cada trinca gera um aminoácido:

```
protein.push(codons[codon] || "?");
```

Se o códon não estiver na tabela, aparece "?".

Assim, o script transforma sequência genética em produto funcional.

FAZENDO APARECER



Depois de ver uma versão simplificada da expressão gênica, agora você pode experimentar algo mais próximo da realidade biológica: o *código genético completo*. O próximo script da [Figura 7.3](#) permite que você coloque qualquer sequência de DNA válida para ser traduzida em proteína. Assim como acontece de verdade nas células vivas.

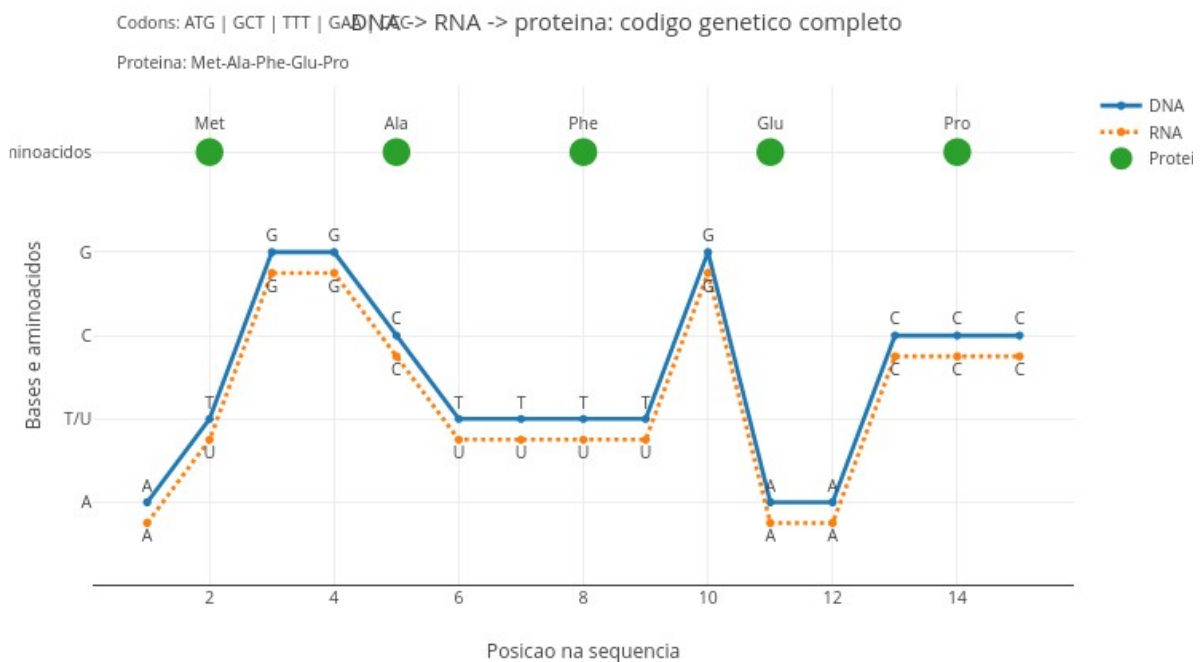


Figura 7.3: A sequência do DNA leva à leitura do RNA cuja tradução resulta numa função (proteína). Clique com o botão direito neste [LINK](#) e abra em nova aba.

Esse novo JSPlotly mostra a tradução completa do DNA numa proteína usando o código genético real. Agora você pode acompanhar, simultaneamente, a sequência de DNA e de RNA, os códons (trincas), bem como os aminoácidos formados. O script roda um gráfico que organiza a informação em três níveis: DNA para sequência original, RNA para versão transcrita e proteína para o resultado funcional. A altura do gráfico não é quantitativa, mas simbólica. Ela representa as bases (A, T/U, C, G), e com os aminoácidos acima. Novamente, a ideia básica do *app* é mostrar que a vida funciona como um sistema de leitura e tradução de informação.

Agite antes de usar

Execute o aplicativo inicialmente com a sequência padrão:

```
const dnaEntrada = "ATGGCTTTTGAACCC";
```

Depois experimente sequências distintas. Por exemplo:

```
const dnaEntrada = "ATGAAATTTGGGCCC";
const dnaEntrada = "ATGCGTACGTTAG";
```

Agora altere apenas uma base e observe como a proteína muda.

```
const dnaEntrada = "ATGGCTTTTGAACCA";
```

A pergunta “*que não quer calar*” aqui é: *como o DNA “sabe” qual proteína produzir ?*

E a resposta está no código genético. Cada trinca de bases (códon) corresponde a um aminoácido. Por exemplo, ATG → Met (início), GCT → Ala, e TGA → Stop (parada). Assim, o DNA contém as instruções para produzir as proteínas. Seguem “instruções” rápidas para o *app*.

1. Rode o script;
2. Observe o DNA no gráfico;
3. Compare com o RNA logo abaixo;
4. Identifique os códons (grupos de 3);
5. Veja os aminoácidos gerados;
6. Localize o códon de início (ATG);
7. Procure por códons de parada (TAA, TAG, TGA);
8. Modifique uma base;
9. Veja se o códon muda;
10. Observe o efeito na proteína.

Depois praticar um pouco, você verá que uma pequena alteração no DNA pode alterar pouco uma proteína, ou mudá-la completamente. Vão alguns contextos também pra você sedimentar as ideias.

1. *Sequência funcional completa*. A proteína é formada corretamente até o códon de parada ?

```
const dnaEntrada = "ATGGCTTTTGAACCC";
```

2. *Mutação silenciosa*. Diferentes códons podem gerar o mesmo aminoácido ?

```
const dnaEntrada = "ATGGCC";
```

3. *Mutação com mudança de aminoácido*. Uma troca altera a função da proteína ?

```
const dnaEntrada = "ATGGTT";
```

4. *Parada precoce*. O que acontece quando a tradução para cedo demais ?

```
const dnaEntrada = "ATGTAA";
```

5. *Sem códon de início*. Toda sequência é traduzida ou é preciso um ponto de partida ?

```
const dnaEntrada = "GCTGCTGCT";
```

6. *Sequência longa*. Como proteínas maiores emergem de sequências repetidas ?

```
const dnaEntrada = "ATGGCTGCTGCTGCTGCTGCTTGA";
```

Por dentro do script

O script começa limpando a sequência:

```
const dna = dnaEntrada.toUpperCase().replace(/^[^ATCG]/g, "");
```

Depois cria o RNA:

```
const rna = dna.replace(/T/g, "U");
```

Em seguida, percorre a sequência em trincas:

```
for (let i = 0; i < dna.length; i += 3)
```

Cada códon é traduzido:

```
aminoacidos.push(codigoGenetico[codonDNA] || "?");
```

Depois o gráfico mostra três camadas: DNA (bases), RNA (transcrição) e proteína (tradução). E ainda exibe os códons identificados e a sequência final da proteína.

VOLTA PRO MUNDO



A Biotecnologia moderna ocupa-se de compreender os organismos vivos sob uma visão molecular voltada a estudos de proteínas, hereditariedade, mutações e doenças genéticas. O processo que leva da transcrição do DNA à tradução em proteínas oportuniza o surgimento de aplicações diversas que dependem dessa lógica, como novas vacinas de RNA, terapias gênicas, edição genética (CRISPR), plantas transgênicas, testes de DNA, diagnóstico molecular e engenharia de proteínas, por exemplo.

O DNA recombinante está na base de muitas dessas aplicações modernas, incluindo a produção de insulina por bactérias, os organismos geneticamente modificados (OGM), enzimas industriais, pesquisa biomédica, e agricultura biotecnológica. O JSPlotly a seguir trata um pouco sobre isso, abordando um modelo simples de DNA recombinante como uma sequência que recebe uma nova informação.

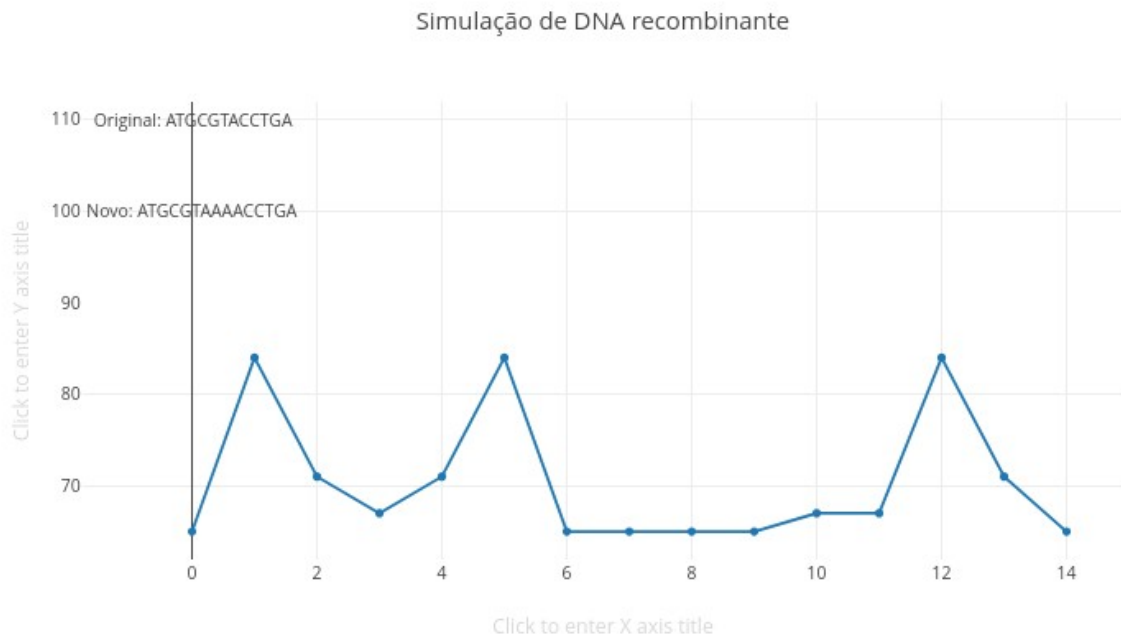


Figura 7.4: DNA recombinante: recortar, inserir, apagar, combinar e expressar informação genética. Clique com o botão direito neste [LINK](#) e abra em nova aba.

Depois de traduzir sequências de DNA em RNA e proteína, este script dá um passo biotecnológico da hora: modificar a sequência original. A ideia agora é simular, de forma simples, o princípio do *DNA recombinante*, em que um fragmento genético pode ser inserido em outro DNA para formar uma nova molécula.

Agite antes de usar

O script simula a inserção de um pequeno trecho de DNA em uma sequência original. A sequência inicial é cortada em uma posição escolhida e um novo fragmento é inserido nesse ponto, resultando numa nova sequência, o DNA recombinante. O gráfico resultante representa a sequência recombinante como uma linha simbólica. O mais importante está nas anotações, que mostram a sequência original junto com a nova sequência.

Inicialmente execute o script com:

```
let dna_original = "ATGCGTACCTGA";
let insercao = "AAA";
let corte = 6;
```

Depois altere a inserção. Você pode usar, por exemplo, "GGG", "TAA", ou "ATG".

E também mude a posição do corte, como `let corte = 3` ou `let corte = 9`.

Observe como a sequência final muda quando adicionamos uma nova informação dentro de uma molécula de DNA. Em Biotecnologia, inserir um

trecho de DNA pode alterar a sequência final, a leitura dos códons, a proteína produzida e até a função biológica resultante. Mesmo uma pequena inserção pode ter efeitos grandes, principalmente se mudar o modo como a sequência é lida. Segue um pequeno tutorial de uso do *app*.

1. Rode o script com os valores originais;
2. Compare a sequência original com a sequência nova;
3. Observe onde a inserção apareceu;
4. Altere *insercao*;
5. Veja como o DNA recombinante muda;
6. Altere *corte*;
7. Observe como a posição da inserção modifica a sequência final;
8. Use inserções com 3 bases;
9. Depois teste inserções com 1, 2, 4 ou 5 bases;
10. Observe que nem toda inserção preserva a leitura em códons.

Observe que inserir DNA muda sua interpretação biológica, mais do que somente aumentar a sua sequência. Mais alguns cenários pra você “brincar” de manipular genes:

1. *Inserção simples*. Como uma pequena inserção altera a sequência final ?

```
let insercao = "AAA";
let corte = 6;
```

2. *Inserção em outro ponto*. A mesma inserção causa o mesmo efeito em posições diferentes ?

```
let insercao = "AAA";
let corte = 3;
```

3. *Inserção de códon de parada*. O que pode acontecer se um sinal de parada for inserido no meio da sequência ?

```
let insercao = "TAA";
let corte = 6;
```

4. *Inserção de novo início*. O que significa inserir um novo possível ponto de início ?

```
let insercao = "ATG";
let corte = 0;
```

5. *Inserção fora do tamanho de códon*. Por que inserir uma quantidade de bases que não é múltipla de 3 pode bagunçar a leitura ?

```
let insercao = "AA";
let corte = 5;
```

6. *Sequência maior*. Como fragmentos maiores podem acrescentar novas informações ao DNA ?

```
let dna_original = "ATGGCTTTTGAACCC";
let insercao = "GGGCCC";
let corte = 9;
```

Em síntese, modificar a sequência modifica a informação, o que evidencia o potencial da intervenção biotecnológica.

Por dentro do script

O script começa com três informações principais:

```
let dna_original = "ATGCGTACCTGA";
let insercao = "AAA";
let corte = 6;
```

A variável `dna_original` guarda a sequência inicial. A variável `insercao` guarda o novo fragmento, e a variável `corte` indica a posição onde o DNA será aberto. A recombinação acontece nesta linha:

```
let novo_dna = dna_original.slice(0, corte) + insercao +
dna_original.slice(corte);
```

Essa linha faz três coisas: pega o início do DNA original, adiciona a sequência inserida e completa com o restante do DNA original. Assim, o código imita uma operação básica de montagem genética.

CRISPR

Antes de iniciar a seção *E se...* na sequência dessa, e se a gente dispusesse de uma tecnologia de tesoura e cola para o DNA recombinante ?

Isso existe é uma das técnicas mais empregadas atualmente para edição genética, *CRISPR (Clustered Regularly Interspaced Short Palindromic Repeats)*. Um palavrão em inglês, e que se traduz como *Repetições Palindrômicas Curtas Agrupadas e Regularmente Interespçadas*. Ou seja, outro palavrão, mas agora em português !

CRISPR é uma tecnologia de edição genética que atua como uma “tesoura molecular” precisa, permitindo cortar e modificar o DNA das células. É baseada em um sistema de defesa bacteriano que utiliza uma enzima (geralmente Cas9) e um RNA guia para localizar e editar sequências genéticas específicas, com alta precisão, rapidez e baixo custo para alterar o código genético. Como técnica verdadeiramente revolucionária da última década, o CRISPR permite corrigir mutações genéticas, estudar função de genes, desenvolver terapias, modificar organismos, bem como avançar na medicina personalizada. Diferente do DNA recombinante clássico, o CRISPR atua com precisão. É como editar uma palavra específica dentro de um texto, em vez de apenas inserir novos parágrafos.

Depois de inserir sequências (DNA recombinante), agora você pode experimentar um JSPlotly ainda mais refinado: *editar diretamente o DNA existente*. O script simula, de forma simplificada, o princípio do CRISPR — localizar um trecho específico e substituí-lo por outro.

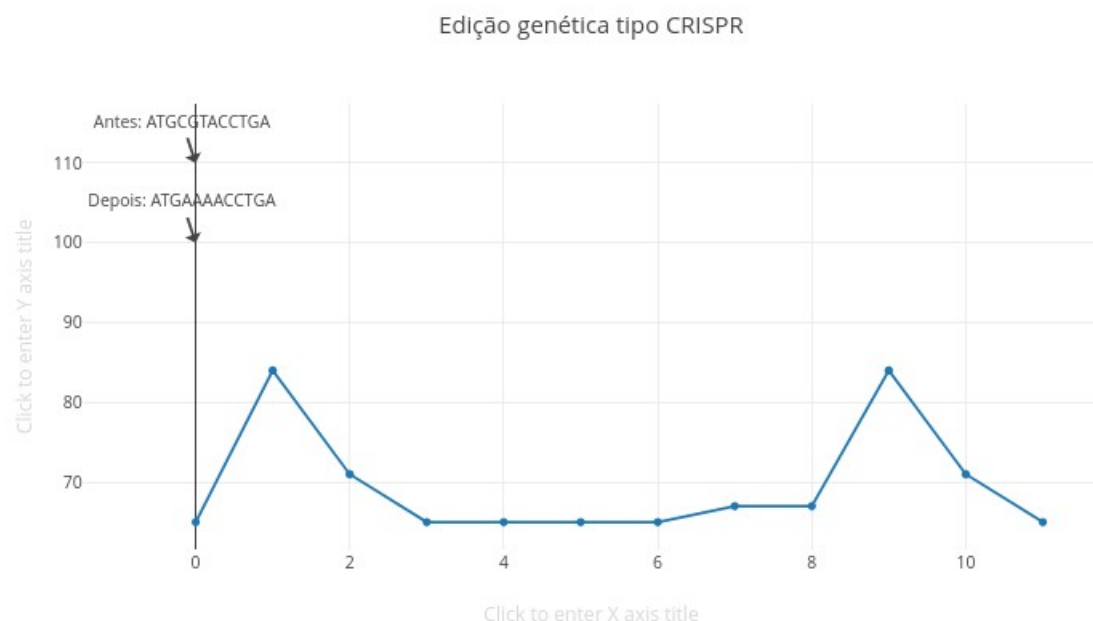


Figura 7.5: CRISPR: encontrar, modificar, e alterar a função. Clique com o botão direito neste [LINK](#) e abra em nova aba.

O script simula uma edição genética dirigida. Você define uma sequência original de DNA, um trecho alvo, e uma nova sequência substituta. O sistema então encontra o alvo e o substitui diretamente. Um gráfico representa a sequência editada, anotando a sequência antes e a sequência depois. A diferença entre elas representa a edição genética. Guardadas as longínquas comparações com o método em laboratório, a simulação apresenta a base do CRISPR, uma técnica precisa para edição da informação genética.

Agite antes de usar:

Inicialmente, execute o script com:

```
let dna = "ATGCGTACCTGA";
let alvo = "CGT";
let substituto = "AAA";
```

Depois teste:

```
let alvo = "ACC";
let substituto = "GGG";
```

E também:

```
let alvo = "ATG";
let substituto = "TAA";
```

Observe como a sequência muda quando se altera diretamente um trecho específico do DNA. Diferente da inserção aleatória do JSPlotly da [Figura 7.4](#), aqui há precisão. Isso pode corrigir mutações, alterar proteínas, interromper funções ou mesmo criar novas funções. E, mais

uma vez nessa obra, mostra-se que uma pequena alteração localizada pode ter grandes efeitos. Isso é quase uma citação do Tio Ben do Homem-Aranha, inclusive.

O modo de usar do script segue abaixo.

1. Rode o script;
2. Observe a sequência original;
3. Veja a sequência editada;
4. Identifique onde ocorreu a substituição;
5. Altere o alvo;
6. Teste se ele existe na sequência;
7. Altere o substituto;
8. Compare diferentes edições;
9. Tente substituir por sequências maiores ou menores;
10. Observe quando nada muda (alvo inexistente).

Perceba então que editar um trecho específico é mais produtivo do que inserir sequências aleatórias. Vão alguns contextos para você experimentar no seu laboratório virtual de engenharia genética:

1. *Edição simples*. Como uma substituição altera a sequência global ?

```
let alvo = "CGT";
let substituto = "AAA";
```

2. *Alvo inexistente*. O que acontece quando o trecho não é encontrado ?

```
let alvo = "GGG";
```

3. *Substituição por códon de parada*. Interromper uma sequência pode impedir a produção de proteína ?

```
let substituto = "TAA";
```

4. *Alteração do início*. Modificar o ponto inicial altera toda a leitura ?

```
let alvo = "ATG";
let substituto = "CCC";
```

5. *Substituição maior*. Substituir por um trecho maior altera o tamanho e a leitura da sequência ?

```
let substituto = "GGGCCC";
```

6. *Substituição menor*. Reduzir o tamanho da sequência pode bagunçar a leitura em códons ?

```
let substituto = "A";
```

Por dentro do script

O script começa com três elementos:

```
let dna = "ATGCGTACCTGA";
let alvo = "CGT";
let substituto = "AAA";
```

A variável dna contém a sequência original. A variável alvo define o trecho

que será buscado. E a variável substituto define o que entrará no lugar. A edição ocorre nesta linha:

```
let editado = dna.replace(alvo, substituto);
```

Essa função procura o primeiro trecho igual ao alvo e o substitui. Assim, o código simula a lógica básica de uma edição genética dirigida.

E SE...



Antes de nos perdemos nessa belicosa seção, que tal um último JSPlotly bem “animado”, juntando as ideias principais do capítulo ? Depois de aprender a traduzir DNA e editar sequências, agora você vai observar o efeito de uma mutação acontecendo em tempo “real” durante a tradução !

No *app* que segue, o ribossomo “caminha” pela sequência, permitindo que você compare a sequência normal com a sequência mutada, e a proteína resultante.

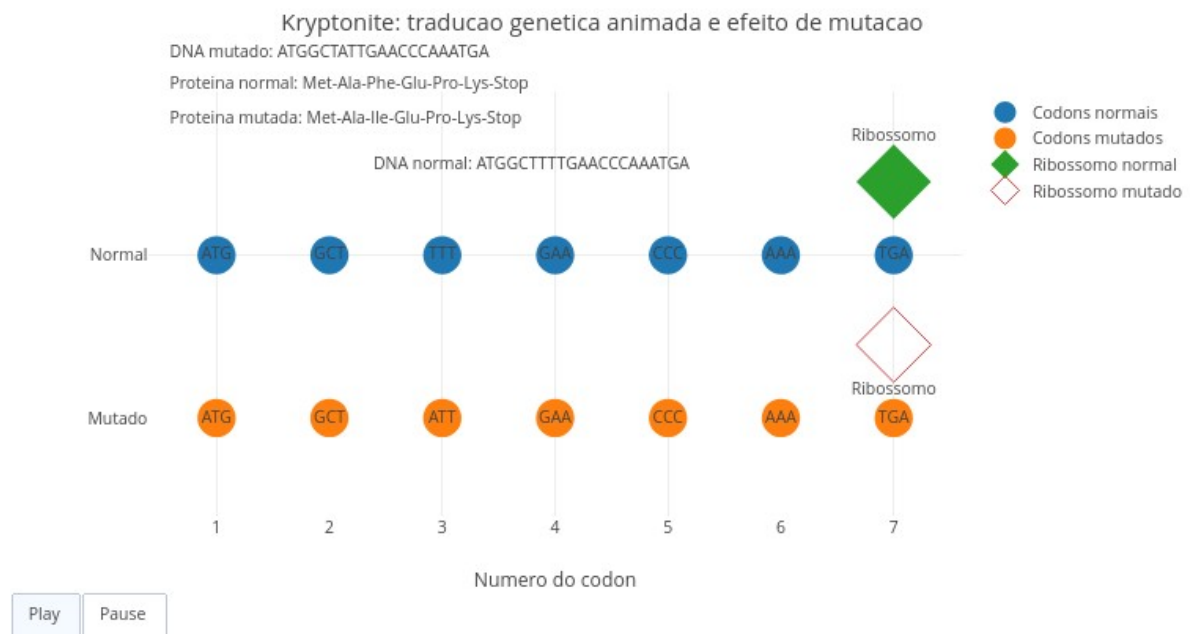


Figura 7.6: A mutação representa um processo com consequências ao longo da leitura. Clique com o botão direito neste [LINK](#) e abra em nova aba.

Agite antes de usar

O script mostra a tradução genética como um processo dinâmico. No gráfico, os losangos representam o ribossomo. O eixo horizontal do gráfico representa os códons. Já o eixo vertical separa a sequência normal da sequência mutada. Pela animação, você acompanha os códons sendo lidos passo a passo, o ribossomo avançando na sequência, a diferença entre DNA normal e mutado, e o impacto da mutação na proteína.

Execute o script e clique em Play. Observe então o ribossomo avançando, os códons sendo “lidos”, e a comparação entre normal e mutado. Depois altere:

```
const posicaoMutacao = 7;
const novaBase = "A";
```

E teste também:

```
const posicaoMutacao = 3;
const novaBase = "T";
```

Desligue a mutação:

```
const aplicarMutacao = false;
```

E compare os resultados. Durante a tradução o ribossomo lê códons sequencialmente. Cada códon define um aminoácido e a sequência define a proteína. Assim, uma mutação pode alterar um aminoácido, gerar um códon de parada precoce, deslocar toda a leitura (dependendo do tipo), ou ainda não alterar nada (mutação silenciosa). O efeito depende de *onde* e *como* a mutação ocorre.

Por dentro do script

O script começa com a sequência original:

```
const dnaNormal = "ATGGCTTTTGAACCCAAATGA";
```

Depois aplica a mutação:

```
const dna1 = mutarDNA(dna0, posicaoMutacao, novaBase);
```

A função abaixo substitui uma única base na posição escolhida:

```
function mutarDNA(seq, pos, base)
```

Depois ocorre a tradução:

```
const normal = traduzir(dna0);  
const mutado = traduzir(dna1);
```

E a “mágica” acontece na animação:

```
frames.push({
```

Cada frame representa o ribossomo avançando um códon, permitindo que você veja a tradução acontecendo passo a passo. Segue um tutorial de uso do *app*.

1. Rode o script;
2. Clique em Play;
3. Observe o ribossomo avançando;
4. Compare linha normal e mutada;
5. Veja onde aparece a diferença;
6. Observe a proteína final;
7. Altere posicaoMutacao;
8. Veja se o impacto muda;
9. Altere novaBase;
10. Compare diferentes tipos de mutação;
11. Desative a mutação (`false`);
12. Compare com o caso original.

Perceba que, dependendo da situação, uma mutação muda pouco ou altera toda uma sequência funcional. Seguem alguns contextos para você experimentar junto à animação.

1. *Mutação silenciosa*. Por que algumas mutações não alteram a

proteína ?

```
const posicaoMutacao = 4;
const novaBase = "C";
```

2. *Mudança de aminoácido.* Quando uma mutação altera um aminoácido, o que pode acontecer com a função da proteína ?

```
const posicaoMutacao = 7;
const novaBase = "G";
```

3. *Parada precoce.* O que acontece quando surge um códon STOP no meio da sequência ?

```
const novaBase = "T";
```

4. *Mutação no início.* Alterar o início muda toda a tradução ?

```
const posicaoMutacao = 1;
```

5. *Sem mutação.* Como o sistema se comporta quando não há alteração ?

```
const aplicarMutacao = false;
```

6. *Comparação dinâmica.* Observe a animação completa. Em que momento a mutação começa a fazer diferença ?

A animação mostra uma leitura progressiva da informação genética, mostrando o que uma mutação faz com a leitura do código ao longo do tempo.

Agora sim, uma “saraivada” de “E...se”s”tecnologicamente recombinantes” ! Segue:

- E se uma única letra no DNA fosse capaz de mudar completamente uma proteína — e, com isso, alterar o funcionamento de uma célula inteira ?
- E se pequenas mutações que parecem insignificantes fossem, na verdade, responsáveis pela diversidade de todos os seres vivos ?
- E se o mesmo código genético que produz proteínas no seu corpo pudesse ser reescrito em laboratório ?
- E se uma sequência de DNA pudesse ser “copiada, colada e editada” como um texto digital ?
- E se inserir um pequeno trecho de DNA fosse suficiente para uma bactéria produzir um hormônio humano ?
- E se mutações silenciosas realmente passassem despercebidas, mas ainda assim influenciassem o funcionamento do organismo ?
- E se uma mutação criasse um códon de parada no meio de um gene — o que deixaria de ser produzido ?
- E se o ribossomo fosse como um leitor automático, que percorre o código sem “entender” o significado, mas apenas seguindo regras ?
- E se a vida fosse, em essência, um sistema de leitura e interpretação de informação molecular ?
- E se diferentes combinações de três letras fossem suficientes para codificar toda a diversidade de proteínas conhecidas ?
- E se mudar o ponto de leitura (*frame*) fosse como mudar o idioma de uma frase, ou seja, transformando completamente o sentido ?

- E se editar genes com precisão pudesse corrigir doenças hereditárias antes mesmo de elas se manifestarem ?
- E se fosse possível reprogramar células para produzir novas funções, como sensores ou medicamentos ?
- E se a biotecnologia permitisse projetar organismos com características específicas ?
- E se o limite entre “natural” e “modificado” fosse cada vez mais difícil de definir ?
- E se o mesmo processo que permite evolução também fosse a base de doenças ?
- E se a diferença entre saúde e doença estivesse, às vezes, em uma única base do DNA ?
- E se a evolução fosse, em grande parte, um experimento contínuo de mutações sendo testadas ao longo do tempo ?
- E se mutações que hoje parecem neutras se tornassem vantajosas em outro ambiente ?
- E se fosse possível prever o efeito de uma mutação antes mesmo de ela acontecer ?
- E se proteínas fossem apenas uma das formas de “interpretar” o DNA, e existissem outras ainda desconhecidas ?
- E se a vida pudesse ser descrita como um fluxo contínuo de informação sendo copiada, editada e traduzida ?
- E se o DNA fosse mais parecido com um código de programação do que com uma estrutura estática ?
- E se aprender Biologia fosse, no fundo, aprender a ler, interpretar e modificar esse código ?
- E se, no futuro, editar genes fosse tão comum quanto editar um documento hoje ?
- E se a maior revolução científica não estivesse em descobrir novos elementos, mas em aprender a reescrever a vida ?
- E se nós mesmos fôssemos, ao mesmo tempo, leitores e resultados desse código em constante transformação ?

MESMO PADRÃO, OUTROS MUNDOS



A lógica explorada nesse capítulo, de que informação define comportamento, aparece em muitos outros contextos, não sendo exclusiva da Biologia Molecular. Em diferentes sistemas, a forma como a informação é organizada e interpretada determina o funcionamento do todo.

Na Biologia, isso é direto: vimos que sequências de DNA armazenam instruções que são transcritas e traduzidas em proteínas. Pequenas mudanças nessa sequência podem alterar estruturas, funções e até características de um organismo inteiro. A vida, nesse sentido, pode ser entendida como um sistema contínuo de leitura, interpretação e execução de informação.

Nas Ciências da Computação, esse mesmo padrão aparece de forma evidente. Códigos, algoritmos e programas são sequências de instruções que, quando executadas, produzem comportamentos complexos. Alterar uma linha de código pode mudar completamente o resultado — assim como uma mutação pode alterar uma proteína.

Na linguagem humana, palavras e frases seguem regras semelhantes. A ordem e a combinação dos elementos determinam o significado. Uma pequena alteração pode mudar completamente a interpretação. Assim como no DNA, a sequência e o contexto importam tanto quanto o conteúdo.

Na Biotecnologia moderna, isso também vira uma ferramenta. Editar, inserir ou substituir sequências permite reprogramar sistemas biológicos inteiros. O mesmo raciocínio aparece em engenharia genética, Medicina personalizada e até Inteligência Artificial. Em todos esses “mundos”, controlar a informação permite que se controle o comportamento.

POR DENTRO DA FERRAMENTA



Nessa seção, damos sequência a um aprendizado panorâmico em *JavaScript*, voltado ou não à edição de objetos com JSPlotly, como realizado no capítulo anterior. Nesse sentido, segue um pouco sobre a estrutura de *JS* para aritmética e *arrays* (vetores).

E agora um truque bem bacana !!! **Você poderá treinar a linguagem JavaScript utilizando um JSPlotly específico para isso !**. Esse JSPlotly personalizado está disponível na [Figura 7.7](#) abaixo. Abra-o num navegador e deixe-o quietinho lá. Quando quiser rodar algum trecho desses treinamentos, basta copiar, colar o trecho no campo “ÁREA PARA DIGITAR COMANDOS”, e rodá-lo. Muito legal, não ?!

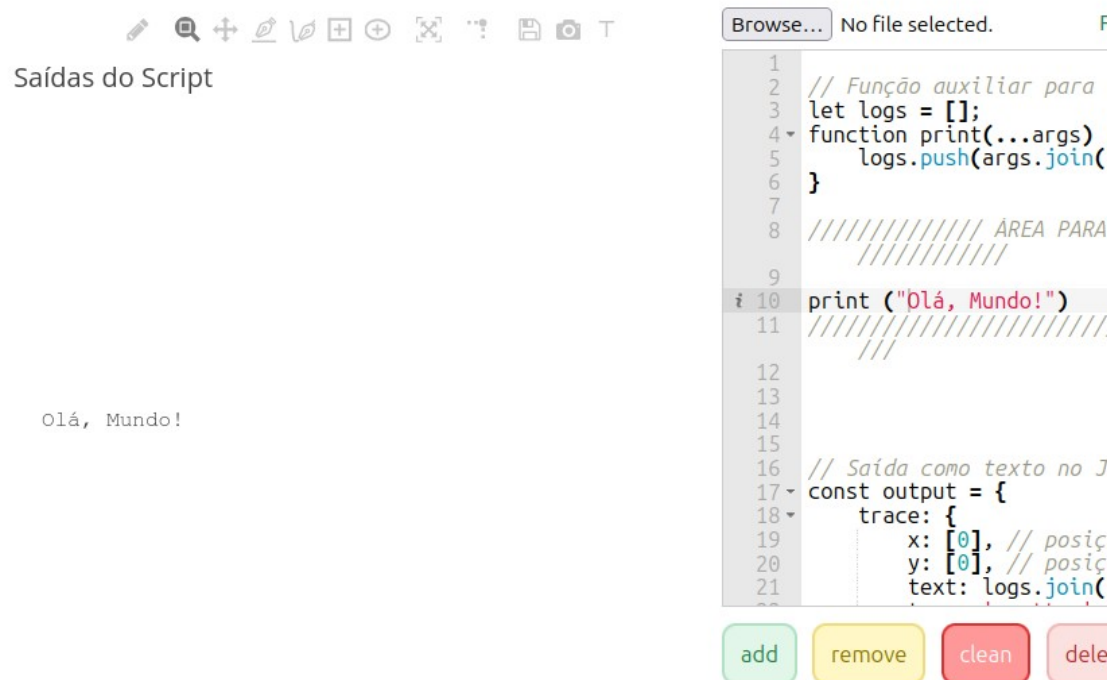


Figura 7.7: JSPlotly personalizado para treinamento da linguagem JavaScript [JSPlotly_treinoJS.html](#). Basta clicar com o botão direito do mouse para abri-lo em outra janela do navegador. Deixe-o quietinho lá. Quando quiser executar algum trecho de código ao longo desse treinamento, basta copiar, colar no campo “ÁREA PARA DIGITAR COMANDOS”, e rodar no JSPlotly personalizado.

Aritmética

Diversas operações matemáticas são possíveis, quer pelo *JS* puro, quer pela instalação de bibliotecas específicas, tais como *Plotly.js* utilizada no JSPlotly, *mathjs* ou *numjs*. Segue abaixo uma lista de operações aritméticas realizadas com a linguagem:

1. + : adição de números ou concatenação de texto (*strings*);
2. - : subtração;
3. * : multiplicação;
4. / : divisão;
5. % : módulo;
6. ++ : incremento de valor;
7. -- : decremento;

8. ****** : exponenciação.

Obs: o operador de módulo % divide o primeiro operando pelo segundo operando e retorna o resto.

Os exemplos a seguir ilustram o uso desses operadores aritméticos, bem como de algumas constantes matemáticas. Para testá-los, cole no *JSPotly* customizado, como informado acima. Você pode experimentá-los separadamente ou em bloco.

```
let resultado = 0;

resultado = 5 + 3;           print("5 + 3 = ", resultado);
// 8
resultado = 10 - 4;         print("10 - 4 = ", resultado);
// 6
resultado = 6 * 7;          print("6 * 7 = ", resultado);
// 42
resultado = 20 / 5;         print("20 / 5 = ", resultado);
// 4
resultado = 17 % 3;         print("17 % 3 = ", resultado);
// 2
resultado = 2 ** 3;         print("2 ** 3 = ", resultado);
// 8
let contador = 5; contador++; print("contador++ = ", contador);
// 6
let decrementar = 8; decrementar--; print("decrementar-- = ",
decrementar); // 7
let soma = 10; soma += 5;    print("soma += 5 = ", soma);
// 15
let diferenca = 20; diferenca -= 7; print("diferença -= 7 = ",
diferença); // 13
let produto = 3; produto *= 4;   print("produto *= 4 = ",
produto); // 12
let quociente = 24; quociente /= 3; print("quociente /= 3 = ",
quociente); // 8
resultado = Math.sqrt(25);      print("sqrt(25) = ", resultado);
// 5
resultado = Math.abs(-15);      print("abs(-15) = ", resultado);
// 15
resultado = Math.floor(7.8);    print("floor(7.8) = ",
resultado); // 7
resultado = Math.ceil(7.2);     print("ceil(7.2) = ", resultado);
// 8
resultado = Math.round(9.4);    print("round(9.4) = ",
resultado); // 9
resultado = Math.max(42, 27);   print("max(42,27) = ",
resultado); // 42
resultado = Math.min(42, 27);   print("min(42,27) = ",
resultado); // 27
resultado = Math.sin(Math.PI/2); print("sin(π/2) = ", resultado);
```

Arrays

Um *array* em *JavaScript* representa uma estrutura de dados para

armazenamento de elementos ordenados. Esses elementos podem envolver qualquer tipo de dado, como números, strings, objetos, etc. Na prática, um *array* pode ser criado, acessado, filtrado, ou mapeado. Seguem exemplos:

Criação

```
const numeros = [1, 2, 3, 4, 5];
const cores = ['vermelho', 'verde', 'azul'];
```

Acesso

```
print(numeros[0]);
print(cores[2]);
```

Ou seja, para você acessar o *array* `numeros`, por exemplo, basta colar e rodar o seguinte trecho no JSPotly de treinamento:

```
const numeros = [1, 2, 3, 4, 5];
const cores = ['vermelho', 'verde', 'azul'];

print(numeros[0]);
```

Mapeamento (*map*)

```
const numeros = [1, 2, 3, 4, 5];
const quadrados = numeros.map(numero => numero * numero);
print(quadrados);
```

Uma palavrinha sobre métodos

Observe que, diferente do que já foi explicitado, agora aparece um comando múltiplo, como “*numeros.map*”. Existe uma infinidade de comandos com esta sintaxe em *JS*. Ele representa a função que será aplicada a cada elemento do *array* *numeros* pelo *método* *map*. Além disso, surge a sintaxe “=>” ou sintaxe de *arrow function* (função de seta), uma forma concisa de atribuir um mapeamento. Assim, o comando inteiro:

Em *JavaScript* *objetos* são tratados como no mundo real, ou seja, possuem *atributos* e *comportamentos*. Atributos descrevem as características que um objeto possui, e os comportamentos as ações que podem realizar. A notação em *JS* para referenciar as propriedades de um objeto podem ser, alternativamente:

```
objeto.propriedade
```

```
...ou...
```

```
objeto[propriedade]
```

Voltando ao exemplo acima e esmiuçando-o um pouquinho mais, o *método* *map()* (`numeros.map`) é chamado no *array* *numeros* (um *Array.prototype*). O *protótipo* permite transmitir propriedades e métodos para um objeto *JS*. No caso, o *método* *map()* itera sobre cada elemento do *array* original, aplica uma função a cada um deles e retorna um novo *array* com os resultados. Assim, *numero => numero * numero* é a *função* de

seta. E nesse caso, *numero* é o parâmetro de entrada da função, um atalho para o elemento atual do *array* sendo processado pelo *map*. Em adição, “=>” representa o separador entre o(s) parâmetro(s) e o corpo da função. Finalmente, “*numero * numero*” é a expressão que é executada, com o resultado sendo retornado pela função. Seguem outros exemplos para arrays e mapeamento:

```
const x = [0, 1, 2, 3, 4];
const y = x.map(valor => Math.pow(2, valor))
print(y);

const Vmax = 100;
const Km = 10;
const S_values = Array.from({length: 10}, (_, i) => i * 5);

const V_values = S_values.map(S => (Vmax * S) / (Km + S));

print("Valores de S:", S_values);
print("Valores de V:", V_values);
```

JavaScript, uma linguagem orientada a objetos

Ainda que o esforço para traduzir os comandos acima possa auxiliar como funciona a linguagem, é por óbvio esperar uma certa confusão de termos. Sendo assim, vamos subindo de nível!

Como diz o título acima, *JS* é uma linguagem orientada a objetos. Isso significa que o código é organizado em torno de *objetos*, que combinam dados (*propriedades*) e comportamentos (*métodos*) para modelar entidades do mundo real. Isso permite um código mais modular, reutilizável e fácil de gerenciar, ao organizar a lógica em “*partes*” menores e mais compreensíveis (*pensamento computacional*).

Objetos, propriedades, e métodos

Um *objeto* é uma estrutura que agrupa dados e comportamentos. Os dados ficam guardados em *propriedades*, e os comportamentos são implementados por *métodos* (funções dentro do objeto). Assim, propriedade são pares nome-valor armazenados dentro de um objeto, podendo variar como números, textos, arrays, e outros objetos. Já os métodos são funções associadas a um objeto, e que descrevem ações que ele pode executar.

Objetos em *JS* “*podem ser comparados a objetos da vida real*”, e guardam informações relacionadas entre si. Por exemplo, o objeto “*ponto*” abaixo representa um ponto no plano:

```
const ponto = { x: 2, y: 5 }; // objeto com duas propriedades
```

Assim, as propriedades são as características do objeto:

```
print(ponto.x); // mostra 2
print(ponto.y); // mostra 5
```

“Propriedades de objetos são basicamente as mesmas que variáveis normais em JavaScript, exceto pelo fato de estarem ligadas a objetos”. Propriedades podem ser acessadas por colchetes, [...].

Assim, um objeto pode ser encarado como um “pacote” contendo nome, conteúdo e ferramentas próprias. Virtualmente, tudo em *JavaScript* — arrays, funções, strings, números — é, de alguma forma, um objeto com suas propriedades e métodos. Segue um exemplo:

```
const carro = {
  marca: "Toyota",      // propriedade (dado)
  ano: 2022,            // propriedade
  ligar: function() {   // método (ação)
    console.log("O carro está ligado!");
  }
};

carro.ligar();           // chama o método
console.log(carro.marca); // acessa a propriedade
```

Reforçando (não custa nada...), um objeto *JavaScript* envolve uma coleção de propriedades, onde cada propriedade é definida por um par chave-valor, permitindo uma representação ordenada de elementos. Objetos podem conter diferentes tipos de dados como valores, como números, strings, booleanos, arrays, outras funções e até mesmo outros objetos. Objetos criados são facilmente acessados por seus atributos. Seguem exemplos para testes.

Criação

```
const pessoa = {
  nome: 'João',
  idade: 30,
  cidade: 'São Paulo'
};
```

Acesso

```
print(pessoa.nome);
print(pessoa['idade']);
```

Atributos

```
pessoa.profissao = 'Engenheiro';
print(pessoa);
```

Para um exemplo fechado de objetos:

```
let pessoa = {
  nome: "João",
  idade: 30,
  profissao: "Desenvolvedor",
```



```
hobbies: ["leitura", "música", "jogos"],
endereco: {
  rua: "Rua Principal, 123",
  cidade: "São Paulo",
},
saudacao: function () {
  print("Olá, meu nome é " + this.nome);
},
};

print(pessoa.nome); // Saída: "João"
print(pessoa.hobbies[0]); // Saída: "leitura"
pessoa.saudacao(); // Saída: "Olá, meu nome é João"
```

E por aí vai !! No próximo capítulo essa seção irá explorar um pouco sobre variáveis e funções.

8. Corpo Humano: Integração, Controle e Vida em Funcionamento



O MUNDO TE CHAMA

Você está vivo ! E por óbvio, já que está lendo esse trecho. Mas já parou pra pensar na complexidade que agora atua em seu corpo sem que você perceba ? Seu coração está batendo, seu cérebro está processando informações, seu pulmão está trocando gases, seu corpo está regulando temperatura e seu sangue está transportando uma quantidade imensa de moléculas. E tudo isso acontece ao mesmo tempo, em perfeita coordenação. Independentemente de qual órgão esteja se pronunciando agora, o corpo humano não é um conjunto de partes separadas, mas um sistema integrado por sistemas.

O sistema nervoso atua controlando rapidamente tudo, enquanto o sistema endócrino possui controle bem mais lento. O sistema cardiovascular permite o transporte de oxigênio e nutrientes, ao passo que o sistema respiratório troca gases com o ambiente. Por sua vez, o sistema digestório troca matéria e energia, enquanto o sistema excretor garante o equilíbrio interno do organismo. O sistema muscular atua no movimento do corpo humano e o sistema imunológico, na sua defesa.

Esse capítulo busca fazer um estudo diferente desses sistemas, integrando-os às funções do corpo humano de forma conjunta. Assim como um carro não anda somente juntando motor, rodas e volante, também não há rim, cérebro ou fígado que opere sozinho e sem interação com o restante do corpo (...ainda).

Depois de explorar, você consegue:

- Entender como sistemas do corpo se comunicam;
- Interpretar respostas fisiológicas dos diversos sistemas;
- Relacionar energia, transporte e controle no corpo humano;
- Explorar cenários reais com modelos computacionais interativos.



MEXA ANTES DE ENTENDER

O corpo atua como um sistema de controle em tempo real. Durante seu funcionamento, as respostas rápidas do sistema nervoso e as duradouras do sistema endócrino (hormônios) se equilibram na comunicação biológica entre seus diversos tecidos vivos.

8.0.1 Sistema nervoso e sistema endócrino

O JSPlotly que abre esse capítulo na [Figura 8.1](#) busca um panorama sobre

essa integração entre o impulso rápido e o sinal prolongado. Experimente-o antes de tentar abordar os conceitos.

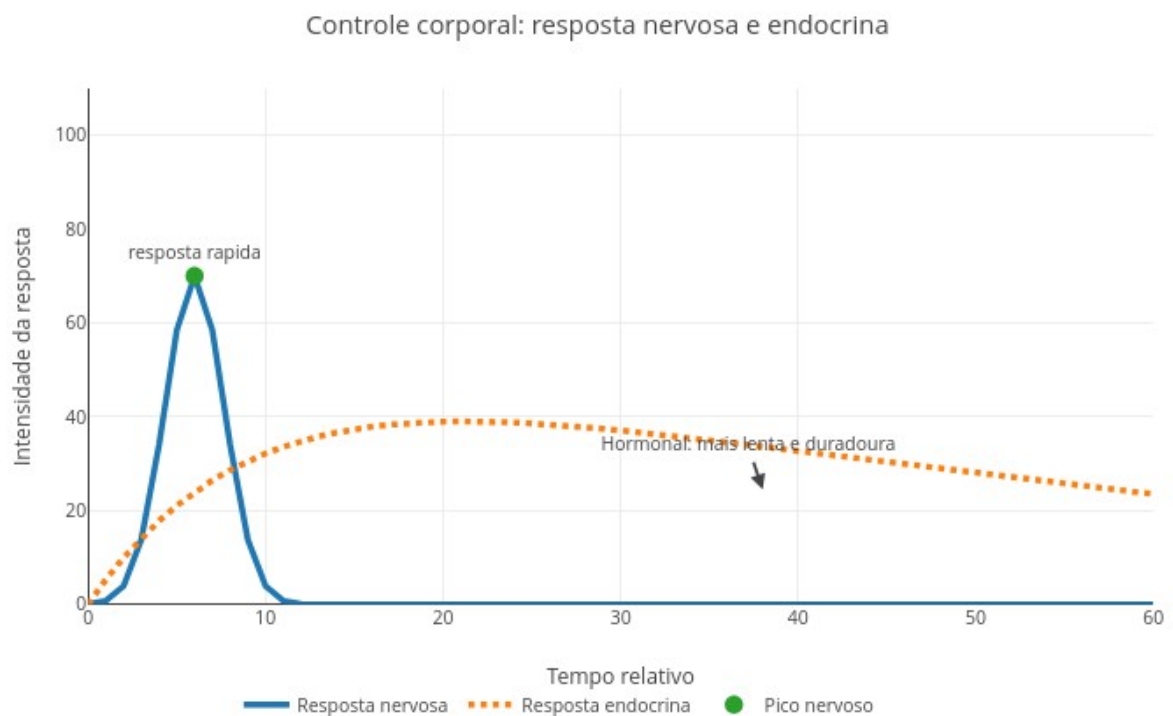


Figura 8.1: O corpo usa diferentes sinais para lidar com diferentes necessidades. Clique com o botão direito do mouse neste [LINK](#) para carregar o app.

O script compara o controle do sistema nervoso com o sistema endócrino do corpo humano. A resposta nervosa aparece como um pico rápido e breve. A resposta endócrina aparece como uma curva mais lenta e prolongada.

Agite antes de usar

O app gera um gráfico em que a curva “nervosa” mostra uma resposta rápida, pontual e intensa. Já a curva “endócrina” mostra uma resposta mais gradual e sustentada.

Execute o script com:

```
const tipo = "ambos";
const estimulo = 70;
```

Depois teste:

```
const tipo = "nervoso";
const tipo = "endocrino";
```

E altere a intensidade:

```
const estimulo = 30;
const estimulo = 100;
```

Observe como as duas respostas mudam.

Por que o corpo precisa de respostas rápidas e respostas lentas ?

Algumas situações exigem reação imediata, como retirar a mão de algo quente, ajustar a postura, responder a um susto ou contrair um músculo rapidamente. Outras exigem controle prolongado, como é comum no metabolismo, na regulação do crescimento, na manutenção sanguínea de glicose, no ajuste do equilíbrio hídrico, e no preparo para estresse prolongado. Esse equilíbrio corporal é obtido nas “conversas” entre o sistema nervoso e o sistema endócrino. Segue um tutorial para uso rápido do script.

1. Rode o script com tipo = "ambos";
2. Observe a resposta nervosa;
3. Observe a resposta endócrina;
4. Compare a velocidade das duas curvas;
5. Troque para tipo = "nervoso";
6. Veja apenas a resposta rápida;
7. Troque para tipo = "endocrino";
8. Veja apenas a resposta prolongada;
9. Aumente estímulo;
10. Observe se as duas respostas ficam mais intensas.

Veja que o corpo está respondendo rápido e sustentando a resposta, tudo ao mesmo tempo. Experimente também alguns contextos para sedimentar essa ideia:

1. *Resposta nervosa isolada.* Por que uma resposta rápida também precisa acabar rapidamente ?

```
const tipo = "nervoso";
```

2. *Resposta endócrina isolada.* Por que sinais hormonais costumam durar mais tempo ?

```
const tipo = "endocrino";
```

3. *Comparação simultânea.* Em que momentos as duas respostas são mais diferentes ?

```
const tipo = "ambos";
```

4. *Estímulo fraco.* Um estímulo pequeno gera controle suficiente ?

```
const estimulo = 25;
```

5. *Estímulo intenso.* O corpo responde apenas aumentando a intensidade ou também mudando o tempo da resposta ?

```
const estimulo = 100;
```

Depois de testar como acima, perceba que o sistema nervoso funciona como um alerta imediato, enquanto que o sistema endócrino funciona como uma regulação mais prolongada. Ou seja, a vida em funcionamento exige velocidade e duração.

Por dentro do script

O script começa com dois controles principais:

```
const tipo = "ambos";
const estimulo = 70;
```

A variável `tipo` escolhe quais curvas aparecem e a variável `estimulo` controla a intensidade da resposta. As respostas são calculadas em um modelo bastante simplificado, mas o suficiente para que se perceba as diferentes atuações dos dois sistemas.

A resposta nervosa é calculada como um pico rápido:

```
const n = estimulo * Math.exp(-0.18 * Math.pow(i - 6, 2));
```

Ela sobe e cai em pouco tempo. Já a resposta endócrina é mais lenta:

```
const e = estimulo * (1 - Math.exp(-0.08 * i)) * Math.exp(-0.018 * i);
```

Ela cresce gradualmente e demora mais para desaparecer. Embora os modelos matemáticos envolvidos constituam uma simplificação extrema do que ocorre no mundo real, o gráfico permite a comparação entre os dois tipos de controle fisiológico. Quando você leva um susto, corre, sente fome, regula temperatura ou mantém glicose no sangue, seu corpo combina sinais rápidos e lentos. O corpo humano depende de integração.

8.0.2 Sistema circulatório

Nenhum órgão funciona sozinho no corpo. Nervos, hormônios, músculos, glândulas e órgãos conversam o tempo todo para manter o corpo funcionando. Para que isso ocorra, é necessário também que haja um sistema de distribuição para as substâncias que vão permitir essa comunicação, o que se dá pelo sistema circulatório. Assim, o corpo usa o tecido sanguíneo como um meio de transporte para as substâncias, decidindo para onde mandar o sangue conforme a necessidade.

O JSPlotly da [Figura 8.2](#) simula a distribuição do fluxo sanguíneo para diferentes regiões do corpo. Você observa como o sangue é direcionado para o cérebro, músculos, pele, sistema digestório, rins e coração.

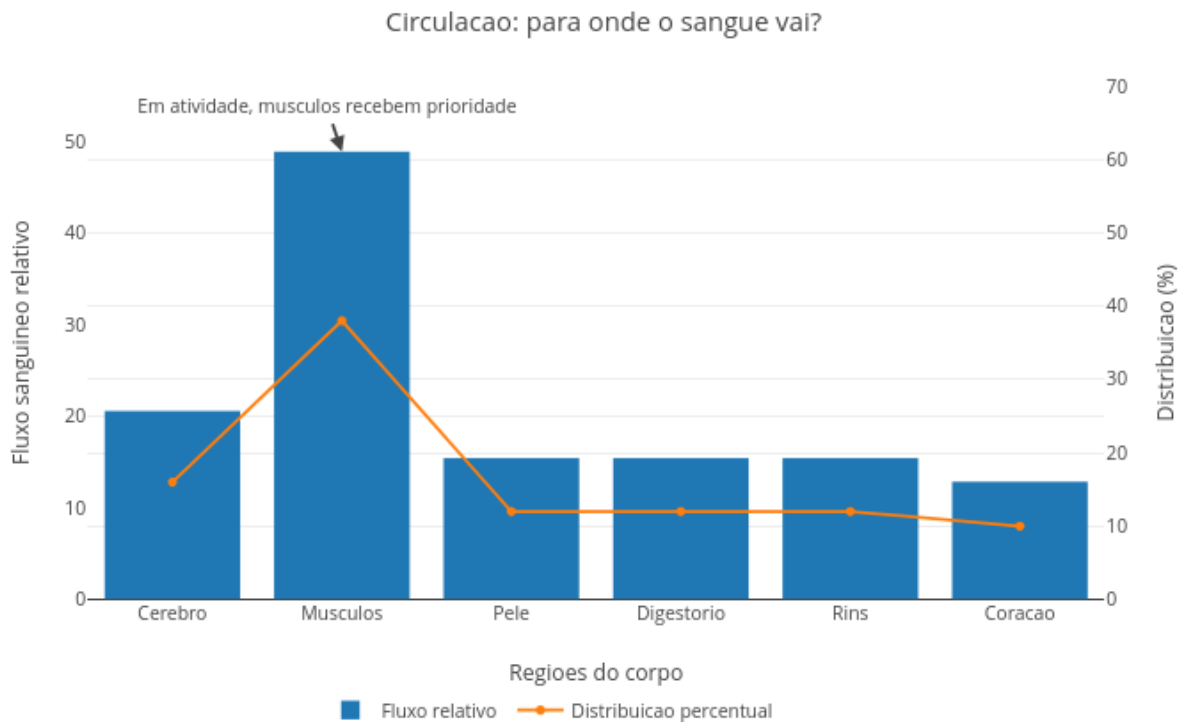


Figura 8.2: O corpo distribui sangue conforme a situação. Clique com o botão direito do mouse neste [LINK](#) para carregar o app.

Agite antes de usar

As barras do gráfico mostram o fluxo relativo para cada região, enquanto a linha mostra a distribuição percentual.

Execute o script com:

```
const atividade = "moderada";
const frequencia = 90;
const resistencia = 1.0;
```

Depois teste:

```
const atividade = "repouso";
const atividade = "intensa";
```

E altere:

```
const frequencia = 140;
const resistencia = 1.8;
```

Observe como o fluxo muda.

Como o corpo decide quais órgãos devem receber mais sangue ?

Em repouso, órgãos do sistema digestório e rins recebem grande parte do fluxo sanguíneo. Durante atividade intensa, por outro lado, os músculos passam a receber prioridade. Isso ocorre porque o sangue transporta oxigênio, nutrientes, hormônios, calor e resíduos metabólicos. Assim, o coração bate e impulsiona o sangue estrategicamente para distribuir

oxigênio e nutrientes para os tecidos. Segue um modo de uso rápido para o *app*.

1. Rode o script em atividade "moderada";
2. Observe quais órgãos recebem mais fluxo;
3. Troque para "repouso";
4. Veja o aumento relativo para digestório e rins;
5. Troque para "intensa";
6. Observe a prioridade muscular;
7. Aumente frequência;
8. Veja o fluxo relativo subir;
9. Aumente resistencia;
10. Observe o efeito contrário sobre o débito.

“Brincando” de comandar a circulação como acima, você deve perceber que o corpo aumenta e redistribui o fluxo total de sangue ao mesmo tempo. Seguem alguns cenários para você “circular”.

1. *Repouso*. Por que digestório e rins recebem mais fluxo quando o corpo está em repouso ?

```
const atividade = "repouso";
const frequencia = 60;
```

2. *Atividade moderada*. Como o corpo começa a priorizar os músculos ?

```
const atividade = "moderada";
const frequencia = 90;
```

3. *Atividade intensa*. Por que os músculos passam a dominar a distribuição do fluxo ?

```
const atividade = "intensa";
const frequencia = 150;
```

4. *Alta resistência*. Por que maior resistência dificulta o fluxo sanguíneo ?

```
const resistencia = 2.0;
```

5. *Baixa resistência*. Como vasos mais dilatados favorecem maior fluxo ?

```
const resistencia = 0.6;
```

Observe que, quando a atividade aumenta, o padrão muda, com os músculos recebendo mais sangue, e o sistema digestório e rins recebendo menos. Quando você corre, estuda, dorme, come ou sente frio, seu sistema circulatório muda sua distribuição. O coração bombeia, os vasos regulam e os órgãos disputam prioridade conforme a necessidade. O corpo humano funciona porque consegue ajustar fluxo, pressão e entrega de recursos continuamente. Em outras palavras, viver é distribuir energia, oxigênio e matéria no momento certo.

Por dentro do script

Outra vez, o modelo matemático inserido no script é apenas didático, estando a anos-luz do que realmente ocorre nos tecidos do corpo.

O script começa escolhendo o nível de atividade:

```
const atividade = "moderada";
```

Depois define a frequência cardíaca:

```
const frequencia = 90;
```

E a resistência vascular:

```
const resistencia = 1.0;
```

O débito relativo é calculado por:

```
const debito = (frequencia / 70) * fatorAtividade / resistencia;
```

Depois o script escolhe a distribuição percentual conforme a atividade.

Por exemplo, em atividade intensa:

```
distribuicao = [14, 58, 16, 4, 4, 4];
```

Isso significa que os músculos recebem uma fração muito maior do fluxo. Por fim, o fluxo relativo é calculado por:

```
return v * debito;
```

Dessa forma, o gráfico mostra tanto o total relativo quanto a distribuição entre órgãos.



Os JSPlotly das [Figura 8.1](#) e [Figura 8.2](#) acima mostram que o corpo humano funciona como um sistema de informação. O corpo detecta sinais por receptores sensoriais, processa esses sinais no cérebro juntamente com os hormônios, e responde a esses sinais com os músculos e glândulas. Ou seja:

sensores → processamento → resposta

Metabolismo

As respostas corporais integradas pela circulação sanguínea permitem ajustes variados em função do “bombeamento” de oxigênio e de nutrientes. Isso possibilita ajustar as reações bioquímicas junto à respiração e ao metabolismo celular, permitindo que demandas energéticas diferentes exijam esforços do corpo também diferentes. O JSPlotly que segue busca avançar um pouquinho nessa ideia, a de que o corpo tenta sustentar energia enquanto o esforço cobra sua conta.

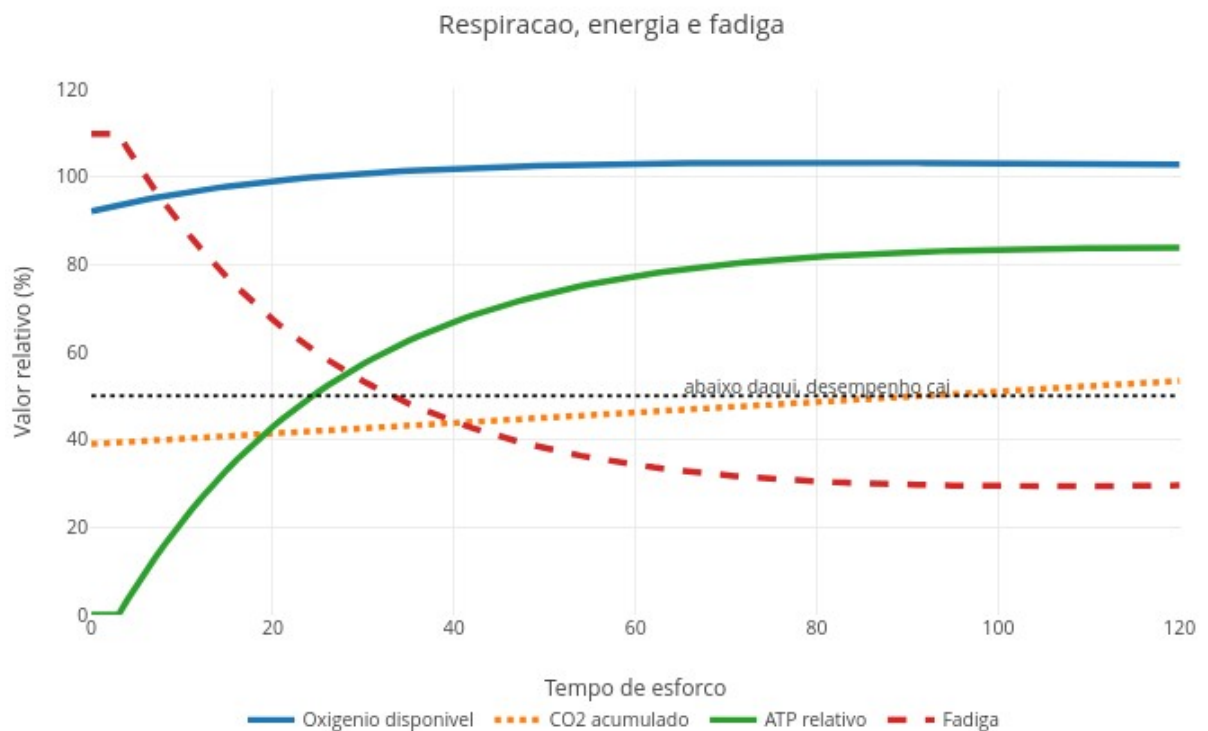


Figura 8.3: O desempenho corporal depende do equilíbrio entre demanda de energia e oferta de oxigênio. Clique com o botão direito neste [LINK](#) e abra em nova aba.

Respiração, energia e fadiga estão interligados quando o esforço físico encontra seu limite. O *app* da [Figura 8.3](#) mostra como respiração, oxigênio, gás carbônico, produção de ATP e fadiga se relacionam durante um esforço.

ATP, ou *adenosina trifosfato*, é a “moeda” de troca energética dos seres vivos. No corpo humano, são produzidos em torno de 30 a 40 kg diariamente !! Achou estranho por que você pesa quase isso ? Mas é assim mesmo. Ocorre que o corpo produz essa quantidade toda, mas

também gasta tudo ao longo do dia, fazendo com que seu peso não mude por conta desse ciclo do ATP.

Agite antes de usar

No aplicativo você acompanha quatro curvas:

- oxigênio disponível;
- CO_2 acumulado;
- ATP relativo;
- fadiga.

A linha horizontal marca uma região crítica. Quando energia ou equilíbrio funcional caem abaixo desse limite, o desempenho tende a piorar. O ATP relativo mostra a capacidade de sustentar trabalho. A fadiga mostra o acúmulo de dificuldade fisiológica. E o oxigênio e gás carbônico mostram o papel da respiração.

Execute o script com:

```
const intensidade = 70;
const oxigenio = 100;
const freqResp = 22;
```

Depois teste:

```
const intensidade = 95;
const freqResp = 35;
```

E também:

```
const oxigenio = 60;
```

Observe como ATP e fadiga mudam.

Por que o corpo se cansa quando a demanda supera a capacidade de sustentação ?

Durante o esforço, os músculos precisam de ATP. Para produzir ATP de forma eficiente, o corpo depende de O_2 (inspiração), o qual é trazido pela respiração, convertido em CO_2 nas células, e removido depois pela mesma respiração (expiração). Quando a demanda cresce demais ou a oferta não acompanha, o ATP relativo cai e o CO_2 pode acumular, aumentando a fadiga e diminuindo o desempenho. O cansaço, nesse modelo, aparece como sinal de desequilíbrio.

Nesse simulação didática pra lá de simplificada, o `nivelO2` representa oxigênio disponível, enquanto o `nivelCO2` representa acúmulo de gás carbônico. Complementando, a variável `energia` representa ATP relativo e a variável `cansaco` representa fadiga. Assim, o gráfico mostra o corpo tentando equilibrar respiração, metabolismo e desempenho. Segue um tutorial para você experimentar.

1. Rode o script com os valores originais;
2. Observe o ATP relativo;
3. Observe a fadiga;

4. Compare oxigênio e CO₂;
5. Aumente intensidade;
6. Veja se a fadiga cresce mais rápido;
7. Aumente freqResp;
8. Observe se melhora a sustentação energética;
9. Reduza oxigenio;
10. Veja como a oferta limitada afeta o sistema.

Depois de “brincar” um pouco com o *app*, você perceberá que o limite do desempenho vem de um equilíbrio entre a intensidade do esforço e a capacidade do corpo em fornecer oxigênio. Vão abaixo alguns cenários para você experimentar, também.

1. *Esforço moderado*. O corpo consegue sustentar ATP por mais tempo ?

```
const intensidade = 50;
const oxigenio = 100;
const freqResp = 18;
```

2. *Esforço intenso*. Por que a fadiga cresce rapidamente ?

```
const intensidade = 95;
const freqResp = 25;
```

3. *Respiração aumentada*. Maior frequência respiratória compensa completamente a demanda ?

```
const intensidade = 80;
const freqResp = 40;
```

4. *Baixo oxigênio*. Como menor disponibilidade de O₂ afeta ATP e fadiga ?

```
const oxigenio = 55;
```

5. *Alta intensidade com pouco oxigênio*. Por que esse cenário leva rapidamente a queda de desempenho ?

```
const intensidade = 90;
const oxigenio = 50;
```

Quando você corre, sobe escadas, pratica esporte ou sente falta de ar, seu corpo está tentando equilibrar demanda energética e oferta de oxigênio. Pulmões, sangue, coração, músculos e mitocôndrias trabalham juntos. Se uma parte falha, todo o desempenho muda. Portanto, fadiga não é “fraqueza”, mas uma resposta integrada do organismo diante de limites metabólicos.

Por dentro do script

O script começa com três controles principais:

```
const intensidade = 70;
const oxigenio = 100;
const freqResp = 22;
```

A demanda aumenta com intensidade e tempo:

```
const demanda = 0.45 * intensidade + 0.8 * i / 10;
```

A oferta depende do oxigênio disponível e da frequência respiratória:

```
const oferta = oxigenio * (freqResp / 20) * (1 - Math.exp(-0.04 * i));
```

Depois o script calcula:

```
nivelO2  
nivelCO2  
energia  
cansaco
```

VOLTA PRO MUNDO



Depois de praticar nos JSPlotly do capítulo, você já consegue responder a alguns “por quês fisiológicos”. Por exemplo, por que seu coração acelera, por que você suar, e por que você cansa. Essa resposta integrada do corpo para manter as coisas em ordem é a homeostase, a qual permite que o corpo equilibre (ou não) tudo ao mesmo tempo.

Que tal então um JSPlotly para ver essa tal de *homeostase* trabalhando a seu favor ?

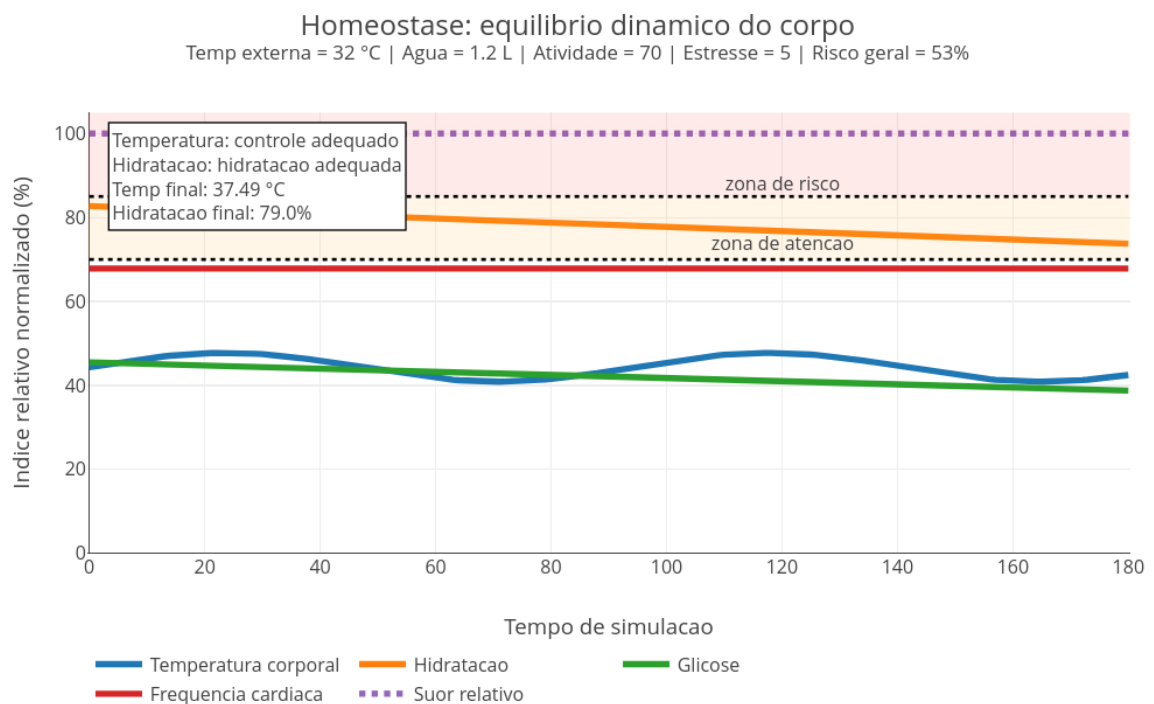


Figura 8.4: O corpo tenta manter equilíbrio, embora pressionado por múltiplas variáveis ao mesmo tempo. Clique com o botão direito neste [LINK](#) e abra em nova aba.

O aplicativo da [Figura 8.4](#) simula o funcionamento integrado do corpo diante de múltiplos fatores, como temperatura externa, hidratação,

atividade física e estresse. No gráfico você acompanha as curvas de temperatura corporal, hidratação, glicose, frequência cardíaca e suor.

Agite antes de usar

As zonas coloridas do gráfico indicam região de atenção e região de risco. Quando as variáveis entram nessas zonas, o equilíbrio está sendo perdido, o corpo está sob estresse e o sistema pode falhar.

Execute o script com:

```
const tempExterna = 32;
const agua = 1.2;
const atividade = 70;
const estresse = 5;
```

Depois teste:

```
const tempExterna = 40;
const agua = 0.5;
```

E também:

```
const atividade = 90;
const estresse = 8;
```

Observe quando as curvas entram nas zonas de atenção e risco. Quando tudo está equilibrado a temperatura se mantém estável, a hidratação sustenta regulação, a energia é suficiente e o sistema então funciona em sua plenitude. Contudo, quando múltiplos fatores interferem, a temperatura pode subir, a água pode se tornar insuficiente, o coração acelera e o sistema entra em tensão. Isso mostra que, contrariamente ao ditado popular que diz que “o corpo é uma máquina perfeita”, a homeostase tem seus limites.

Teste o pequeno tutorial abaixo para uso do aplicativo.

1. Rode o script com os valores iniciais;
2. Observe todas as curvas juntas;
3. Identifique quais variáveis mudam primeiro;
4. Aumente tempExterna;
5. Veja o impacto na temperatura e no suor;
6. Reduza água;
7. Observe a perda de regulação;
8. Aumente atividade;
9. Compare com o aumento da frequência cardíaca;
10. Combine fatores.

Note que o que mais pesa no desequilíbrio do corpo é uma combinação de calor, esforço e falta de água. Seguem alguns cenários para você se “hidratar” com o aplicativo.

1. *Condição equilibrada.* Por que o sistema permanece estável ?

```
const tempExterna = 25;
const agua = 2.0;
```

```
const atividade = 40;  
const estresse = 2;
```

2. *Calor elevado.* O corpo consegue compensar apenas com suor ?

```
const tempExterna = 42;
```

3. *Desidratação.* Por que a regulação térmica piora ?

```
const agua = 0.4;
```

4. *Exercício intenso.* Quais variáveis respondem mais rapidamente ?

```
const atividade = 95;
```

5. *Estresse elevado.* Por que o corpo responde mesmo sem esforço físico ?

```
const estresse = 9;
```

6. *Tudo ao mesmo tempo.* Por que o sistema entra rapidamente em zona de risco ?

```
const tempExterna = 38;  
const agua = 0.6;  
const atividade = 85;  
const estresse = 7;
```

Por dentro do script

O modelo conceitual começa com quatro variáveis principais que representam condições externas e internas:

```
const tempExterna  
const agua  
const atividade  
const estresse
```

Essas variáveis funcionam como forças que empurram o sistema. A cada instante de tempo (for (let i = 0; i <= 180; i++)), o modelo calcula como o corpo responde a essas forças. Primeiro, é estimado o quanto o corpo está sendo “pressionado”:

```
const calor = ...
```

Esse termo combina temperatura externa, intensidade da atividade e nível de estresse, representando, portanto, a demanda fisiológica total. Depois, o script calcula a capacidade de compensação:

```
const regulacao = ...
```

Aqui entra a água. Quanto maior a hidratação, maior a capacidade do corpo de dissipar calor e manter o equilíbrio. A temperatura corporal então surge do conflito entre esses dois fatores:

```
const tcorpo = ...
```

Com calor aumentado e pouca regulação, a temperatura sobe. Com menos calor e boa regulação, a temperatura se mantém.

Em seguida, o modelo atualiza outras variáveis importantes:

- hid: nível de hidratação ao longo do tempo;
- gli: glicose, influenciada por atividade e estresse;
- heart: frequência cardíaca;
- sw: suor, que ajuda a resfriar, mas consome água.

Cada variável depende de várias outras, formando um sistema acoplado. Por fim, todas são normalizadas:

```
norm(v, min, max)
```

Isso transforma valores reais em uma escala comum (0-100), permitindo comparar tudo no mesmo gráfico. A “alma” do script está em equilibrar a demanda (calor, esforço, estresse) com a regulação (água e respostas fisiológicas). E quando esse equilíbrio falha, o gráfico mostra.

Em termos de programação, o script segue uma lógica típica de modelos dinâmicos:

1. Definir condições iniciais;
2. Iterar no tempo;
3. Atualizar variáveis a cada passo;
4. Armazenar resultados;
5. Visualizar o comportamento do sistema.

E SE...



“E se” a gente tivesse um painel do corpo humano mostrando os sistemas em funcionamento ?

Resposta: a gente aguardaria mais um pouquinho essa seção de “devaneios científicos” só pra nos deliciarmos com esse painel. Então segue o *dashboard* na [Figura 8.5](#) !

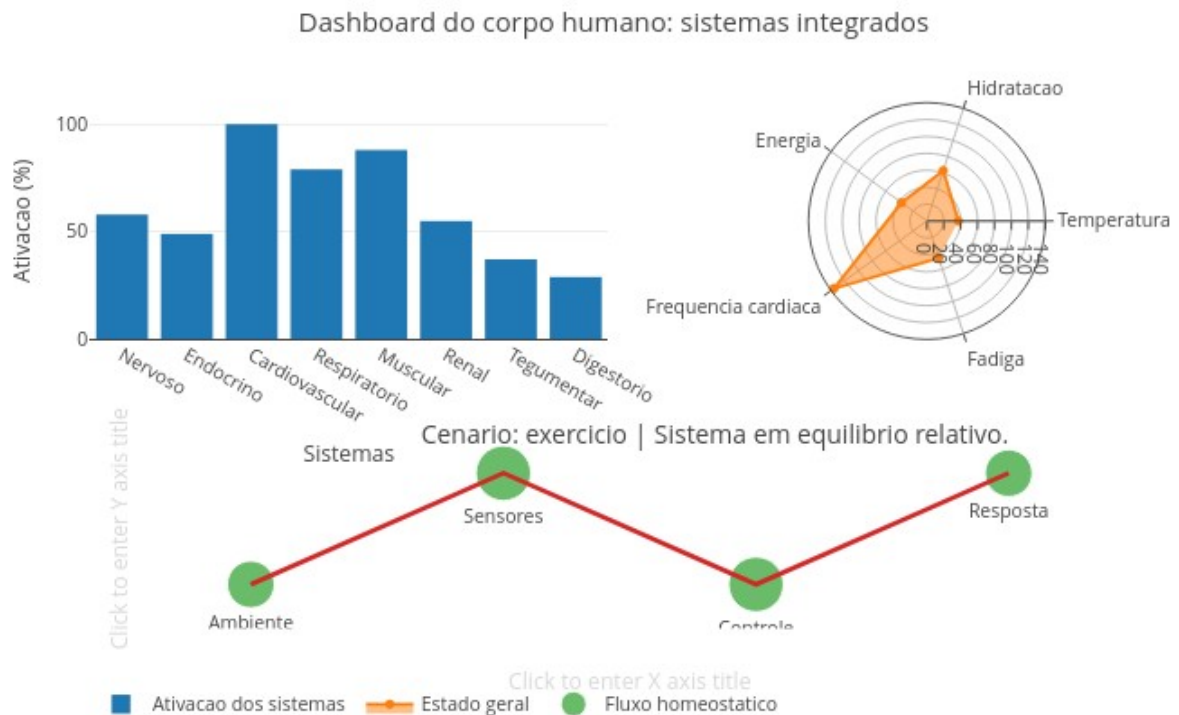


Figura 8.5: O corpo humano funciona como um sistema de sistemas, integrando suas partes e adaptando suas estratégias conforme o cenário. Clique no [LINK](#) para acessar o app.

Agora o *app* reúne vários sistemas do corpo humano em um único painel. Nele você pode acompanhar a ativação dos sistemas corporais, o estado geral do organismo, e o fluxo homeostático, além de uma mensagem de alerta fisiológico.

Agite antes de usar

O *app* permite uma comparação entre os diversos sistemas fisiológicos:

```
"Nervoso",
"Endócrino",
"Cardiovascular",
"Respiratório",
"Muscular",
"Renal",
"Tegumentar",
"Digestório"
```

Após rodar o *app*, veja que o gráfico de barras mostra quais sistemas corporais estão mais ativados. O gráfico radar mostra o estado geral do

organismo. O diagrama inferior resume a lógica da homeostase. E a mensagem central indica o principal alerta do cenário.

Ambiente → Sensores → Controle → Resposta

Execute o script com:

```
const cenário = "exercício";
```

Depois teste:

```
const cenário = "repouso";
const cenário = "calor";
const cenário = "altitude";
const cenário = "estresse";
```

Observe como o painel muda em cada situação. E veja quais sistemas são mais ativados e qual alerta aparece.

Como o corpo integra vários sistemas para responder a diferentes desafios ?

Em repouso, o organismo tende ao equilíbrio. No exercício, músculos, coração e respiração ganham prioridade. No calor, pele, suor e hidratação tornam-se essenciais. Na altitude, a disponibilidade de oxigênio altera todo o sistema. No estresse, os sistemas nervoso e endócrino passam a comandar grande parte da resposta. Em outras palavras, a integração entre os diversos sistemas fisiológicos permite ao corpo adaptar-se a vários cenários. Segue um modo rápido de uso para o *app*.

1. Rode o script com cenário = "exercício";
2. Observe o gráfico de barras;
3. Veja quais sistemas estão mais ativados;
4. Observe o radar de estado geral;
5. Leia a mensagem central;
6. Troque para cenário = "repouso";
7. Compare o equilíbrio dos sistemas;
8. Troque para cenário = "calor";
9. Observe hidratação, temperatura e pele;
10. Troque para cenário = "altitude";
11. Observe o impacto do oxigênio;
12. Troque para cenário = "estresse";
13. Compare nervoso, endócrino e frequência cardíaca.

Perceba que sistemas diferentes dominam a resposta em cada situação. Seguem alguns contextos para você “integrar” ao seu aprendizado.

1. *Repouso*. Como o corpo se comporta quando há baixa demanda ?

```
const cenário = "repouso";
```

2. *Exercício*. Por que músculos, coração e respiração ganham prioridade ?

```
const cenário = "exercício";
```

3. *Calor*. Por que temperatura e hidratação se tornam pontos críticos ?

```
const cenario = "calor";
```

4. *Altitude*. Como a queda de oxigênio altera energia, fadiga e respiração ?

```
const cenario = "altitude";
```

5. *Estresse*. Por que o corpo reage mesmo sem grande esforço físico ?

```
const cenario = "estresse";
```

Quando você corre, sente calor, sobe um morro, passa por estresse ou simplesmente descansa, seu corpo reorganiza prioridades. Nenhum sistema trabalha sozinho, mas um acaba complementando o outro. O sistema nervoso detecta, o endócrino regula, o cardiovascular distribui, o respiratório fornece oxigênio para produção de energia, o muscular consome energia, o renal conserva ou elimina, a pele protege e regula temperatura, e o digestório participa do fornecimento de matéria e energia. Em síntese, o corpo responde a cada cenário como uma rede interconectada junto ao papel fisiológico dos diversos órgãos.

Por dentro do script

O script começa escolhendo um cenário:

```
const cenario = "exercicio";
```

Cada cenário altera variáveis fisiológicas básicas:

```
atividade
tempExterna
oxigenio
agua
estresse
```

Essas variáveis funcionam como condições do ambiente e do corpo. Depois, o script calcula a ativação dos sistemas:

```
const ativacao = [...]
```

Cada sistema responde de forma diferente.

Por exemplo, o cardiovascular depende bastante de atividade e estresse:

```
40 + atividade * 0.7 + estresse * 2
```

O respiratório responde à atividade e à falta de oxigênio:

```
35 + atividade * 0.55 + (100 - oxigenio) * 0.4
```

O tegumentar responde ao calor externo:

```
30 + Math.max(0, tempExterna - 25) * 2.4
```

Depois o script calcula indicadores gerais:

```
temperatura
hidratacao
energia
```

fc
fadiga

Esses indicadores alimentam o gráfico radar. Por fim, o script gera uma mensagem de alerta:

```
mensagem = "Alerta..."
```

A “alma” do script é transformar uma situação do corpo em três leituras simultâneas:

1. quais sistemas foram ativados;
2. como está o estado geral;
3. que tipo de alerta fisiológico aparece.

É uma síntese visual de todo o capítulo.

Tá legal, mas e os “E se...” ?

“E se...” tivesse um E se mais integrador e com todos os Scripts do capítulo ?

- E se você não estivesse olhando para gráficos...mas para um sistema vivo tentando sobreviver ?
- E se cada script desse capítulo fosse apenas uma janela, e agora você pudesse ver tudo ao mesmo tempo ?
- E se, ao iniciar uma corrida em um dia quente:
 - o sistema nervoso disparasse (Script no. 1),
 - o sangue fosse redirecionado para os músculos (Script no. 2),
 - a respiração tentasse sustentar ATP (Script no. 3),
 - a homeostase lutasse contra o calor e a desidratação (Script no. 4),
 - e o *dashboard* mostrasse tudo isso acontecendo simultaneamente (Script no. 5)?

Então você perceberia que não existe “um” sistema dominante, mas uma *coordenação emergente*.

- E se o oxigênio diminuísse, como em grandes altitudes ?

O Script no. 3 mostraria queda na produção de ATP; O Script no. 2 mostraria maior esforço respiratório; O Script no. 5 mostraria aumento da fadiga e reorganização sistêmica.

E o corpo responderia como pode: mais respiração, mais esforço, mais custo.

- E se a água acabasse ?

O Script no. 4 mostraria perda de regulação térmica; O Script no. 1 manteria resposta nervosa, mas com eficiência reduzida; O Script no. 5 indicaria alerta sistêmico.

A homeostase começaria a falhar não por um fator isolado, mas por uma cadeia de dependências.

- E se o estresse fosse o fator dominante ?

Sem movimento, sem calor extremo. Mesmo assim, a frequência cardíaca sobe, a glicose aumenta e os sistemas de controle são ativados. Nesse caso, o Script no. 5 mostraria que o corpo reage ao que percebe, não apenas ao que acontece fisicamente.

- E se todos os fatores se combinassem, calor, esforço, estresse, pouca água e baixo oxigênio ?

Aí não seria mais uma curva, mas um sistema sob pressão máxima. Os scripts deixariam de ser ferramentas isoladas e passariam a fazer parte de um único modelo no qual o corpo está tentando manter coerência interna em um mundo que muda para o colapso.

- E se você fosse capaz de prever isso, de olhar para o gráfico e antecipar:
- quando o desempenho cai;
- quando a fadiga domina;
- quando o risco aparece.

Então você estaria pensando como um sistema vivo, mais do que interpretando dados.

- E se o corpo fosse como um ecossistema, ou uma cidade, ou uma rede elétrica, ou ainda um planeta tentando manter o equilíbrio ?

Os mesmos princípios aparecem:

fluxo → regulação → adaptação → limite

- E se no fim das contas, a ideia geral fosse viver equilibrando continuamente forças que nunca param de mudar, e que saúde não é sinônimo de ausência de variação, mas a capacidade de responder a ela ?

MESMO PADRÃO, OUTROS MUNDOS



Nos limites do corpo humano, vemos princípios que se repetem em outros sistemas complexos. Um ecossistema, por exemplo, também precisa equilibrar fluxo de energia, disponibilidade de recursos e resposta a perturbações externas. Quando há excesso de calor, escassez de água ou aumento de estresse ambiental, espécies e populações reorganizam suas funções — assim como órgãos e sistemas fazem no corpo. A homeostase biológica encontra paralelo direto na estabilidade ecológica, pois ambas são dinâmicas, nunca fixas, e sempre negociadas.

Em escala tecnológica e social, cidades e redes elétricas operam sob uma lógica semelhante. Sensores detectam mudanças, centros de controle processam informações e sistemas de resposta ajustam fluxos de energia, de pessoas, e de dados. Quando a demanda cresce além da capacidade de regulação, surgem falhas, sobrecargas ou colapsos. Assim como no corpo humano, não é um único componente que determina o funcionamento, mas a integração entre todos os componentes. O que chamamos de “equilíbrio” é, na verdade, um processo contínuo de adaptação.

Levando a ideia ainda mais longe, é possível enxergar o próprio planeta — ou até sistemas cósmicos — sob essa perspectiva. Clima, ciclos biogeoquímicos e vida interagem como partes de um grande sistema autorregulado, sujeito a limites e transições críticas. O mesmo raciocínio vale para fluxo, regulação, resposta e limite. Ao compreender o corpo humano como um sistema integrado, você passa a reconhecer esse padrão em diferentes escalas do universo. No fundo, o mundo também é uma extensão da fisiologia humana.



Separação de comandos, inserção de comentários, e boas práticas

Os comandos em *JS* são separados por *ponto e vírgula*, “;”. Já os *comentários* constituem trechos não interpretados, comumente utilizados em programação. Comentários são tremendamente úteis quando se deseja colocar um título no *script* de código, por exemplo, para explicar quem é quem nas variáveis ou determinada ação no código, e apontar uma futura melhoria, entre muitos (lista *ToDo*, ou “para fazer”). Em *JS*, os comentários são alternativamente inseridos por “//” antes de cada comando, ou por “/.../”, para comentários mais extensos.

Códigos em *JS* podem ser inseridos também junto à páginas web (HTML) com seus comandos intercalados pela notação que segue (etiquetas ou *tags*):

Uma característica interessante no *JS* é que tanto faz colocar ou não espaços entre os comandos, pois a linguagem não interpreta esses espaços. Contudo, dentro das *boas práticas em programação*, é aconselhável separar blocos de comandos por *identação* de 2 espaços, de modo a facilitar a leitura do código. Outro cuidado que deve ser apreciado diz respeito ao uso de fonte maiúscula ou minúscula. *JS* interpreta diferentemente termos em caixa alta ou baixa do teclado, então é bom prestar atenção pra não incorrer em erro na interpretação.

Declaração de variáveis

Entre os diversos tipos de variáveis, duas são frequentemente utilizadas: *const* e *let*. A variável *const* é utilizada para se atribuir um valor fixo a uma variável no código, enquanto a variável *let* é empregada quando se deseja reatribuir um valor a ela, por meio de algum cálculo ou laço iterativo. Exemplificando, quando se deseja variáveis de controle que mudam ao longo do tempo (*sliders*, passos, intervalos).

Obs: em versões antigas de *JS* é também empregado *var*, hoje em desuso.

“Botando a mão na massa”

Como esse treinamento envolve instruir o algoritmo com uma *entrada* para que sua interpretação resulte numa *saída*, aqui propomos essa última pelo comando *print*, como segue.

```
const a = -0.2;
print(a)
```

Para esse treinamento, use o *JSPlotly* personalizado do capítulo anterior. Assim, teste a atribuição da variável *const* e do comando *print* por cópia do trecho acima e cola no campo de texto **ÁREA PARA DIGITAR COMANDOS** do script. Existem outros comandos de saída para *JS*, como *console.log()* e *document.body.appendChild()*, mas não utilizáveis pelo *JSPlotly*.

Pode-se atribuir também um texto para apresentar a constante, veja:

```
const a = -0.2;
print("constante:", a)

const a = 5; // variável constante (não se altera ao longo do código)
const b = 10; // variável que pode alterar seu valor no código
print(a*b)
```

Outro exemplo, com inserção de texto antes do resultado:

```
let a = 5;
const b = 10;

let resultado = a + b;
print("Soma de a + b:", resultado);
```

const versus *let*

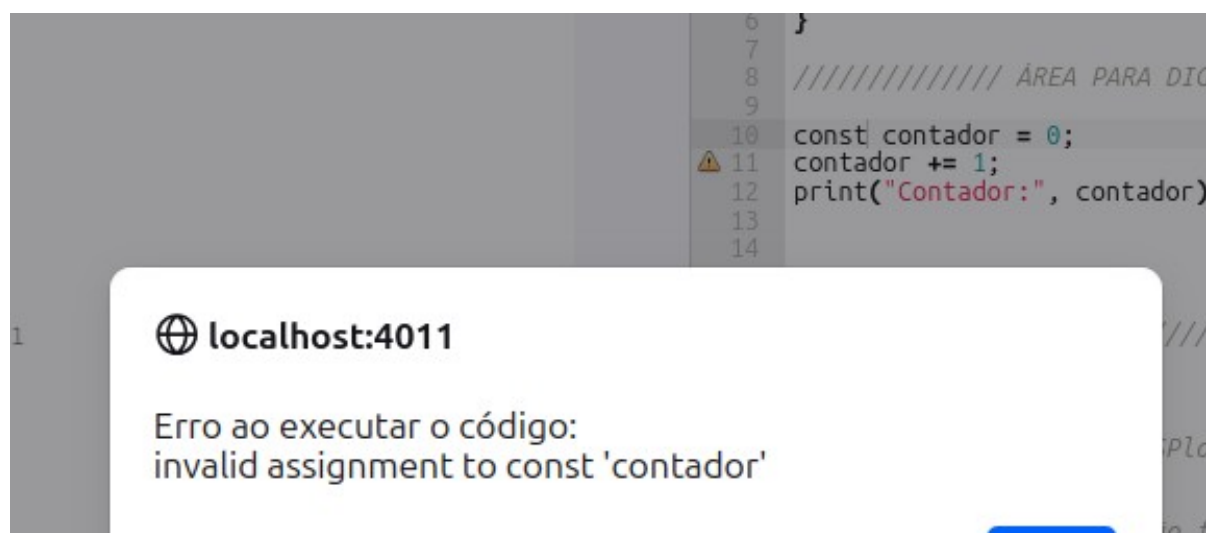
Pode parecer que *const* e *let* são distintas apenas na aparência, já que operam de modo intercambiável. Contudo, para explicitar a diferença entre os dois comandos, apague o ecrã gráfico (*clean*) e substitua a linha de código da seção anterior pela que segue na **ÁREA PARA DIGITAR COMANDOS** do script:

```
let passo = 0.5; // usuário pode alterar
let x0 = 0; // domínios dinâmicos
print(passo, x0)

// ... mais tarde:
passo = 0.2; x0 = 30;
print(passo, x0)
```

Aparentemente, o resultado sugere que na prática nada muda, sendo a diferença mais centrada na clareza e segurança do código. Mas não é bem assim. Experimente agora trocar o campo pra executar o código abaixo:

Agora substitua a variável do “contador” para *const* ao invés de *let*. O resultado incorrerá no erro abaixo:



Resumindo: para constantes, use *const* e para variável reatribuível use *let*.

Tipos de dados

Como é comum também em outras linguagens, *JavaScript* possui alguns tipos comuns para definição de dados. Para uso de *Plotly.js*, são bem significativos:

Seguem alguns exemplos práticos. Experimente trocar cada um na saída *print*:

```
// Numbers:
let comprimento = 116;
let peso = 75;

// Strings:
let color = "Yellow";
let lastName = "Johnson";

// Booleans
let x = true;
let y = false;

// Object:
const membro = {firstName:"John", lastName:"Doe"};

// Array object:
const carros = ["Saab", "Volvo", "BMW"];

// Date object:
const datas = new Date("2022-03-25");

// Saídas:
print(comprimento)
```

Constantes

Por vezes, é necessária a inserção de constantes matemáticas em equações para construção de gráficos ou para fazer cálculos. Seguem alguns exemplos. Para a saída no *JSPlotly* customizado abaixo, basta o bom e velho *print* que segue ao também bom e velho *clean*.

```
Math.PI - valor de (pi), aproximadamente 3.14159
Math.E - valor da constante de Euler, aproximadamente 2.71828
Math.LN2 - logaritmo natural de 2, aproximadamente 0.693
Math.LN10 - logaritmo natural de 10, aproximadamente 2.302
Math.LOG2E - logaritmo de e na base 2, aproximadamente 1.442
Math.LOG10E - logaritmo de e na base 10, aproximadamente 0.434
Number.MAX_VALUE - maior número positivo
Number.MIN_VALUE - menor número positivo...
```

Para constantes naturais, contudo, é necessário defini-las antecipadamente, como em:


```
// Definindo constantes físico-químicas comuns
const Avogadro = 6.02214076e23; // Número de Avogadro (mol-1)
const Boltzmann = 1.380649e-23; // Constante de Boltzmann (J/K)
const Planck = 6.62607015e-34; // Constante de Planck (J·s)
const gas = 8.31446; // Constante dos gases ideais (J/(mol·K))
const luz = 299792458; // Velocidade da luz no vácuo (m/s)
```

Funções

Funções constituem um bloco de códigos para efetuar alguma ação. Em *JavaScript* uma função possui a seguinte sintaxe:

```
function nome(parâmetro1, parâmetro2, parâmetro3) {
  // código para execução
}
```

Seguem alguns exemplos de funções para testes.

```
function soma(a) { // define o nome e os "parâmetros" da função
  return a + a // define os argumentos da função (no caso, a*a),
               // pra retorna dos valores
}
```

```
// Teste
let x = soma(7); // aplica parâmetros
print(x) // saída da função
```

Obs: O comando *return*, por vezes omitido, se apresenta útil quando se deseja que o “retorno” seja concluído somente ao final do fluxo do script, sem resultados intermediários.

```
function expoente(a, b) { // define o nome e os "parâmetros" da
  função
  return a ** b; // define os argumentos da função (no caso, a^b),
                // pra retorna dos valores
}
```

```
// Teste da função
let x = expoente(4, 3); // aplica parâmetros
print(x); // saída da função
```

```
function indexa(vetor){
  return vetor + 1
}
```

```
var vetor = [1, 2, 3, 4, 5];
print(indexa(vetor))
```

```
function Celsius(Fahrenheit) {
  return (5/9) * (Fahrenheit-32);
}
```

```
let conversao = Celsius(115);
print(conversao)
```

Funções de seta (arrow functions)

Como visto anteriormente, uma notação alternativa é a função *arrow*, que abrevia a sintaxe de funções. Segue um exemplo:

```
const multiplica = (a, b) => a * b;  
print(multiplica(4, 6));
```

Se gostou até aqui...não perca as **estruturas de controle** do capítulo final desta obra.